كلية التربية النوعية
قسم معلم الحاسب الآلي

معالجة النصوص



إعـــداد

أ.م.د/ منى عصمت عبد الحميد عوض

استاذ الحاسب الآلي المساعد

كلية التربية النوعية – جامعة المنصورة

اللهم إني أسألك فهم النبيين، و حفظ المرسلين،
و الملائكة المقربين، اللهم اجعل ألسنتنا عامرة
بذكرك، و قلوبنا بخشيتك، وأسرارنا بطاعتك،
إنك على كل شيء قدير

المحتويات

الموضوع	رقم الصفحة
الفصل الأول : أنواع الخوارزميات	٢
الفصل الثاني : الدوال البرمجية للتعامل مع السلاسل النصية.	٥٦
الفصل الثالث : هياكل البيانات	١١٧
الفصل الرابع : الكائنات البرمجية للتعامل مع السلاسل النصية	١٥٣
الفصل الخامس: التعرف الضوئي علي الحروف	٢٢٨

الفصل الأول

أنواع الخوارزميات

مقدمة

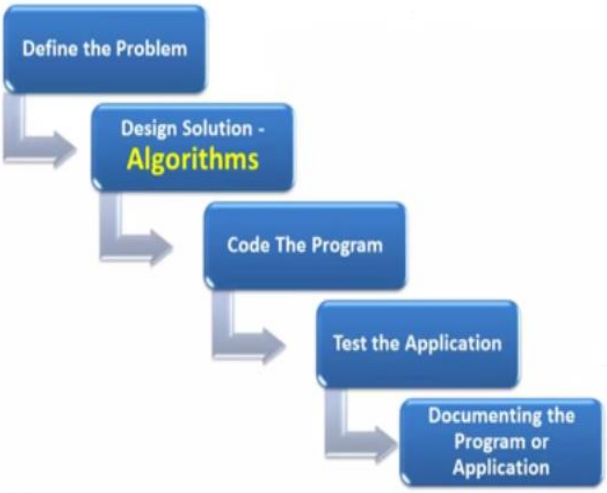
يمكننا تعريف الخوارزمية بأنّها أي سلسلة من الخطوات الحسابية المعرفة بشكل جيد، والتي تأخذ قيمة أو أكثر كدخل Input، وتعطي قيمة أو أكثر كخرج Output. بهذا المفهوم فإن الخوارزمية عبارة عن أداة لحل المسائل الحسابية. ونقصد بكلمة مسألة: العلاقة بين الدخل والخرج، وبالتالي فإن دور الخوارزمية توضيح الخطوات الضرورية للوصول إلى العلاقة المطلوبة. يمكن تعريفها أيضاً على أنها فكرة مجردة لبرنامج ليتم تنفيذه على الحاسب. سميت الخوارزمية بهذا الاسم نسبة إلى العالم المسلم أبو جعفر محمد بن موسى الخوارزمي الذي ابتكرها في القرن التاسع الميلادي. كلمة خوارزم (algorism) في الأصل كانت مقتصرة على القوانين الرياضية التي تستخدم الأرقام العربية و طُوِرت في اللاتينية من الخوارزمي (al-Khwarizmi) لتصبح (algorithm) في القرن الثامن عشر الميلادي لتشمل جميع إجراءات حل المشكلات و تنفيذ المهمات.

١- أنواع الخوارزميات:

- خوارزميات حسابية: تهتم بالمسائل الرياضية مثل (حل معادلة من الدرجة الأولى).
- خوارزميات غير حسابية: لا تهتم بالمسائل الرياضية ولكنها تحتاج إلى حل منطقي مثل (طريقة التدقيق الإملائي لنص ما، اتخاذ قرار بالذهاب إلى مكان ما وتحديد الطريق الأمثل للوصول إليه).

٢- الخطوات اللازمة لحل مشكلة بالحاسب:

- تعريف المشكلة، وتحليلها Problem Definitions and Analysis.
- وضع خوارزمية الحل Writing the Algorithm.
- كتابة الخوارزمية بإحدى لغات البرمجة Writing the Program.
- ترجمة ذلك البرنامج إلى لغة الآلة Compilation.
- تنفيذ البرنامج Execution.



مراحل تصميم برنامج

٣- خصائص الخوارزميات

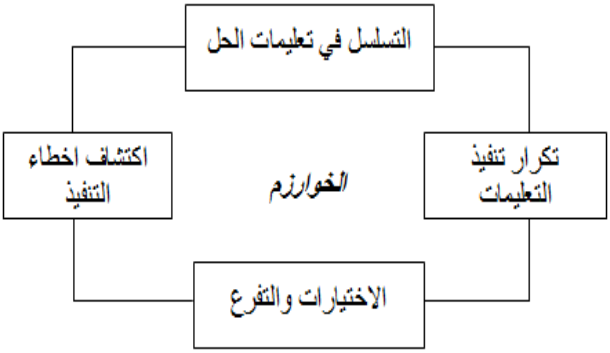
يشترط توفر الخصائص التالية في خطوات أي خوارزم:

١. أن تكون خطوات الخوارزمية مرتبه.
٢. خطوات الخوارزمية تنفذ أعمال بسيطة.
٣. خطوات الخوارزمية معرفه ومنتهية.
٤. طريقة الحل يجب ان تكون عامة.
٥. استخدام جمل قصيرة.

٦. فعالة وعند تنفيذها يراعى استخدام اقل زمن في المعالجهprocessing time واستخدام أقل مساحة في الذاكرة.

٧. أن تكون خطوات الخوارزمية قابلة للحل.

٤- أسلوب الخوارزم Algorithm



١- التسلسل Sequence

تعد مسألة حساب مرتبات العمال الشهرية مشكلة ؟

١. عدد كبير من العمال ٥٠٠٠ عامل مثلاً.

٢. يومية العامل – يخصم منه أيام الغياب.

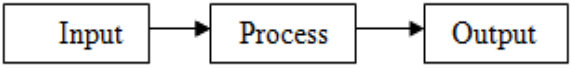
٣. مرتب العامل= اجر اليوم * عدد أيام العمل الفعلية.

٤. زيادة المرتب لجميع العاملين بنسبة ١٠ % كل عام.

٥. كل عامل يحصل على مساعدات شهرية حسب التصنيف.

فني	١٠٠٠ جنية
أول	٥٠٠ جنية
ثاني	٢٥٠ جنية
مساعد	١٠٠ جنية

لحل أي مشكلة نتبع الخطوات التالية:



- **المدخلات Input**

المطلوب تحديد المدخلات لحل المشكلة:

١. اسم العامل.

٢. مرتب العامل الشهري.

٣. عدد أيام العمل الفعلية للعامل.

٤. تصنيف العامل.

- **العملية Process**

تحديد أجر العامل عن أيام العمل الفعلية

أجر العامل X ، عدد أيام العملY، المرتب الشهري Z ، الأجر المستحق W .

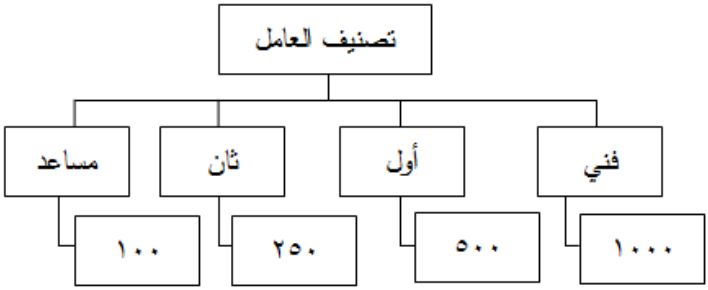
X=(Z/30)

W=X*Y

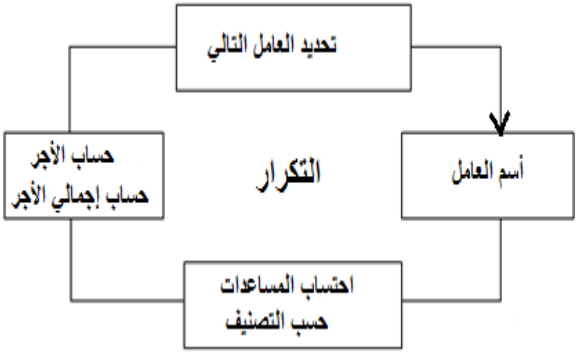
?	W	الأجر المستحق
?	X	الأجر اليومي
25	Y	عدد أيام العمل
3000	Z	أجر الشهر

100	X= 3000/30
2500	W=X*25

٢- الاختيار والاختبار



٣- التكرار



- المخرجات Output

اسم العامل	الأجر الشهري	عدد أيام العمل	أجر اليوم	اجمالي الأجر	مساعدات شهرية	اجمالي المستحق
علي	٣٠٠٠	٢٠	١٠٠	٢٠٠٠	١٠٠٠	٣٠٠٠
ماجد	١٥٠٠	١٥	٥٠	٧٥٠	٥٠٠	١٢٥٠
خالد	٩٠٠	٣٠	٣٠	٩٠٠	١٠٠	١٠٠٠

٥- طرق تمثيل الخوارزميات

يتم تمثيل الخوارزميات بإحدى الطرق التالية:

- **اللغة الطبيعية Natural Language**

يتم تمثيل الخوارزمية بإحدى لغات البشر مثل اللغة العربية واللغة الإنجليزية، بطرق سهلة وبسيطة وبدون تعقيد، فلا يحتاج المستخدم إلى أتباع قوانين اللغة في كتابة الخوارزمية.

مثال :

اذا أردت كتابة برنامج لقسمة رقمين، فيجب عليك أولا أن تفهم المشكلة جيدا، وتحدد خطوات حلها والخطوات التالية تستخدم لقسمة رقمين:

١- أبدأ.

٢- أقرأ الرقم الأول.

٣- أقرأ الرقم الثاني.

٤- أختبر هل الرقم الثاني يساوي صفر.

نعم:

٤-١- أطبع رسالة "لا يمكن القسمة على صفر"

٤-٢- أذهب إلى الخطوة رقم (٧).

٥- اقسم الرقم الأول على الرقم الثاني.

٦- أطبع ناتج القسمة.

٧- النهاية.

- **اللغة الرمزية Pseudocode**

وفيها يتم تمثيل الخوارزميات باستخدام أوامر ودوال مختصرة، دون التعرض للتفاصيل.

مثال: ما هو الأجر الصافي لعامل مرتبه الاجمالى الشهرى ٢٠٠٠جنيه علما بأنه يتم خصم ١٠ % من قيمة المرتب للضرائب و ٢ % للتأمين الصحى.

Pseudo code

1- Read Total

2- Tax = Total* 10/ 100

3- Health = Total * 2/100

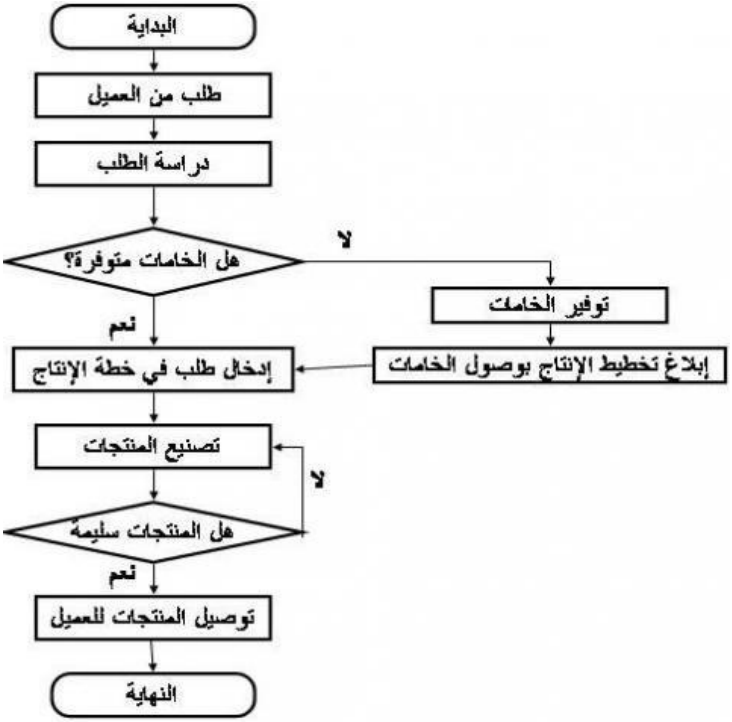
4- NetWage = Total – Tax – Health

5- print NetWage

٦- End Process

- **المخططات الانسيابية Flowchart**

وهي عبارة عن تمثيل رسومي للخوارزمية، يوضح حل المشكلة من البداية إلى النهاية. مع إخفاء بعض التفاصيل لتوضيح الصورة العامة للمشكلة. وتستخدم خرائط التدفق مجموعة من الرموز، وكل رمز له مدلول ثابت في الخرائط، لا يتغير من خريطة الى اخرى.



٧- أداء الخوارزميات Algorithm's Performance

ويوجد خمسة معايير بموجبها نستطيع اختيار الخوارزمية المناسبة لأداء العمل المطلوب:

١- صحة النتائج

الطريقة المثلى للتأكد بأن الخوارزمية تعمل بشكل الصحيح هي استخدام قواعد رياضيات للمعطيات والنتائج ومن ثم تطبيق هذه القواعد على الخوارزميه للتأكد من صحتها.

٢- كمية العمل المطلوب

لابد من أن نحلل الخوارزمية وخاصة الجزء الذي يتكرر بعدد من المعطيات مثل for loop, while وغيرها من الحلقات التكرارية وما تحتويه من أوامر هى التي تحدد كمية العمل المطلوب نظراً لأنها تتكرر عدد مرات اكثر كلما كبر حجم المعطيات.

٣- المثالية

للوصول للمثالية علينا أن نركز في التقليل من الخطوات مع الأخذ في الاعتبار أن هناك خطوات لابد منها.

٤- الذاكرة المستخدمة

ويستخدم ذلك المعيار لتحديد حجم الذاكرة التي تستخدمها الخوارزمية حتى اكتمال تنفيذها. وتحليل الخوارزمية من حيث هل يمكن ضغط المعطيات بحيث تأخذ حيز أقل. كذلك الحلقات التكرارية ونقارن المتغيرات وطريقة حفظها في الذاكرة.

٥- السهولة

في المعتاد أن سهولة الخوارزمية شئ مطلوب لكن في بعض الأحيان تكون الخوارزمية السهلة ليست هي الفعالة لذلك عند اختيار خوارزمية معينة لابد أن نضع في الاعتبار كثرة استخدامها، فإذا كانت ستستخدم بطريقة مستمرة قد يكون اختيار الخوارزمية الأكثر تعقيداً هو الاختيار الأمثل.

٨- ترميز الحروف النصية ASCII

الترميز American Standard Code for Information Interchange ASCII، هي مجموعة رموز ونظام ترميز مبني على الابجدية اللاتينية بالشكل الذي تستخدم به في الإنجليزية الحديثة ولغات غرب أوروبا. من أكثر الاستخدامات شيوعا للنصوص المكتوبة بالآسكي استخدامها في

أنظمة الحاسوب، وفي أجهزة الاتصالات وأنظمة التحكم التي تتعامل مع نصوص. عرف ASCII العلاقة بين رمز أو حرف معين وترتيب من البتات؛ إلى جانب حجز مجموعة للرموز التحكم للبيانات المخزنة في هيئة سطور. السبب في استخدام نظام الآسكي تكمن في كون أجهزة الحاسب ليس بمقدورها فهم غير الأرقام. ومن هذا المنطلق فإن شفرة الآسكي تشكل التمثيل العددي للحروف و الأرقام و الأوامر. ، والجدول التالي يوضح ASCII Code لحروف اللغة الإنجليزية.

Character	ASCII
a	97
b	98
c	99
d	100
e	101
f	102
g	103
h	104
i	105
j	106
k	107
l	108
m	109

Character	ASCII
n	110
o	111
p	112
q	113
r	114
s	115
t	116
u	117
v	118
w	119
x	120
y	121
z	122

Character	ASCII
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77

Character	ASCII
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Character	ASCII
0	48
1	49
2	50
3	51
4	52
5	53
6	54
7	55
8	56
9	57

وفقاً لقانون شفرة (الكود) الآسكي ، الآن يمكننا تحويل أي حرف أو أي علامة من علامات الترقيم التي تكتب من قبل المستخدم إلى عدد؛ وبهذا يصبح من الممكن لجهاز الحاسب إدراكها واستيعابها.

على سبيل المثال الحرف A يمكننا ترجمته إلى عدد بواسطة استخدام جدول الآسكي ليصبح معادلاً العدد ٦٥.

مثال آخر كتابة كلمة: " java "

Int j = 106

Int a = 65

Int v = 118

Int a = 65

٩- أنواع الخوارزميات

يمكن تصنيف الخوارزميات بحسب الوظيفة التي تقوم بها .

٩-١- خوارزميات الترتيب Sort Algorithm :

هدفها ترتيب مجموعة من العناصر ترتيباً متتالياً اعتماداً على أحد بنود العناصر أو على اجتماع عدة بنود محددة.

- **خوارزمية الترتيب الفقاعي Bubble Sort Algorithm**

خوارزمية الترتيب الفقاعي من خوارزميات الترتيب الأساسية، سُميت خوارزمية الترتيب الفقاعي بهذا الإسم لسبب لطيف. فكما تأخذ الفقاعة طريقها لتعلو خارج الماء فإن العناصر في هذه الخوارزمية تأخذ طريقها لمكانها الصحيح بطريقة تشبه الفقاعة، و ستشاهد هذا في المثال.

الفكرة الاساسية لخوارزمية الترتيب الفقاعي هي أن تضع أكبر عنصر في المصفوفة في مكانه الصحيح في كل دورة.

١. ماذا يحدث في كل دورة ؟ تضع المؤشر في أول عنصر بالمصفوفة، ثم تقارنه مع العُنصر التالي.

٢. إذا كان العنصر الأول أصغر من العنصر الثاني، أنقل المؤشر إلى العنصر التالي(لا يوجد تبديل).

٣. إذا كان العنصر الأول أكبر من العنصر الثاني، بدل العنصر الثاني مع العنصر الأول ثم أنقل المؤشر إلى العنصر التالي(يوجد تبديل).

٤. نواصل العملية حتى العنصر قبل الأخير بالمصفوفة.

٥. عدد عناصر المصفوفة التي ترتبها خوارزمية الترتيب الفقاعي في كل دورة تقل بواحد حتى تنتهي العناصر، فإذا كان عدد عناصر المصفوفة ٥، ففي الدورة الثانية ستُرتب ٤ و الدورة الثالثة ستُرتب ٣ عناصر و هكذا. لأنه في كل دورة تضع الخوارزمية عنصراً في مكانه الصحيح، فبالتالي لا نحتاج لإدخاله في عملية الترتيب مرة أخرى.

٦. عدد الدورات = عدد عناصر المصفوفة – ١. في أول دورة نأخذ العناصر كاملة، و يقل عدد العناصر بواحد كلما إنتقلنا إلى دورة التالية و هكذا حتى نصل إلى أن يكون عدد العناصر المتبقية ٢. هنا.. إذا كانت العناصر مرتبة فقد إكتمل الترتيب، و إذا لم تكن مرتبة فسترتبها و إكتمل الترتيب، إذاً فما فائدة ترتيب العنصر الأخير لوحده؟ لهذا تجد أن عدد الدورات في خوارزمية الترتيب الفقاعي = عدد العناصر – ١ .

٧. إذا ما كانت المصفوفة مرتبة أصلاً، و أجريت عليها خوارزمية الترتيب الفقاعي، كيف تعرف أن المصفوفة مرتبة مسبقا؟ إذا ما وجدت أن الخوارزمية أكلمت دورة

كاملة دون أن تجري عملية تغيير واحدة، عندها تعلم أن المصفوفة مرتبة .

٨. في أي مرحلة إذا ما علمت أن مصفوفتك مرتبة فلا داعي لإكمال خوارزمية الترتيب الفقاعي، و ذلك بإستخدام القاعدة السابقة، فقد يكون العنصر الأول و الثاني فقط في غير موضعهم، و بإجراء أول عملية تبديل يكون قد إكتمل ترتيب المصفوفة. في الدورة التالية لن يحدث أي تبديل و بذلك تكون قد علمت أن المصفوفة مرتبة... أوقف الخوارزمية.

Bubble Sort Algorithm
BUBBLESORT(A) 1. **for** *i* ← 1 to *length*[A] 2. **do for** *j* ← *length*[A] **downto** *i* + 1 3. **do if** A[*j*] < A[*j* – 1] 4. **then** exchange A[*j*] ↔ A[*j* – 1]

ويمكن تطبيقه ككود كما يلي:

```
int list[MAX], i, n, sorted, k, tmp;
n = MAX - 1;
k = n; sorted=0;
while ((k > 0) && (sorted == 0))
  { sorted= 1;
    for (int j = 0;j < k; j++)
     { if(list[j] > list[j+1])
        { tmp = list[j];
          list[j] = list[j+1];
          list[j+1] = tmp;
          sorted=0;
        }
     }
    k--;
}
```

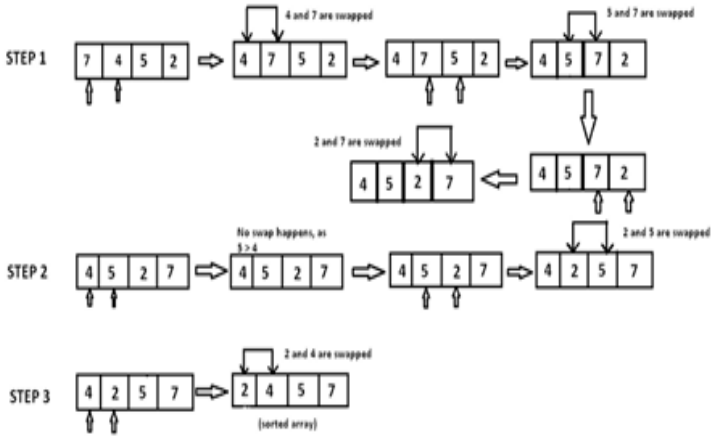
يعتبر هذا الخوارزم الأبطأ من بين أقرانه، حيث يقوم هذا الخوارزم بترتيب عناصر المصفوفة عنصراً عنصراً عن طريق مقارنته بالعنصر الذي بعده، ويقوم بالتبديل في موقعهما

في المصفوفة متى ما استلزم الأمر. يكرر هذا الخوارزم هذه الدورة على جميع العناصر عنصراً خلف الآخر إلى أن يصل إلى دورة لا يتغير فيها موقع أي عنصر، حينها يتوقف وتكون العناصر قد ترتبت!

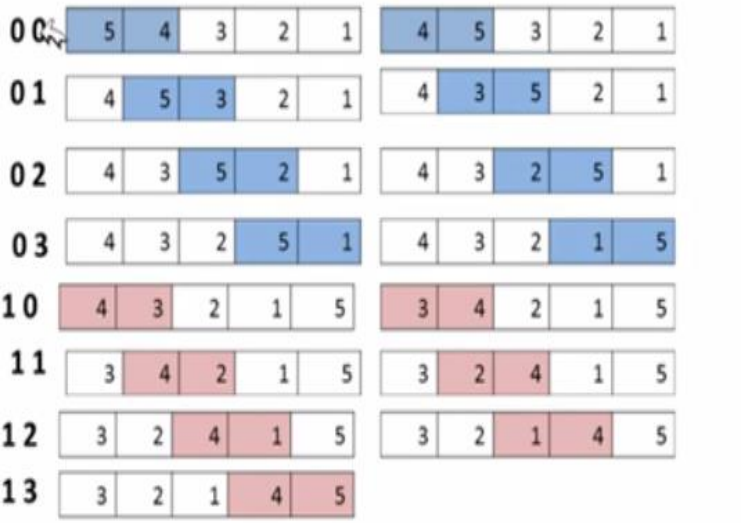
بعبارة أخري يعمل هذا الخوارزم على ترتيب عناصر مصفوفة ليست مرتبة العناصر، وتتم عملية الترتيب على مراحل أو دورات phases كل مرحلة تتضمن عملية مقارنة زوجين متجاورين من العناصر واستبدال موضعهما إذا كان ترتيبهما غير صحيح... وهكذا.

- **آلية عمل الخوارزم:**

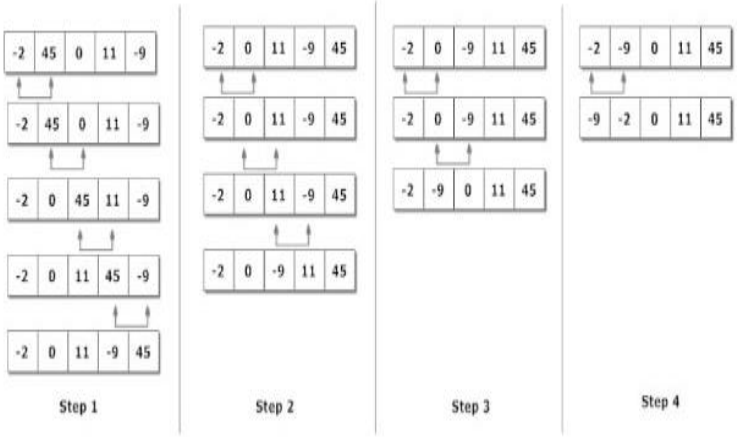
فالنفترض أن لدينا مصفوفة A غير مرتبة تحتوي على n عنصر، والمطلوب ترتيب العناصر ترتيب تصاعديا. فالنفترض أن المصفوفه تحتوي على العناصر الآتية: {7,4,5,2}


STEP 1
4 and 7 are swapped
5 and 7 are swapped
2 and 7 are swapped
STEP 2
No swap happens, as 5 > 4
2 and 5 are swapped
STEP 3
2 and 4 are swapped
(sorted array)

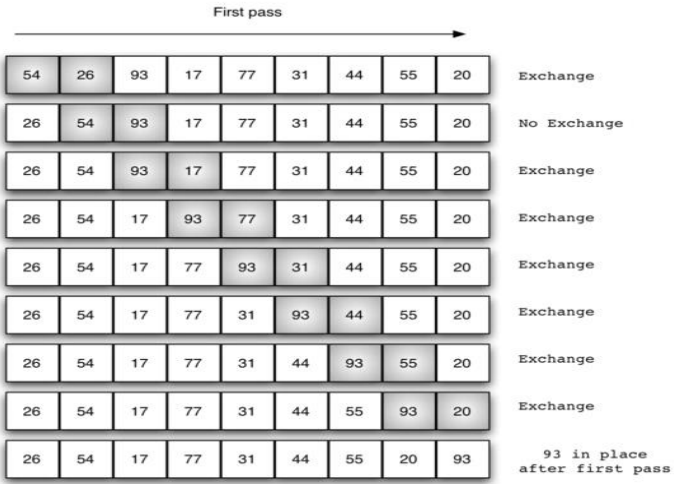
مثال ١ :


00
01
02
03
10
11
12
13

مثال٢ :


-2 45 0 11 -9
-2 45 0 11 -9
-2 0 45 11 -9
-2 0 11 45 -9
-2 0 11 -9 45
Step 1
-2 0 11 -9 45
-2 0 11 -9 45
-2 0 11 -9 45
-2 0 -9 11 45
Step 2
-2 0 -9 11 45
-2 0 -9 11 45
-2 -9 0 11 45
Step 3
-2 -9 0 11 45
-9 -2 0 11 45
Step 4

مثال٣:


First pass
54 26 93 17 77 31 44 55 20 Exchange
26 54 93 17 77 31 44 55 20 No Exchange
26 54 93 17 77 31 44 55 20 Exchange
26 54 17 93 77 31 44 55 20 Exchange
26 54 17 77 93 31 44 55 20 Exchange
26 54 17 77 31 93 44 55 20 Exchange
26 54 17 77 31 44 93 55 20 Exchange
26 54 17 77 31 44 55 93 20 Exchange
26 54 17 77 31 44 55 20 93
93 in place after first pass

للتعامل مع السلسلة النصية يتم تحويل كل حرف إلى ASCII Code المناظر له اولاً ثم اتباع خطوات الخوارزم.

تدريب

رتب السلسلة النصية التالية:

H	C	A

- **خوارزمية الترتيب بالاختيار Selections Sort Algorithm**

سُميت هذه الخوارزمية بهذا الاسم لأنها تعتمد على مبدأ اختيار أصغر عنصر بالمصفوفة في كل مرة و وضعه في مكانه الصحيح.

طريقة عمل خوارزمية الترتيب بالاختيار

١. ابحث عن أصغر عنصر بالمصفوفة و بادله بأول عنصر بالمصفوفة، و المبادلة تعني أن تضع كل عنصر مكان الآخر، و إذا وجدت أن أصغر عنصر بالمصفوفة في أول خانة منها فلا داعي لإجراء تغيير، فهو في مكانه الصحيح.

٢. ابحث عن أصغر عنصر في المصفوفة ناقصاً الجزء المُرتب.

٣. لتوضيح معنى الجزء المرتب، عندما وضعت في الخطوة الأولى أصغر عنصر في الخانة الأولى فأنت تعلم أن هذا العنصر قد أخذ مكانه الصحيح وهنا نسمي الخانة الأولى بالجزء المرتب، و لإيجاد ثاني أصغر عنصر بالمصفوفة ستبحث في عناصر عددها = طول المصفوفة -١، في الخطوة الثانية يكون لديك عنصران مرتبان، الخانة الأولى و الخانة الثانية و هذان هما الجزء المرتب، و بالتالي ستبحث في عناصر عددها = طول المصفوفة -٢ لوضع العنصر الثالث في مكانه.

٤. تُعيد الخطوة الثانية حتى تصل لنهاية المصفوفة، بتعبير آخر، يصبح طول الجزء المرتب من المصفوفة مساوياً لطول المصفوفة.

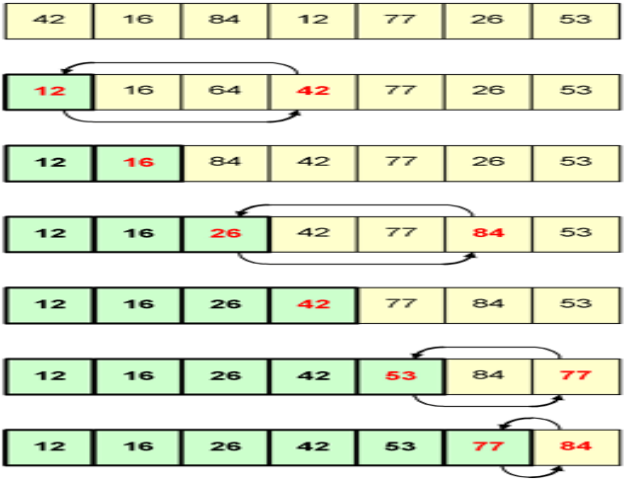
Selections Sort Algorithm
Step 1 − Set MIN to location 0 Step 2 − Search the minimum element in

the list Step 3 − Swap with value at location MIN Step 4 − Increment MIN to point to next element Step 5 − Repeat until list is sorted

الكود التقريبي لخوارزمية الترتيب بالاختيار

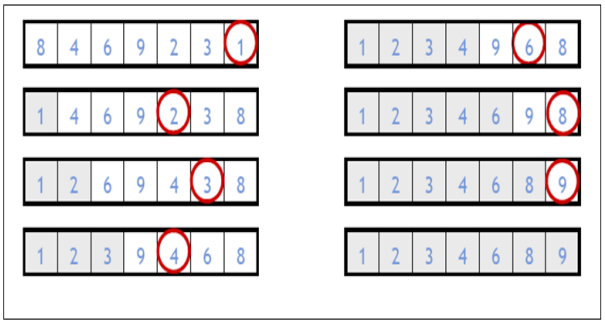
```
SELECTION-SORT(A)
for j ← 1 to n-1
smallest ← j
for i ← j + 1 to n
] if A[ i ] < A[ smallest
smallest ← i
Exchange A[ j ] ↔ A[ smallest]
```

مثال١ :



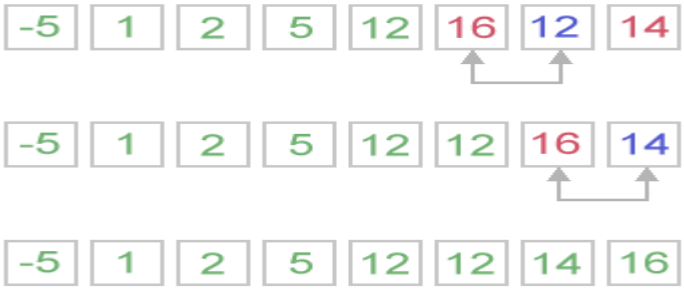
مثال٢ :

هذا الشكل يرتب هذة المصفوفة :⟨8,4,6,9,2,3,1⟩



Example. Sort {5, 1, 12, -5, 16, 2, 12, 14} using selection sort.





تدريب

رتب السلسلة النصية التالية:

N	A	F	X	C

٩-٢- خوارزميات البحث

تُعتبر خوارزمية البحث ركنا أساسيا من أركان علم الخوارزميات و تتخذ هذه الخوارزمية عدة أشكال، من أبسطها البحث عن عدد في مصفوفة مُحددة الحجم و يزداد الأمر تعقيدا عند الانتقال إلى البحث عن كلمة داخل نص، تماما كما ترى في محررات النصوص العادية والتي تحتوي على خاصية Find & Replace حيث أن أغلب المحررات الحالية تستخدم خوارزميه بحث تسمى Boyer-Moore Searching التي تُعد تقريبا من أسرع الخوارزميات في مجال البحث .

هناك أيضا بحث من نوع آخر، فماذا لو كانت لدينا مجموعة حروف و نريد إيجاد جميع الكلمات التي تبدأ بهذه الحروف؟؟ عادة ما يُسمى هذا النوع من الخوارزميات بـ prefix searching و لهذا النوع تطبيقات كثيرة خصوصا في محركات البحث و القواميس و المتصفحات التي تستخدم هذه الخوارزمية عند كتابتك لموقع يبدأ بحرف كنت قد زرته سابقا.

لا تقتصر خوارزمية البحث على ما ذكرناه آنفا، فهناك نوع آخر من الخوارزميات يُستخدم لإيجاد نص قريب من النص الذي كنت تبحث عنه، حيث تقوم الخوارزمية بالبحث عن 4أو 5 كلمات قريبة من الكلمات الخاطئة، تماما كما يفعل Google عند الترجمة أو Office Word عند كتابة نص يحتوي على أخطاء إملائية و تعتمد أغلب هذه الخوارزميات على الوزن الصوتي للحرف و من أشهرها Soundex. Searching ليس هذا فقط، فمضادات الفيروسات تستخدم خوارزميات بحث سريعة للبحث عن وجود توقيع مطابق لأحد بيانات الملف المراد فحصه، وبما أن قواعد بيانات التواقيع تكون ضخمة للغاية فالبحث عن كل توقيع سيكون بطيئا جدا وهناك الأفضل، حيث توجد خوارزميات بإمكانها البحث عن

عده تواقيع في نفس اللحظة وتستخدم بنية شجرية للقيام بهذا الأمر، هناك أيضا خوارزميات أخرى تعتمد على مفاهيم مختلفة مثل Hash Table وعدة أمور أخرى، و يُسمى هذا النوع من الخوارزميات بـ Multiple pattern searching و هو من أصعب الخوارزميات، وهناك العديد من مضادات الفيروسات التي تستخدم مثل هذه الخوارزميات مثل ClamAV.

- **خوارزمية البحث الخطي Liner Search Algorithm**

الهدف : البحث عن قيمة المفتاح key داخل المصفوفة X.

الفكرة : تعتمد هذه الخوارزمية على البحث التسلسلي (Sequential Search) في المصفوفة X حيث يبدأ البحث من أول عنصر إلى أن تنتهي المصفوفة، و في كل مرة نقارن محتوى الخانة الحالية X[i] مع المفتاح Key، فإذا كانت القيمتان متساويتان (X[i] = Key) ستتم إعادة قيمة المتغير **i**الذي يُمثل مكان وجود المفتاح Key في المصفوفة X أما إذا

كانت القيمتان مختلفتان فسننتقل إلى الخانة الموالية X[i+1] للبحث من جديد و هكذا.

النتيجة : إذا كانKey موجود في X فسنحصل على رقم الخانة التي يوجد فيها المفتاح و إلا ترجع الخوارزمية بقيمة مختلفة عن قيم فهرس المصفوفة ولتكن 0 أو 1-، وهذا يدل على عدم وجود العنصر المراد البحث عنه ضمن السلسلة المراد البحث فيها.

الخوارزمية بسيطة و تقليدية أيضا كما لا يُشترط الترتيب عند البحث، ولكنها بطيئة و غير عملية خصوصا عند معالجة المصفوفات الضخمة.

- مثال لطريقة البحث في خوارزمية البحث الخطي
- نرمز إلى طول المصفوفة بالرمز n
- نرمز إلى العنصر المراد البحث عنه بالرمز x
- نرمز إلى مؤشر الفهرس بالرمز j
- نرمز إلى قيمة أي عنصر بالمصفوفة بالرمز A[j]

لنفترض أن لدينا المصفوفة أدناه حيث n = 6 ، x=4

المصفوفة



الفهرس

١. تبدأ عملية البحث بمقارنة العنصر بالخانة رقم ١ حيث j=1، هلA[j]=x ؟

لا إذن ننتقل إلى الخطوة الثانية.

٢. مقارنة العنصر بالخانة رقم ٢ حيثj=2 ، هل A[j]=x؟

لا إذن ننتقل إلى الخطوة الثالثة.

٣. مقارنة العنصر بالخانة رقم ٣ حيثj=3 ، هل A[j]=x؟

لا إذن ننتقل إلى الخطوة الرابعة.

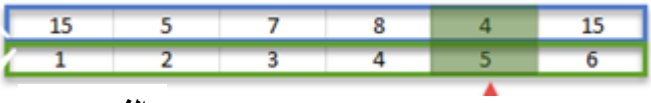
٤. مقارنة العنصر بالخانة رقم ٤ حيثj=4 ، هل A[j]=x؟

لا إذن ننتقل إلى الخطوة الخامسة.

٥. مقارنة العنصر بالخانة رقم ٥ حيثj=5 ، هل A[j]=x؟

نعم إذن هنا نحتفظ بالرقم ٥ و تُعتبر هذه نتيجة البحث.

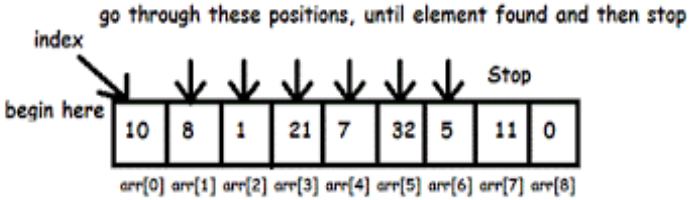
المصفوفة



الفهرس

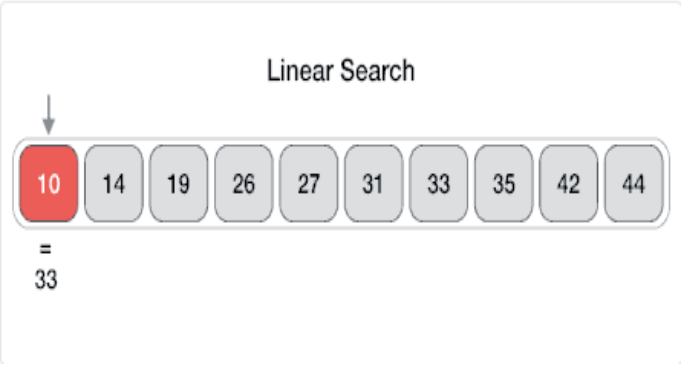
Liner Search Algorithm
Given a list *L* of *n* elements with values or records L_0 ... L_{n-1}, and target value *T*, the following subroutine uses linear search to find the index of the target *T* in *L*. 1. Set *i* to 0. 2. If $L_i = T$, the search terminates successfully; return *i*. 3. Increase *i* by 1. 4. If $i < n$, go to step 2. Otherwise, the search terminates unsuccessfully.

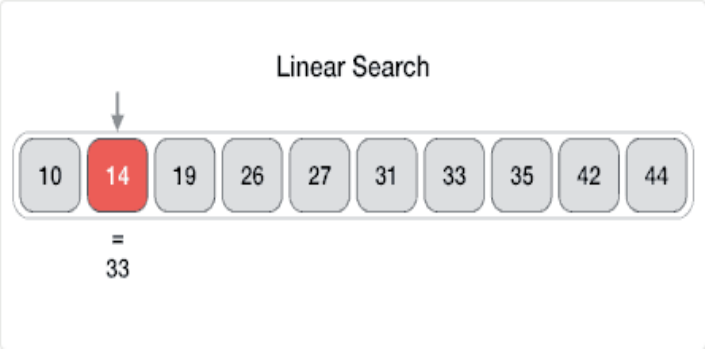
مثال ١ :

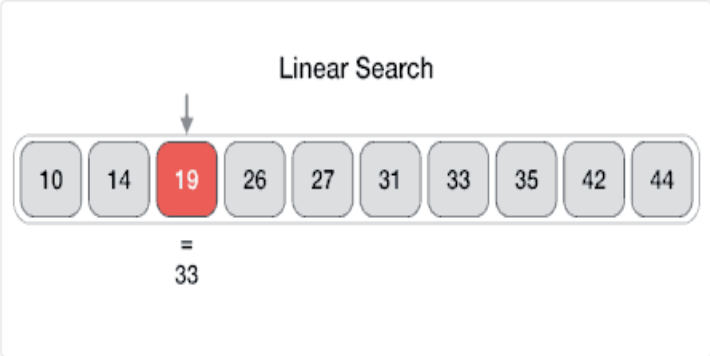


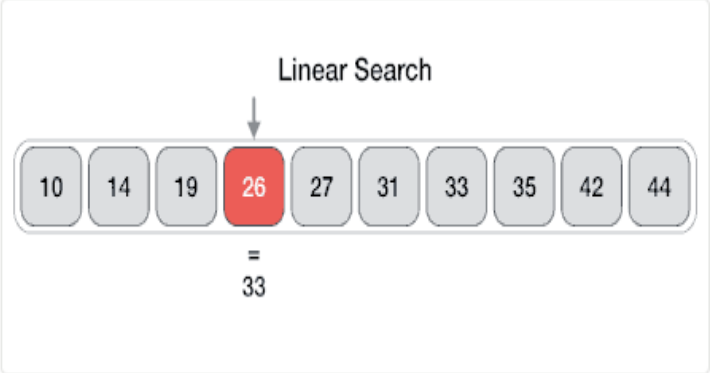
Element to search : 5

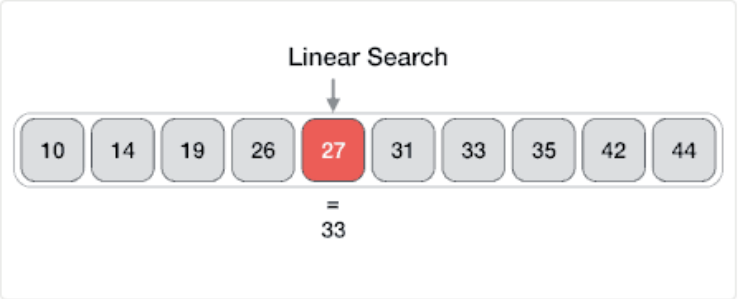
مثال ٢ :

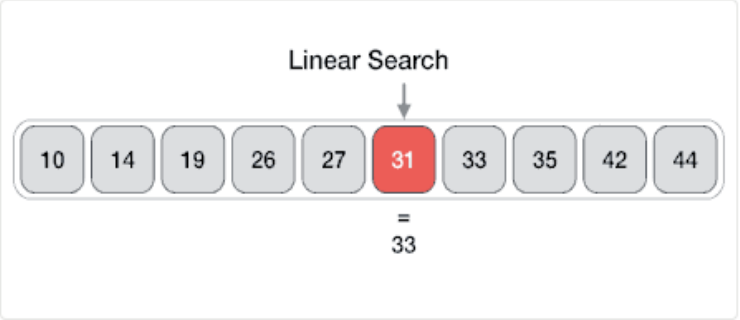


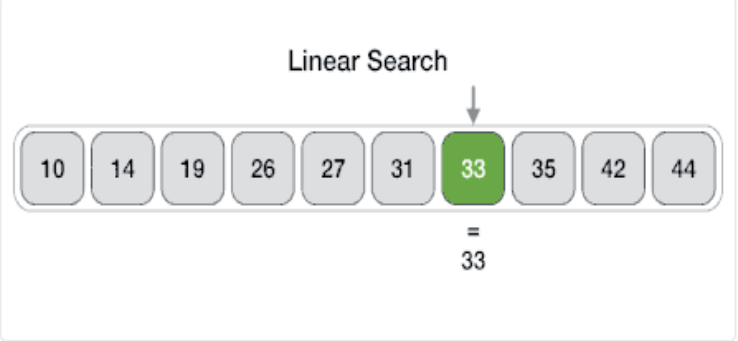

Linear Search
10
14
19
26
27
31
33
35
42
44
=
33

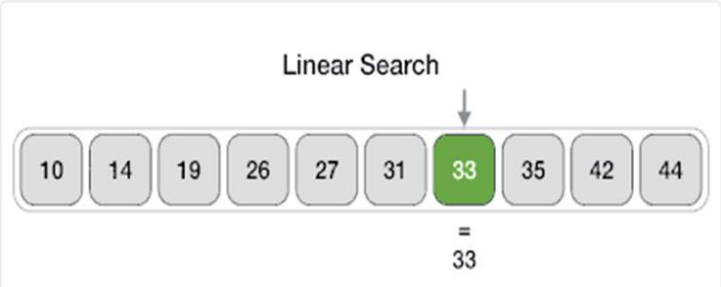

Linear Search
10
14
19
26
27
31
33
35
42
44
=
33


Linear Search
10
14
19
26
27
31
33
35
42
44
=
33


Linear Search
10
14
19
26
27
31
33
35
42
44
=
33


Linear Search
10
14
19
26
27
31
33
35
42
44
=
33


Linear Search
10
14
19
26
27
31
33
35
42
44
=
33



تدريب

أبحث عن السلسلة "X"، ضمن السلسلة النصية التالية:

Z	N	C	F	X	A	B

ملحوظة

حول الحرف المراد البحث عنه وكذلك حروف السلسة النصية إلى ASCII Code المناظر له.

- **خوارزمية البحث الثنائي Binary Search Algorithm**

١. خوارزمية البحث الثنائي كغيرها من الخوارزميات لا بد من توافر المدخلات المطلوبة و التي تتمثل في العُنصر المراد البحث عنه و المصفوفة التي سيتم البحث فيها .

٢. يجب أن تكون المصفوفة مرتباً تصاعدياً)من الأصغر إلى الأكبر)، في حال لم تكن المصفوفة مرتبة تصاعدياً يجب عليك أن تُرتبها تصاعدياً حتى تستطيع تطبيق خوارزمية البحث الثنائي .

٣. مصفوفة البحث لها فهرس (Index) يبدأ من الصفر أو الواحد، لكليهما نفس النتيجة .

٤. تبدأ الخوارزمية عملها بوضع مؤشر الفهرس في وسط المصفوفة حتى تقسم المصفوفة إلى قسمين، و هنا بداية الاستفادة الفعلية من مبدأ .

٥. توجد مصفوفات عدد عناصرها زوجي و أخرى عدد عناصرها فردي فكيف يكون مؤشر المصفوفة في الحالتين في الوسط. إن عملية اختيار وسط المصفوفة بسيطة و سهلة، كل ما عليك إتباع المعادلة التالية:

$$\lfloor (low + high)/2 \rfloor$$

و تعني هذه المعادلة (floor (low + high)) /2 فيكون وسط مصفوفة عناصرها ٥ هو العنصر الثاني، و وسط مصفوفة عناصرها ٤ أيضاً العنصر الثاني.

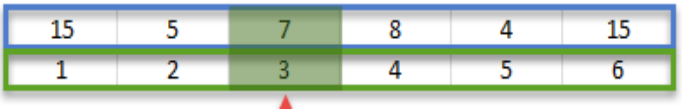
مهمة الدالة floor هي إيجاد أكبر قيمة صحيحة (ليست كسر) أقل من القيمة المُستقبلة في الدالة.

floor(3.9) = 3 ، floor(100.1) = 100

floor(-10.3) = -11 ،

مثال:

مصفوفة بعدد عناصر زوجي.



وسط المصفوفة

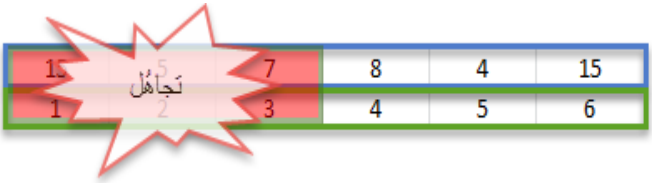
٦. تتم مقارنة العنصر بالوسط (حيثُ المؤشر) بالعُنصر المُراد البحث عنه و النتيجة إحدى ثلاث حالات :

١. أن تساوي قيمة العُنصر بالمصفوفة قيمة العُنصر المُراد البحث عنه، و بالتالي يتم الإحتفاظ بقيمة مؤشر الفهرس و يُوقف البحث.



قيمة مؤشر الفهرس

٢. أن تكون قيمة العُنصر المراد البحث عنه أكبر من قيمة العنصر حيث المؤشر، في هذه الحالة يتم تجاهل جميع عناصر النصف الأيسر من المصفوفة و يُصبح الجزء الأيمن من المصفوفة مُدخلاً جديداً لعملية بحث عن طريق خوارزمية البحث الثنائي.



٣. أن تكون قيمة العُنصر المراد البحث عنه أقل من قيمة العنصر حيثُ المؤشر، في هذه الحالة يتم تجاهل جميع عناصر

النصف الأيمن من المصفوفة و يُصبح الجزء الأيسر من المصفوفة مُدخلاً جديداً لعملية بحث عن طريق خوارزمية البحث الثنائي.

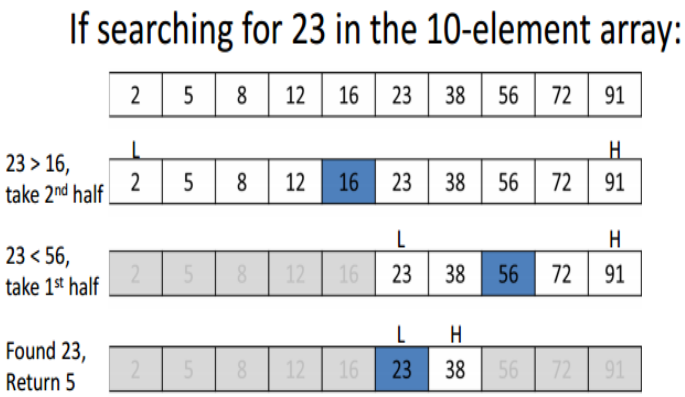


٤. تُكرر عملية البحث (**الخطوة ٦**) في النصف المتُبقي حتى يتم الوصول إلى خانة واحدة فقط تُمثل موقع العنصر المُراد البحث عنه.

Binary Search Algorithm
Input: An array A[1..n] of n elements sorted in nondecreasing order and an .element x Output: j if x = A[j]; 1 <= j <= n; and .0 otherwise

```
low=1; high=n; j=0
(while (low <= high) and (j = 0
( mid=floor( (low + high)/2
if x = A[mid] then j=mid
else if x < A[mid] then high=mid - 1
else low=mid + 1
end while
return j
```

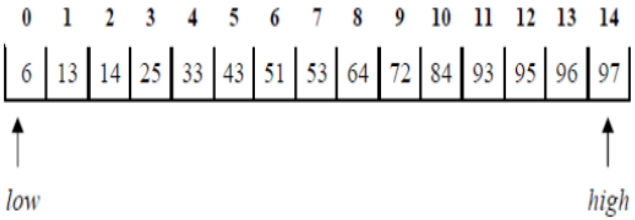
مثال ١:



خوارزمية البحث الثنائي من أسرع الخوارزميات المستخدمة في البحث، وأكثرها شيوعا. حيث تُنصف(تقلص للنصف)عدد عناصر المصفوفة في كل تكرار، لذا تستغرق عملية البحث زمنا لوغاريتميا. ولكنها أكثر تعقيدا من خوارزمية البحث الخطي التقليدية.

مثال٢:

ابحث عن القيمة 33 في المصفوفة التالية :

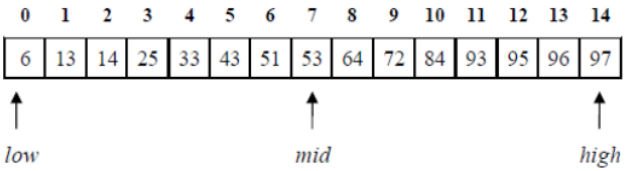


الحل:

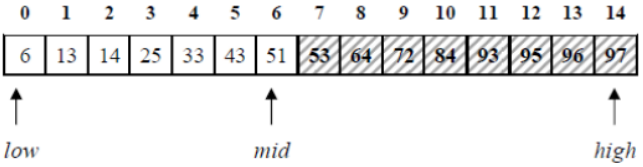
1. نحدد منتصف المصفوفة:

$$Mid = \frac{low + high}{2} = \frac{(0 + 14)}{2} = 7$$

2. تقسيم المصفوفة إلى قسمين وفقا لموقع mid:

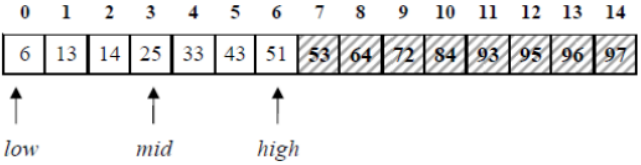


3. بما أن 33 أصغر من 53, فعملية البحث ستنفذ على النصف الأول:

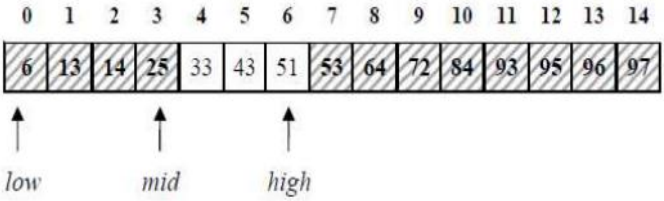


4. نعيد عملة التقسيم على النصف الأول و تصبح قيمة mid تساوي:

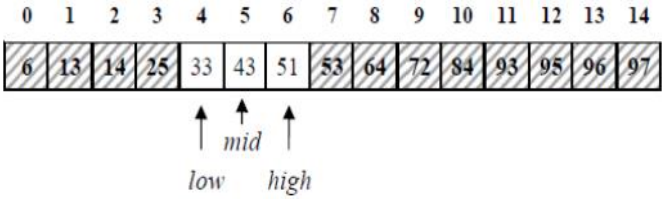
$$Mid = \frac{(0 + 6)}{2} = 3$$



5. بما أن 33 أكبر من 25, سيتم استبعاد النصف الأول و تستمر عملية البحث في النصف الثاني:



6. من جديد, نعيد عملية التقسيم حيث تصبح قيمة mid تساوي 5:



7. و أخيرا نجد أن القيمة 33 موجودة في الموقع low و الذي يساوي 4.

تدريب١:

أبحث عن السلسلة "X"، ضمن السلسلة النصية التالية:

Z	N	C	F	X	A	B

تدريب٢:

أبحث عن السلسلة "Y"، ضمن السلسلة النصية التالية:

Y	N	C	F	X	A	B

ملحوظة

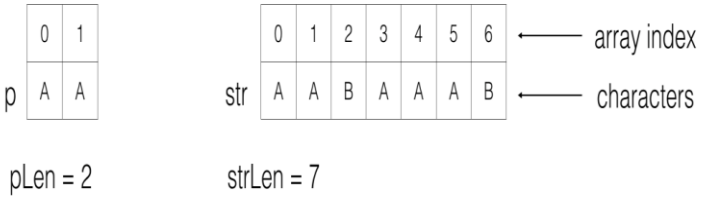
حول الحرف المراد البحث عنه وكذلك حروف السلسة النصية إلى ASCII Code المناظر له

- **خوارزمية Naive String Matching Algorithm**

تعتمد هذه الخوارزمية على تطابق نموذج pattern بالسلسلة string في كافة المواقع بالسلسة، مع الأخذ في الاعتبار طول النموذج pattern (عدد الأحرف) يكون أصغر من أو مساوي طول السلسلة string. بينما إذا كان طول النموذج pattern (عدد الأحرف) أكبر من طول السلسلة string يمكن أن نقول "Pattern Not Found!".

مثال:

أوجد pattern **p** في string **str**



pLen = 2 (no. of characters) of pattern p.

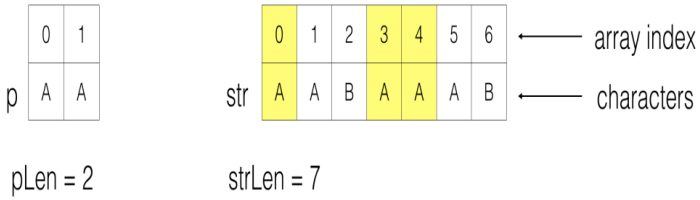
strLen = 7 is the length of the string str.

naive Algorithm

Set Limit = strLen - pLen
for (i = 0; i <= Limit; i++) {
  for (j = 0; j < pLen; j++) {
  if (p[j] != str[i + j])
  break }
  if (j == pLen)
  Print: "Pattern found at position " + i
}

Positions where pattern p is found

```
   +---+---+---+---+---+---+---+
   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | <-- array index
   +---+---+---+---+---+---+---+
str | A | A | B | A | A | A | B | <-- characters
   +---+---+---+---+---+---+---+
   <---p--->  <---p--->
              <---p--->
```



Output

Pattern found at position 0

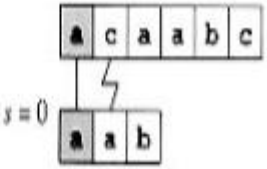
Pattern found at position 3

Pattern found at position 4

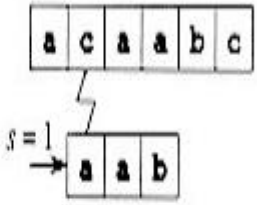
NAÏVE_STRING_MATCHER (T, P)

1. $n \leftarrow$ length [T]
2. $m \leftarrow$ length [P]
3. for $s \leftarrow 0$ to $n - m$ do
4. if $P[1 . . m] = T[s + 1 . . s + m]$
5. then return valid shift s

pattern P = aab and the text T = acaabc.

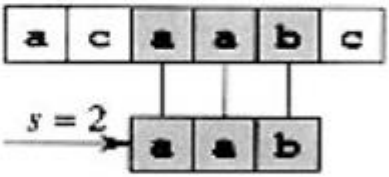


(a)

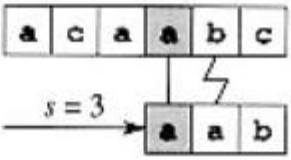


(b)

<centre>



(c)



(d)

Output:

Pattern found at index 2.

تدريب

1) Input:

txt [ ] = “THIS IS A TEST TEXT”

pat [ ] = “TEST”

Output:

Pattern found at index 10.

2) Input:

txt [ ] = “AABAACAADAABAAABAA”

pat [ ] = “AABA”

Output:

Pattern found at index 0

Pattern found at index 9

Pattern found at index 13

- **خوارزمية The Knuth- Morris-Pratt Algorithm**

تبحث عن تحديد قيمة عدد الأحرف التي نتحركها قبل اجراء عملية التطابق.

algorithm kmp_search
input: **an array of characters, S (the text to be searched)** **an array of characters, W (the word sought)** **output:** **an integer (the zero-based position in S at which W is found** **or -1 when W not found in S)** **define variables:** **an integer, m ← 0 (the beginning of the current match in S)** **an integer, i ← 0 (the position of the current character in W)**

```
        an array of integers, T (the table,
computed elsewhere)

    while m + i < length(S) do
        if W[i] = S[m + i] then
            let i ← i + 1
            if i = length(W) then
                (occurrence found, we may return
m or store/use m and prepare for next
search)
                return m
                let m ← m + i - T[i], i ← T[i]
(T[length(W)] can't be -1)
        else
            if T[i] > -1 then
                let m ← m + i - T[i], i ← T[i]
            else
                let m ← m + i + 1, i ← 0
      (if we reach here, we have searched all
of S unsuccessfully)
    return -1
```

مثال:

pattern "nano" in text "bananananobano".

```
   0  1  2  3  4  5  6  7  8  9 10 11
T: b  a  n  a  n  a  n  o  b  a  n  o
```

```
i=0: X
i=1:    X
i=2:       n  a  n  X
i=3:          X
i=4:             n  a  n  o
i=5:                X
i=6:                   n  X
i=7:                      X
i=8:                         X
i=9:                            n  X
i=10:                              X
```

تدريب

1. Input: txt[] = "THIS IS A TEST TEXT"
   pat[] = "TEST"
   Output: Pattern found at index 10

2. Input: txt[] =
   "AABAACAADAABAABA"
   pat[] = "AABA"
   Output: Pattern found at index 0
   Pattern found at index 9
   Pattern found at index 12

الفصل الثاني

الدوال البرمجية للتعامل مع السلاسل النصية

الدوال هي أسماء محجوزة ومعرفة من قبل لغة البرمجة لتقوم بعمل معين مثل دالة القيم المطلقة وغيرها.. أو بمعني ابسط واعم هي عبارة عن برنامج صغير مكتوب مسبقا و محفوظ داخل لغة البرمجة يمكن استدعاءه من خلال برنامجك والاستفادة منه، إضافة إلى ذلك يمكنك أنت أن تكتب دالة وتستدعيها أكثر من مرة داخل برنامجك.

تعرف النصوص برمجياً على أنها عبارة عن مجموعة من الحروف والأرقام او الرموز.

النص ما هو إلا سلسلة من الحروف المتتابعة، فكلمة hello world ما هي إلا سلسة تتكون من ١١ حرفاً، حيث ان المسافة تعتبر حرفاً أيضاً، وكل حرف في السلسله له مكان index يعبر عن ترتيبه داخل السلسلة ويبدأ الترتيب كما هو معروف في السي شارب من الرقم صفر ، بحيث يكون h ترتيبه ٠ و d اخر حرف ترتيبه 11 .

النص في لغة السي شارب هو سلسلة من الحروف char الواقعة بين علامتي الاقتباس المزدوج " "، و أنّ النوع string

هو الذي يمثّل أي نص في سي شارب . يتم تعريف السلاسل النصية كمصفوفات من الأحرف.

مثال:

لتخزين السلسلة النصية "Merkkijono" : يجب أن نكتب:

```
char string[] = "Merkkijono";
```

or

```
char string[] = {'M', 'e', 'r', 'k', 'k', 'i', 'j', 'o', 'n',
'o', '\0'};
```

يمكن إنشاء **سلسلة اضافية طويلة** كما يلي:

عليك تقسيم السلسلة إلى أقسام متعددة ، عن طريق إغلاق القسم الأول بعلامتى اقتباس quote وإعادة وصل السلسلة على السطر التالي (أيضا تبدأ وتنتهى بعلامتى الاقتباس) كما يلى:

```
char string[] = "This is a very, very long "
"string that requires two lines.";
```

يُعتبر التعامل مع النصوص من المواضيع المهمّة التي يحتاجها أيّ مبرمج. توفّر سي شارب#C أساليب متقدّمة للتعامل مع النصوص، تم تسهيل مجموعة من الأدوات والدوال

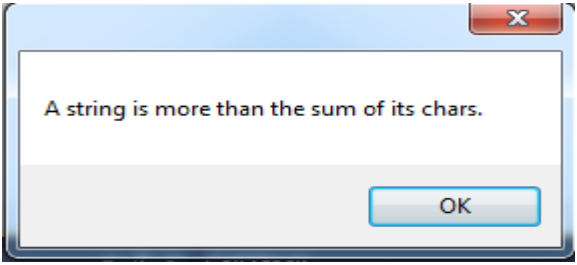
التي تسهل لك التعامل مع النصوص البرمجية. ، وسوف نوضح بعض الأكواد البرمجية و الدوال اللتي تتعلق بالتعامل مع النصوص .

```
const string message4 = "You can't get rid of me!";

        // Use the String constructor only when creating
        // a string from a char*, char[], or sbyte*. See
        // System.String documentation for details.
char[] letters = { 'A', 'B', 'C' };
string alphabet = new string(letters);

string s1 = "A string is more ";
string s2 = "than the sum of its chars.";
```

```
            // Concatenate s1 and s2. This
actually creates a new
            // string object and stores it in s1,
releasing the
            // reference to the original object.
s1 += s2;
System.Console.WriteLine(s1);
            // Output: A string is more than the
sum of its chars.
```



الوصول إلى حرف/ حروف الفردية

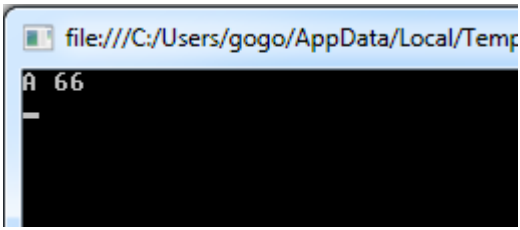
يمكنك استخدام صفيف ذات القيمة فهرس للحصول على القراءة-فقط بالوصول إلى الأحرف الفردية، كـ في المثال التالي :

```
string s5 = "Printing backwards";
for (int i = 0; i < s5.Length; i++)
{
   System.Console.Write(s5[s5.Length - i -
1]);
}
// Output: "sdrawkcab gnitnirP"
```

دوال ترميز الحروف النصية

```
static void Main(string[] args)
      {
         int s1 = 65;
         char c = (char)s1;
         int N = (int)'B';
```

```
        Console.WriteLine(c +" "+ N);
        Console.ReadKey() ;
}
```



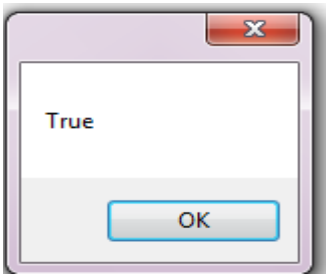
- **Contains**

تستخدم للبحث عن نصوص او حروف ضمن النص المراد البحث عنه، وتعتبر من الدوال القائمَة على خوارزميات البحث المختلفة والتي تم عرضها سابقا.

مثال ١:

```
String S1 = "welcome to c sharp ";
String S2 = " c sharp ";
bool b = S1.contians(S2);
```

```
MessageBox.Show(b. ToString());
```



مثال ٢:

```
private void button1_Click(object sender, EventArgs e)
{
    String S1 = "welcome to c sharp ";

    if (S1.Contains("c sharp"))

        MessageBox.Show("The sequence 'c sharp was found.") ;

}
```



Index Of

وهي تستخدم للبحث عن مكان وجود النص أو حرف.

```
String S1 = "ABC";
int res;
res = S1.IndexOf("B");
MessageBox.Show(res.ToString());
```



للحروف في أيّ نص ترتيب رقمي يبدأ من الصفر ويسمى الدليل .index إذا أردنا أن نبحث عن نص محدّد ضمن نص آخر فيمكن ذلك باستخدام الدالة IndexOf الذي نستدعيه من النص المراد البحث ضمنه، ونمرّر إليه النص المراد البحث

عنه، فيُرجع دليل أوّل حرف للنص المطابق في حال وجوده وإلّا فيرجع -١.

مثال١:

```
string text = "My friend Mohammad is a
developer, Mohammad likes C#.";
int pos = text.IndexOf("Mohammad");
```

عند تنفيذ الشفرة السابقة سيحمل المتغيّر pos القيمة ١٠ والتي تمثّل دليل الحرف M ضمن النص text ربما تكون قد لاحظت أنّ النص Mohammad موجود في النص text مرّتين، فأيّ دليل أعاد الأمر IndexOf؟

تجري عملية البحث باستخدام الأمر IndexOf حسب ترتيب إدخال الحروف في النص text، لذلك فأوّل كلمة Mohammadمطابقة يصادفها تُنهي عمليّة البحث. ففي مثالنا السابق حدثت المطابقة عند كلمة Mohammad الموجودة بعد "My friend".

أمّا إذا استخدمنا كلمة غير موجودة فسنحصل على القيمة -١. انظر الكود التالي :

```
string text = "My friend Mohammad is a
developer, Mohammad likes C#.";
int pos = text.IndexOf("Zyad");
```

سيحمل المتغيّر pos القيمة -١ لأنّ النص الذي نبحث عنه غير موجود في النص text

يخضع الأمر IndexOf في الواقع لزيادة التحميل، حيث أنّ هناك ٩ أشكال مختلفة له. تؤمّن هذه الأشكال المزيد من مزايا البحث، حيث يمكننا مثلًا البحث عن نص ضمن جزء محدّد من النص الموجود في text، أو ابتداءً من دليل محدّد حتى آخر النص، وهكذا .

هناك أمر آخر يسمح لنا بأن نبحث بشكل معكوس، وهو الأمر LastIndexOf لفهم عمله بشكل جيّد سأعيد كتابة الكود السابق ولكن باستخدام الأمر LastIndexOf.

```
string text = "My friend Mohammad is a
developer, Mohammad likes C#.";
int pos = text.LastIndexOf("Mohammad");
```

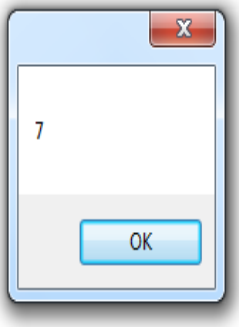
بعد التنفيذ سيحمل المتغيّر pos القيمة ٣٥، وذلك لأنّ المطابقة حدثت هذه المرّة مع كلمة Mohammad الثانية (تأتي قبل كلمة likes) حيث تمثّل القيمة ٣٥ دليل الحرف M لهذه الكلمة بالنسبة للنص text.

تذكر أن الفهرس يبدأ من الصفر.

مثال ٢ :

```
private void button1_Click(object sender, EventArgs e)
{
    String S1 = "ABCDEFGH";
    int res;
    res = S1.IndexOf("H");
    MessageBox.Show(res.ToString());

}
```



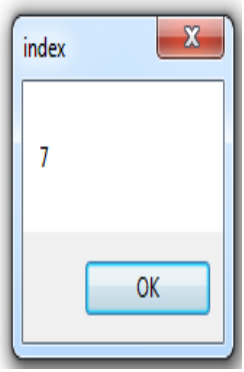
البحث عن حرف أو نص بدا من مكان ما.

مثال ٣:

```
String S1 = "ABC EFG";
int res;
res = S1.IndexOf("E",2);
MessageBox.Show(res.ToString(),"index");
```

مثال ٤:

```
MessageBox.Show(pos.ToString ());
String S1 = "ABCDEFGE";
int res;
res = S1.IndexOf("E", 5);
MessageBox.Show(res.ToString(), "index");
```



البحث عن حرف أو نص مع تحديد مكان بدايته ومكان انتهائه.

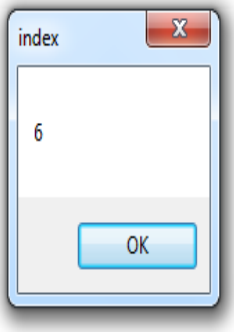
مثال ٥:

```
String S1 = "ABC EFG";
int res;
res = S1.IndexOf("E",3,1);
MessageBox.Show(res.ToString(),"index");
```

يعني أن البحث يتوقف في موقع ٤ لأنه يبدأ من موقع ٣ ويزداد بمقدار واحد.

مثال ٦:

```
String S2 = "ABCDEFEHG";
int res1;
res1 = S2.IndexOf("E", 5, 2);
MessageBox.Show(res1.ToString(), "index");
```



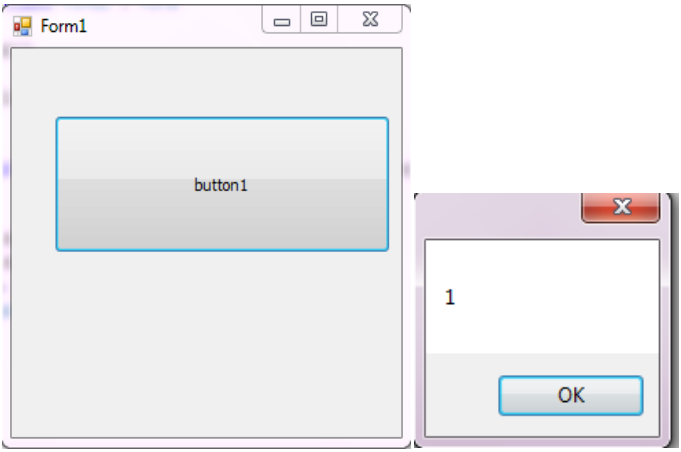
- **دالة Compare**

تستخدم للمقارنة بين نصين .

This method determines the sort order of strings. It checks if one string is ordered before another when in alphabetical order, whether it is ordered after, or is equivalent.

مثال ١:

```
string M1 = "test";
string M2 = "b";
int res = string.Compare(M1, M2);
MessageBox.Show(res.ToString() );
```



لمعرفة نتيجة المقارنة

١	النص الأول أكبر من الثاني
صفر	متساويان
١-	النص الأول أقل من الثاني

مثال٢:

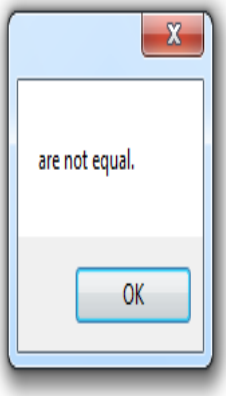
```
private void button1_Click(object sender, EventArgs e)
{
    string str1 = "This is test";
    string str2 = "This is text";

    if (String.Compare(str1, str2) == 0)

        MessageBox.Show(" are equal.");

    else
        MessageBox.Show(" are not equal.");

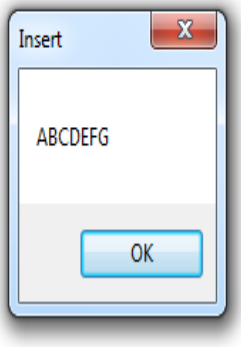
}
```



- **Insert**

لإدراج نص أو حرف داخل String.

```
String ss = "ABFG";
String ress;
ress= ss.Insert(2,"CDE");
MessageBox.Show(ress.ToString(), "Insert");
```



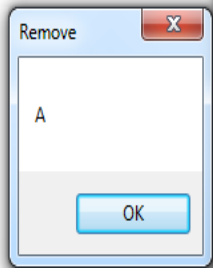

```
        }
    }
}
```

- **Remove**

حذف جزء معين من النص.

```
private void button1_Click(object sender, EventArgs e)
{
    String S1 = "ABFG";
    String res;
    res = S1.Remove(1, 3);
    MessageBox.Show(res.ToString(), "Remove");

}
```



في السطر الثاني استخدمنا الدالة Remove لحذف ثلاثة حروف بدا من حرف B ورقم ٣ يعني عدد الحروف التي يتم مسحه ونتيجة الحذف هي A .

- **Copy**

تستخدم لنسخ النص .

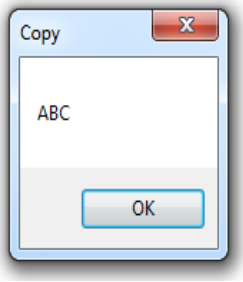
مثال:

```
private void button1_Click(object sender, EventArgs e)
{
    String S1 = "ABC";

    String res = String.Copy(S1);

    MessageBox.Show(res, "Copy");

}
```



Replace

لاستبدال جزء من النص.

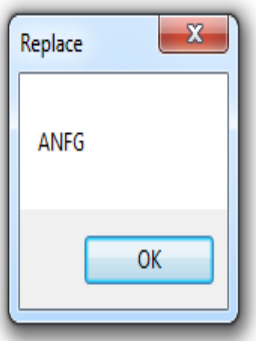
مثال ١:

```
private void button1_Click(object sender, EventArgs e)
{
    String S1 = "ABFG";

    String res;

    res = S1.Replace("B", "N");
    MessageBox.Show(res, "Replace");

}
```



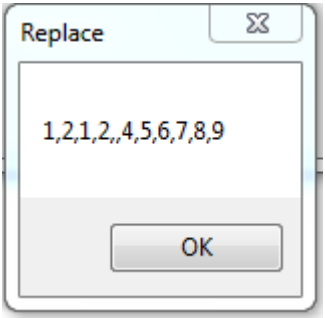
في هذا المثال يتم البحث داخل String عن كل حرف B واستبداله بحرف N.

مثال ٢:

```
        private void button1_Click(object sender, EventArgs e)
        {
 String str = "1 2 1 2  4 5 6 7 8 9";

 MessageBox.Show(str.Replace(' ', ','),"Replace");
```

النتيجة



استخدام الأمر Replace يتطلّب وسيطين: الأوّل oldValueوالذي يمثّل القيمة النصيّة المراد استبدالها، والوسيط الثاني newValue الذي يمثّل القيمة النصيّة الجديدة. يُرجع هذا الأمر قيمة نصيّة تمثّل النص الجديد بعد إجراء عمليّة الاستبدال ، أي أنّ النص الأساسي يبقى دون تغيير. انظر الكود التالي:

```
string text = "My friend Mohammad is a
developer, Mohammad likes C#.";
string newText = text.Replace("Mohammad",
"Zyad");
```

بعد التنفيذ سيحمل المتغيّر newText القيمة النصيّة التالية :

"My friend Zyad is a developer, Zyad likes C#."

وسيبقى النص الأصلي text على حاله دون تغيير .

تدريب

```
string s3 = "Visual C# Express";
System.Console.WriteLine(s3.Substring(7, 2));
// Output: "C#"

System.Console.WriteLine(s3.Replace("C#", "Basic"));
// Output: "Visual Basic Express"

// Index values are zero-based
int index = s3.IndexOf("C");
// index = 7
```

- **ToUpper , ToLower**

يمكنك أن تستخدم الأمر ToUpper من أي متغيّر نصي والذي يعيد حالة الأحرف الكبيرة للنص الموجود ضمن المتغيّر النصي دون أن يؤثّر ذلك على محتوى النص الأساسي ضمن المتغيّر .ونفس الأمر ينطبق على الأمر ToLower والذي يعيد الحالة الصغيرة. يفيد كل من الأمرين السابقين أحيانًا في البحث إذا أردنا عدم التمييز بين الأحرف الكبيرة والصغيرة عند عملية المطابقة بين الكلمات. انظر لكيفيّة استخدام هذين الأمرين :

```
string text = "Hello!";
string upper = text.ToUpper(); //upper will
contains "HELLO!"
string lower = text.ToLower(); //lower will
contains "hello!"
```

مع الانتباه إلى أنّ قيمة المتغيّر text تبقى كما هى.

- **الدالة Trim()**

تستخدم لإزالة مسافات من البداية والنهاية String في بعض الأحيان يقوم المستخدم بإدخال اسم مثلاً في مربع النص مع ترك مسافة في بدايته أو في نهايته. وهذا قد يسبب بعض المشاكل مثلاً البحث عن اسم Nour وهذا موجود في قاعدة بيانات أو مصفوفة عندما يقوم المستخدم بإدخال الإسم بهذه الطريقة (__Nour). فإن قاعدة البيانات لا يجدها، ولحل هذه المشكله نستخدم الدالة Trim .

"Nour" تختلف عن " Nour" أو عن "Nour " .

```
String s= "Nour  ";
MessageBox.Show(s.Trim());
```

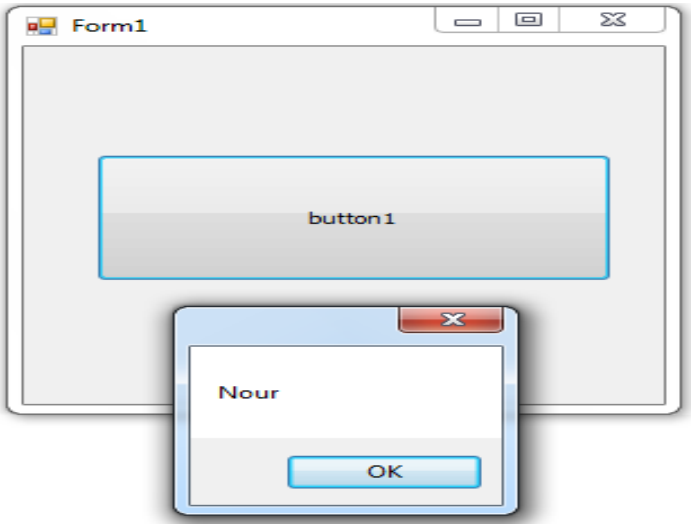
والنتيجة تكون Nour هنا يتم إزالة مسافة من نهاية النص.

يمكن استخدام دالة Trim() لإزالة رمز أوحرف معين.

```
Trim(new char[] trimechar)
```

```
private void button1_Click(object sender, EventArgs e)
{
    String s = "*Nour*";
    MessageBox.Show(s.Trim(new char[] { '*' }));


}
```



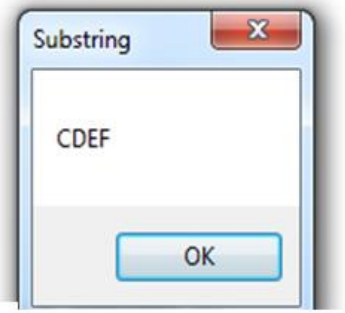
TrimStart() لإزالة المسافات من بداية String فقط

TrimEnd() لإزالة المسافات من نهاية String فقط

- **استخلاص النصوص باستخدام الدالة Substring**

تستخدم لاسترجاع جزء من النص.

```
String st1 = "ABCDEF ";
MessageBox.Show(st1.Substring(2), "Substring")
```

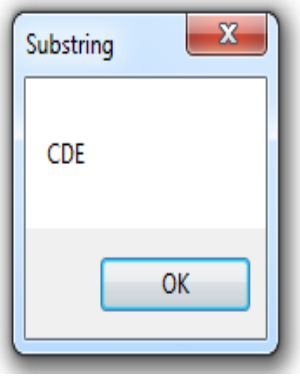


Substring(2) تقوم باسترجاع جزء من النص بداية من الحرف الثاني إلى نهاية النص.

شكل آخر للتعامل مع الدالة

Substring(int startIndex, int Length)

```
String st = "ABCDEF ";
MessageBox.Show(st.Substring(2, 3), "Substring");
```



يمكننا استخلاص جزء من نص باستخدام الدالة SubString والذي يعني نصًّا فرعيًّا أو جزئيًّا. لهذه الدالة شكلان. يسمح الشكل الأوّل باستخلاص نص جزئي ابتداءً من دليل محدّد حتى آخر النص. أمّا الشكل الثاني فيسمح أيضًا باستخلاص نص جزئي ابتداءً من دليل محدّد ولكن بطول محدّد أيضًا. للتمييز بين هاتين الحالتين مثال :

```
string text = "My friend Mohammad is a
developer, Mohammad likes C#.";
```

```
string text1 = text.Substring(24);
```

بعد التنفيذ سيحتوي المتغيّر text1 على النص "developer, Mohammad likes C #.". أمّا إذا استخدمنا الكود التالي:

```
string text = "My friend Mohammad is a
developer, Mohammad likes C#.";

string text1 = text.Substring(24, 9);
```

فسيحتوي المتغيّر text1على النص "developer" فقط. والسبب في ذلك أنّنا بدأنا عمليّة الاستخلاص من الدليل ٢٤ دليل الحرف d وبطول ٩ حروف فقط مما يعني استخلاص الكلمة developer فحسب.

- **Split**

تقوم دالة Split بتقسيم النص الى مجموعه من النصوص ، ويتم التقسيم وفق طريقتين، اما بالحروف او بالنصوص ،

بمعنى اولاً في حالة التقسيم بالحروف ، فإننا نمرر للداله مصفوفه بها الحروف التي نريد التقسيم من خلالها، وستقوم الداله بالبحث عن تلك الحروف بداية من بداية النص، وفي حالة وجدت حرفاً منها فإنها تتوقف، وتقتطع ما سبق من نص قبل هذا الحرف وتضعه كعنصر في المصفوفه التي تخرجها الداله، ثم تبدأ بعد ذلك بالنص الذي يلي الحرف التي توقفت عنده وتكرر نفس العمليه - ونرى هنا ان الحرف التي تتوقف عنده الداله لا يأخذ في الحسبان، لذلك سنجد نصاً خالياً من جميع الحروف التي تقسم من خلالها، وكمثال على ذلك في hello world اذا تم التقسيم وفق حرف o فقط فستنتج لنا مصفوفه بها النصوص التاليه:

hell

w -- شاملة المسافه الفارغة

rld

حيث تم اعتبار حرف o كفاصل لعناصر المصفوفه ، ويمكن ايضاً ان تكون الفواصل اكثر من حرف ، فمثلا اذا استخدمنا حرفي e , o ستكون النتيجه

h

ll

w - شاملة المسافه

rld

ثانياً في حالة التقسيم وفق نصوص معينه ، فبدلا من اعتبار الفاصل كحرف معين , فمثلاً اذا سيتم اعتبار النص كفاصل ، فستكون النتيجه ell كان الفاصل هو

h

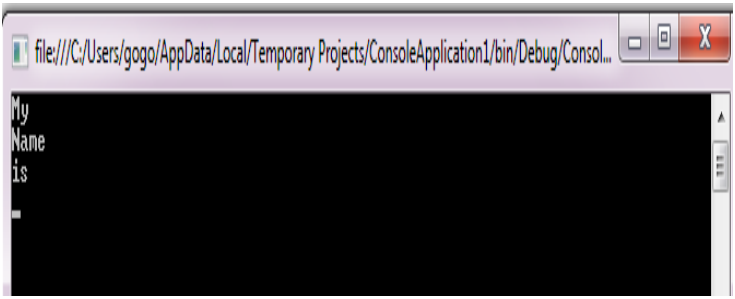
o

world -- شاملة المسافه

مثال١:

```
static void Main(string[] args){
    string statement = "My Name is";
    string[] myWords;
    myWords = statement.Split(' ');
    foreach (string myWord in myWords)
    { Console.WriteLine(myWord); }

    Console.Read();
}
```




```
static void Main(string[] args)
{
    string[] friendNames = { "Mohamed ALmesri",
                             "Nader ALal", "Omar Ahmad"};
    Console.WriteLine("hare are {0} of my Friend", friendNames.Length);
    foreach (string friendName in friendNames )
    {
        Console.WriteLine(friendName);
    }
 }
```

إن الفرق الرئيسي بين Foeach واستخدام الحلقة For استخدام الحلقة

أنها تؤمن لنا وصولا محميا (للقراءة فقط) لمحتويات المصفوفة أى لا يمكننا تعديل محتويات المصفوفة ضمن الحلقة فلا يمكننا

تعديل محتويات المصفوفة ضمن الحلقة فلا يمكننا على سبيل كتابة كود كاتالي:

```
foreach (string friendName in friendNames )
{
    friendName="Gader Amen";
}
```

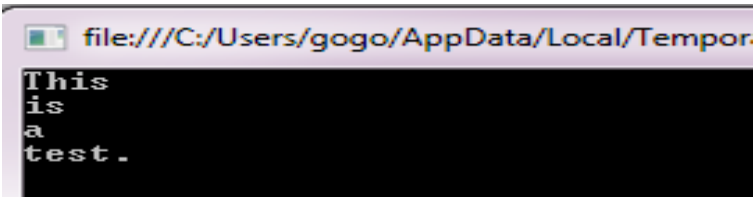
يتسبب هذا الكود في حدوث خطأ من المترجم أما For باستخدام الحلقة

فإنه يمكننا تعديل محتوى المصفوفة كما نريد.

مثال٢:

```
static void Main(string[] args){
    string myString = "This is a test.";
    char[] separator = { ' ' };
    string[] myWords;
    myWords = myString.Split(separator);
    foreach (string word in myWords) { Console.WriteLine(word); }


    Console.Read();
```



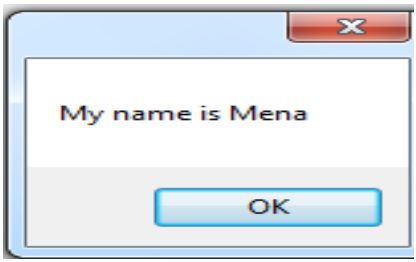
ستقوم هذه الحلقة بالمرور عبر كل عنصر من عناصر المصفوفة

- **دمج النصوص**

✓ يمكن ضمّ النصوص اعتبارًا من نصوص أصغر باستخدام عامل الضم + حيث أنّ استخدامه بسيط. انظر الكود التالي :

```
string text = "My name is " + "Mena";
```

بعد تنفيذ هذه العبارة سيحتوي المتغيّر text على النص :



هناك أمرٌ بسيط لقد أنشأ مترجم سي شارب كائنًا نصيًّا جديدًا ليستوعب النص الجديد، ثم وضع مرجعًا لهذا الكائن ضمن المتغيّر text قد لا يبدو الأمر مقلقًا في حالة مثالنا البسيط هذا، ولكن تخيّل معي ماذا سيحدث في برنامج يتطلّب ضمّ مئات من النصوص مع بعضها (وهذا أمر وارد جدًّا)؟

ستحدث بالتأكيد مشاكل في الأداة، وتمتلأ الذاكرة بمئات الكائنات النصيّة التي لا لزوم لها، والتي تنتظر دورها في التنظيف من قبل جامع النفايات GC الذي يعمل على تحرير ذاكرة تطبيقات دون نت من الكائنات غير المستخدمة في البرنامج .

- ✓ الحل الأمثل في هذه الحالة هو تجنّب استخدام عامل الضم (+)، واستخدام الصنف StringBuilder الموجود ضمن نطاق الاسم System.Text الذي يحل هذه المشكلة بكفاءة عالية. لنستعرض البرنامج التالي لتوضيح هذا الغرض :

```
1       using System;
2       using System.Text;
3
4       namespace Lesson11_01
5       {
6           class Program
7           {
8               static void Main(string[] args)
9               {
10                  StringBuilder sb = new StringBuilder();
11                  string[] arrText = {
12                                      "C# is a powerful language.",
13                                      "It contains advanced features
14                                      "It makes programming tasks mo
15                                      "C# is the main programming la
16                                      };
17
18                  foreach(string text in arrText)
19                  {
20                      sb.Append(text);
21                  }
22
23                  Console.WriteLine(sb.ToString());
24              }
25          }
26      }
```

أنشأنا المتغيّر sb من النوع StringBuilder وأسندنا إليه كائنًا من هذا النوع في السطر ١٠. بعد ذلك أنشأنا المصفوفة arrTextبطريقة مختصرة في السطر ١١. ثمّ استخدمنا حلقة For-Each للمرور على جميع عناصر المصفوفة arrTextوإضافتها واحدًا تلو الآخر إلى المتغيّر sb من

خلال الأمر Append الذي يتطلّب وسيطًا نصيًّا واحدًا (السطر ٢٠). أخيرًا يعمل الأمر WriteLine في السطر ٢٣ على طباعة المحتوى النصيّ الناتج من ضمّ جميع النصوص التي أضفناها باستخدام الأمر Append وذلك من خلال استدعاء التابع ToString من المتغيّرsb ، ستلاحظ بعد تنفيذ البرنامج أنّ النص المعروض سيكون مضمومًا كما لو استخدمنا عامل الضم (+) على الشكل التالي :

C# is a powerful language.It contains advanced features.It makes programming tasks more easier.C# is the main programming language in .NET world.

كان من الممكن استخدام أمر آخر من المتغيّر sb في السطر ٢٠ وهو الأمر .AppendLine يشبه هذا الأمر Append تمامًا باستثناء أنّه يضيف سطر جديد \n تلقائيًّا إلى كلّ نص جديد تتم إضافته .

ملاحظة

استخدمنا في السطر ١١ أسلوبًا مختصرًا في إنشاء المصفوفات. يمكن استخدام هذا الأسلوب عندما تكون عناصر المصفوفة معروفة ومحدّدة. للأسلوب المختصر الشكل التالي بالنسبة للمصفوفات النصيّة :

```
string[] arrText = {"Text_1", "Text_2",
"Text_3", … ,"Text_n"};
```

ويمكن بالطبع استخدام هذا الأسلوب المختصر مع أيّ نوع مصفوفة آخر .

- ✓ باستخدام الدالة **join** يمكننا دمج النصوص

مثال ١:

```
string[] s = { "welcome", "to", "c#" };
string s1 = string.Join(" ", s);
MessageBox.Show(s1, "Join");
```

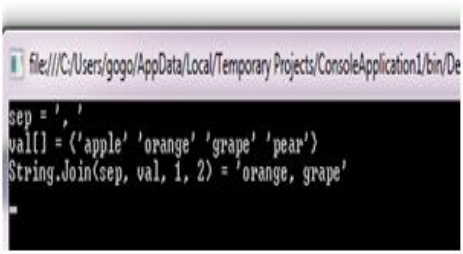


مثال ٢ :

```
static void Main(string[] args)
{
    String[] val = { "apple", "orange", "grape", "pear" };
    String sep = ", ";
    String result;

    Console.WriteLine("sep = '{0}'", sep);
    Console.WriteLine("val[] = {{'{0}' '{1}' '{2}' '{3}'}}", val[0], val[1], val[2], val[3]);
    result = String.Join(sep, val, 1, 2);
    Console.WriteLine("String.Join(sep, val, 1, 2) = '{0}'", result);
    Console.Read();

}
```



مثال ٣:

```
private void button13_Click(object sender, EventArgs e)
{

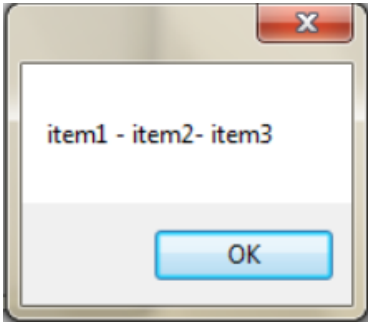
    string lineOfText = "item1, item2, item3";
    string[] wordArray = lineOfText.Split(',');

    string joinedText;

    joinedText = String.Join("-", wordArray);

    MessageBox.Show(joinedText);

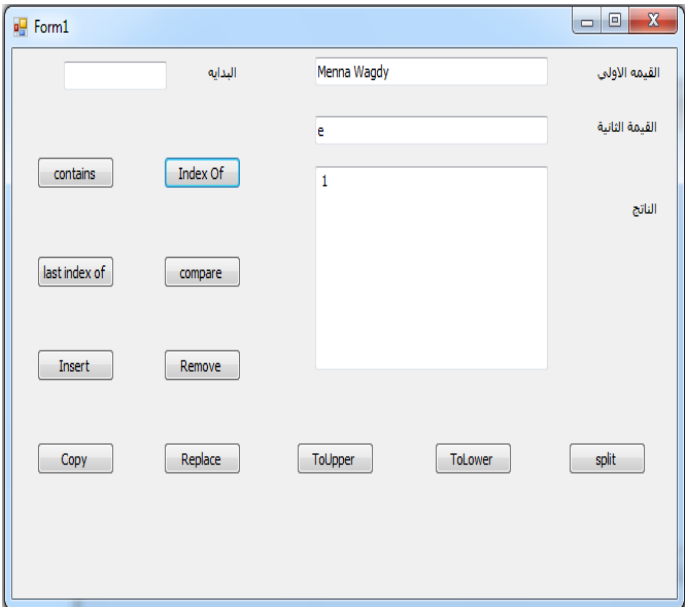
}
```



تدريبات متنوعة على الدوال

Index of :

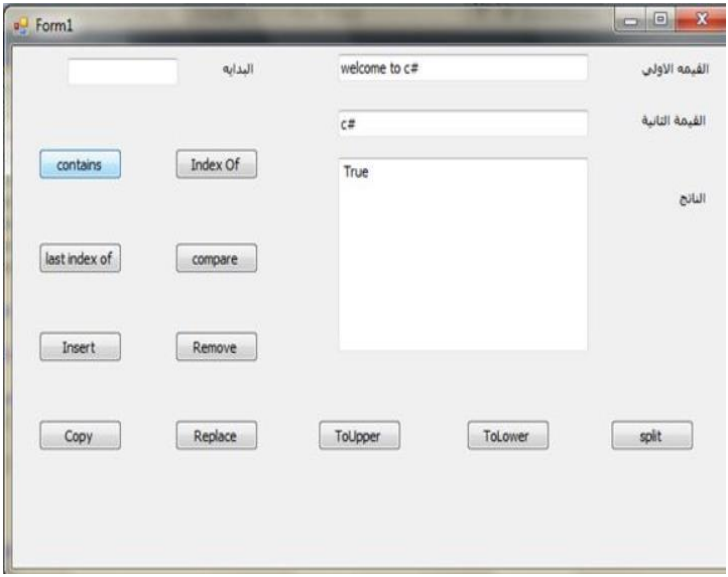
```
private void button2_Click(object sender, EventArgs e)
    {
      string s = textBox1.Text;
      int x = textBox1.Text.IndexOf(textBox2.Text);
      textBox4.Text = (x.ToString());
    }
```



Contains :

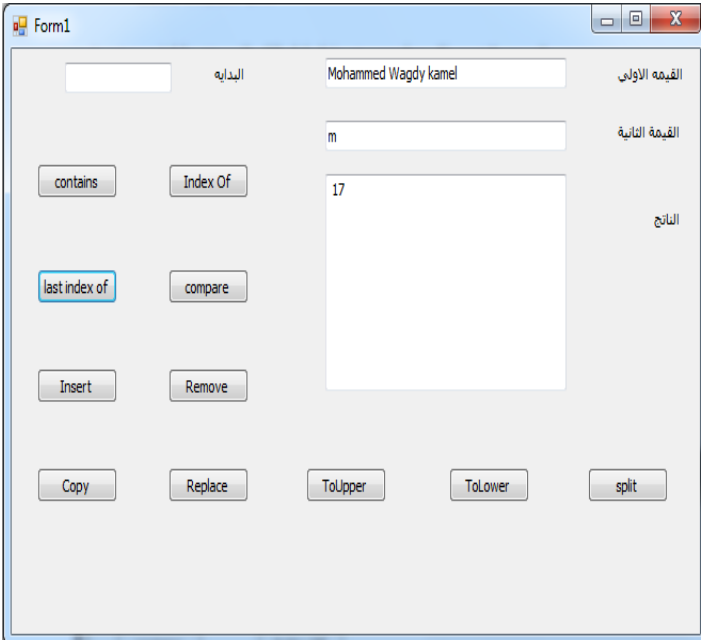
```
private void button1_Click(object sender, EventArgs e)
    {
       string s1 = textBox1.Text;
       string s2 = textBox2.Text;
       bool b = s1.Contains(s2);
       textBox4.Text=(b.ToString());

    }
```



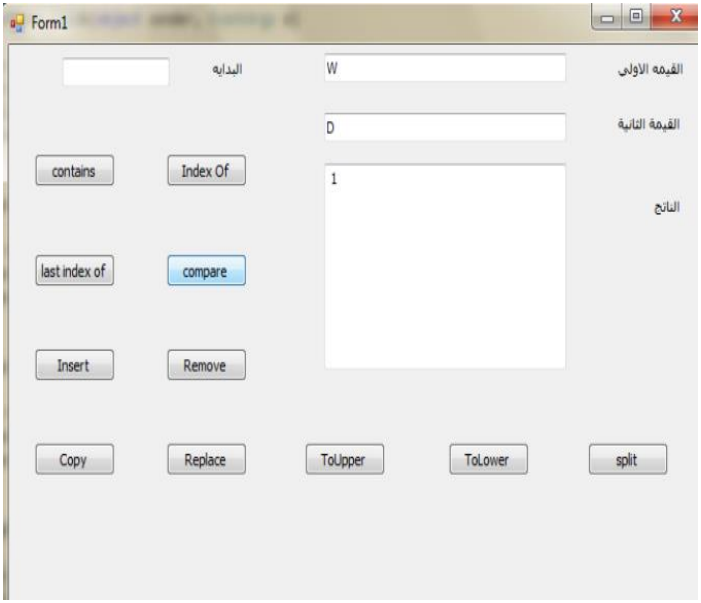
Last index of:

```
private void button3_Click(object sender, EventArgs e)
    {
      string s = textBox1.Text;
      int x = textBox1.Text.LastIndexOf(textBox2.Text);
      textBox4.Text = (x.ToString());
    }
```



Compare:

```
private void button4_Click(object sender, EventArgs e)
    {
       string s1 = textBox1.Text;
       string s2 = textBox2.Text;
       int x = string.Compare(s1, s2);
       textBox4.Text = (x.ToString());
    }
```

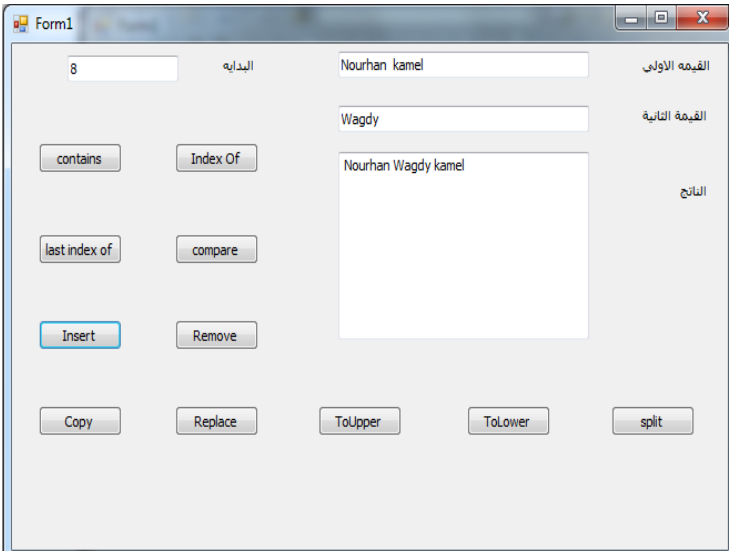


Insert:

```
private void button5_Click(object sender, EventArgs e)
    {
      string s = textBox1.Text;

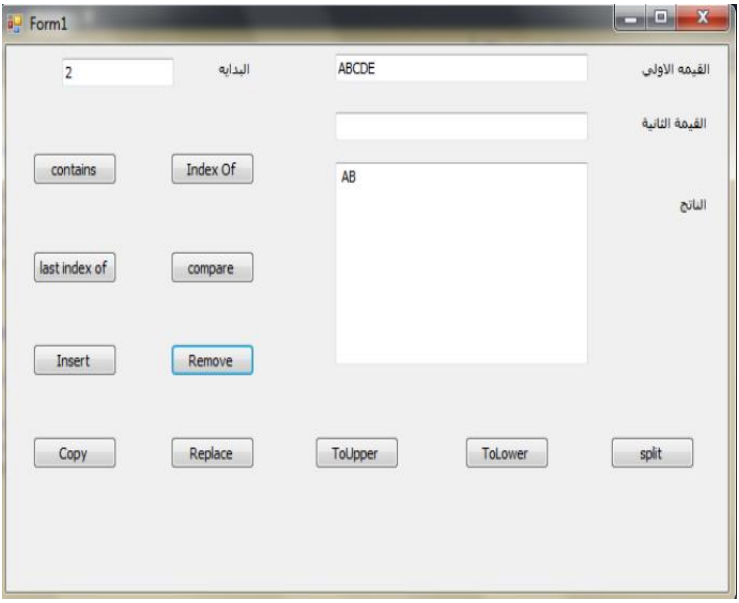
      int y = int.Parse(textBox3.Text);
      string x = s.Insert(y, textBox2.Text);
      textBox4.Text = (x.ToString());


    }
```



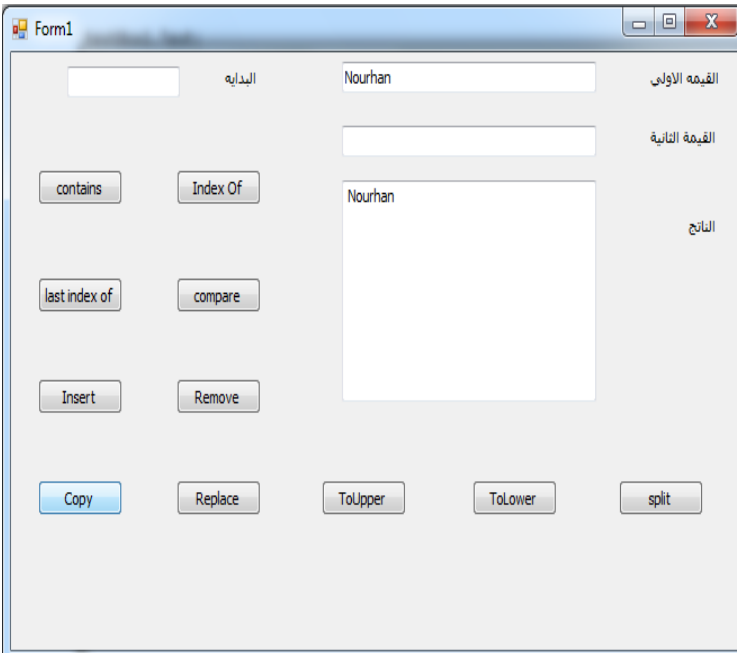
Remove:

```
rivate void button6_Click(object sender, EventArgs e)
    {
        string s = textBox1.Text;
        int x = int.Parse(textBox3.Text);
        string res= s.Remove((x));
        textBox4.Text = (res.ToString());
    }
```



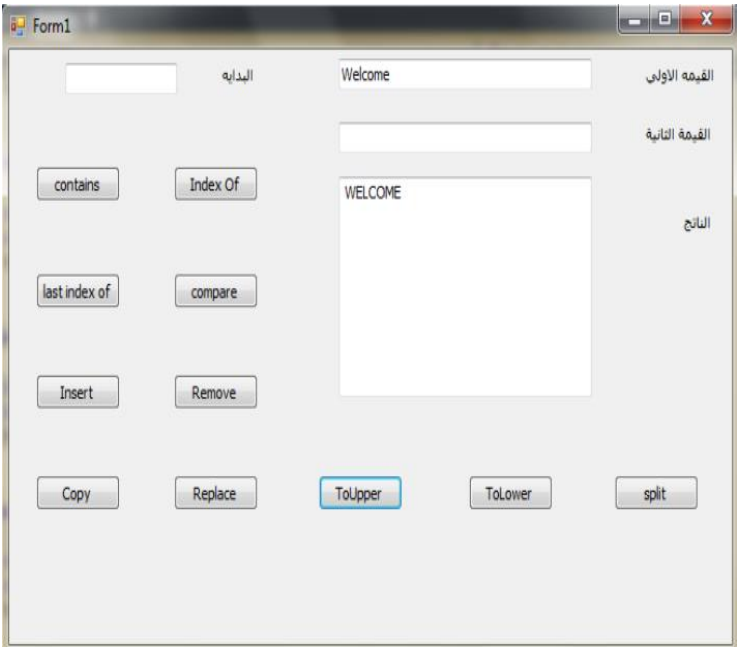
Copy:

```
private void button7_Click(object sender, EventArgs e)
    {
      string s1 = textBox1.Text;
      string res = string.Copy(s1);
      textBox4.Text = (res);
    }
```



ToUpper:

```
private void button9_Click(object sender, EventArgs e)
    {
      string x = textBox1.Text;
      string upper = x.ToUpper();
      textBox4.Text = (upper.ToString());
    }
```



تدريب ١

ليكن لدينا النص التالي :

"black cat and white dog are friends. black cat always brings food to dog, and white dog thanks it."

استبدل كل كلمة white في النص بكلمة brown واعرض النص المعدّل على الشاشة .

```
            string s1 = " Black cat and white dog are friends
. Black cat always bring food to dog and white dog thanks it";
            string res;
            res = s1.Replace("white", "brown");
            Console.WriteLine(res);
            Console.ReadKey();
```



طور البرنامج السابق ليحسب عدد الكلمات black وعدد الكلمات dog الموجودة في النص.

تدريب٢

اكتب برنامجًا يطلب من المستخدم ادخال نص. ثم أعدّ تقريرًا يتضمّن إحصائيّات حول عدد كل كلمة أدخلها المستخدم، واعرض هذا التقرير على الشاشة .

ملحوظة :ربما تحتاج إلى شكل مختلف للأمر IndexOf يتطلّب وسيطين: الأوّل هو النص المراد البحث عنه، والثاني هو حرف الدليل الذي سيبدأ البحث اعتبارًا منه. في أمثلتنا السابقة كنّا نبحث افتراضيًّا من الحرف ذو الدليل صفر.

تدريب٣:

```
class ReplaceSubstrings{
    string searchFor;
    string replaceWith;
    static void Main(string[] args){
```

```
        ReplaceSubstrings app = new
ReplaceSubstrings();
        string s = "The mountains are behind
the clouds today.";
        // Replace one substring with another
with String.Replace.
        // Only exact matches are supported.
        s = s.Replace("mountains", "peaks");
        Console.WriteLine(s);
        // Output: The peaks are behind the
clouds today.

        // Use Regex.Replace for more
flexibility.
        // Replace "the" or "The" with "many" or
"Many".
        // using
System.Text.RegularExpressions
```

```
        app.searchFor = "the"; // A very simple
regular expression.
        app.replaceWith = "many";
        s = Regex.Replace(s, app.searchFor,
app.ReplaceMatchCase,
RegexOptions.IgnoreCase);
        Console.WriteLine(s);
        // Output: Many peaks are behind many
clouds today.
        // Replace all occurrences of one char
with another.
        s = s.Replace(' ', '_');
        Console.WriteLine(s);
        // Output:
Many_peaks_are_behind_many_clouds_tod
ay.
        // Remove a substring from the middle
of the string.
```

```
        string temp = "many_";
        int i = s.IndexOf(temp);
        if (i >= 0)  {
            s = s.Remove(i, temp.Length);       }
        Console.WriteLine(s);
        // Output:
Many_peaks_are_behind_clouds_today.

        // Remove trailing and leading
whitespace.
        // See also the TrimStart and TrimEnd
methods.
        string s2 = "    I'm wider than I need to
be.      ";
        // Store the results in a new string
variable.
        temp = s2.Trim();
        Console.WriteLine(temp);
```

```
        // Output: I'm wider than I need to be.
        // Keep the console window open in
debug mode.
        Console.WriteLine("Press any key to
exit");
        Console.ReadKey();    }
```

تدريب ٤:

```
class ModifyStrings{
   static void Main()    {
        string str = "The quick brown fox
jumped over the fence";
        System.Console.WriteLine(str);
        char[] chars = str.ToCharArray();
        int animalIndex = str.IndexOf("fox");
        if (animalIndex != -1)        {
            chars[animalIndex++] = 'c';
            chars[animalIndex++] = 'a';
            chars[animalIndex] = 't';        }
```

```
        string str2 = new string(chars);
        System.Console.WriteLine(str2);
        // Keep the console window open in
debug mode
        System.Console.WriteLine("Press any
key to exit.");
        System.Console.ReadKey();    }}
/* Output:
  The quick brown fox jumped over the fence
  The quick brown cat jumped over the fence
*/
```

تدريب٥:

```
class TestStringSplit{
    static void Main()    {
        char[] delimiterChars = { ' ', ',', '.', ':', '\t'
};
        string text = "one\ttwo three:four,five six
seven";
```

```
        System.Console.WriteLine("Original
text: '{0}'", text);
        string[] words =
text.Split(delimiterChars);
        System.Console.WriteLine("{0} words
in text:", words.Length);
        foreach (string s in words)        {
            System.Console.WriteLine(s);        }
        // Keep the console window open in
debug mode.
        System.Console.WriteLine("Press any
key to exit.");
        System.Console.ReadKey();    }}
/* Output:
   Original text: 'one     two three:four,five six
seven'
   7 words in text:
   one
```

```
   two
   three
   four
   five
   six
   seven
 */
```

تدريب ٦:

```
class Program   {
      static void Main(string[] args)      {
         string firstname;
         string lastname;
         firstname = "Steven Clark";
         lastname = "Clark";
  Console.WriteLine(firstname.Clone());
// Make String Clone
         Console.WriteLine(firstname.Compar
```

```
eTo(lastname));
//Compare two string value and returns 0 for
true and
1 for false

Console.WriteLine(firstname.Contains("ven")
); //Check whether specified value exists or
not in string
   Console.WriteLine(firstname.EndsWith("n"
)); //Check whether specified value is the
last character of string
            Console.WriteLine(firstname.Equals(
lastname));
//Compare two string and returns true and
false
   Console.WriteLine(firstname.GetHashCod
e());
```

```
//Returns HashCode of String
   Console.WriteLine(firstname.GetType());
//Returns type of string
   Console.WriteLine(firstname.GetTypeCod
e());
//Returns type of string
   Console.WriteLine(firstname.IndexOf("e"));
//Returns the first index position of specified
value
the first index position of specified value
   Console.WriteLine(firstname.ToLower());
//Covert string into lower case
  Console.WriteLine(firstname.ToUpper());
//Convert string into Upper case
   Console.WriteLine(firstname.Insert(0,
"Hello")); //Insert substring into string
```

```
  Console.WriteLine(firstname.IsNormalized(
));
//Check Whether string is in Unicode
normalization
from C
    Console.WriteLine(firstname.LastIndexOf(
"e")); //Returns the last index position of
specified value
 Console.WriteLine(firstname.Length);
//Returns the Length of String
 Console.WriteLine(firstname.Remove(5));
//Deletes all the characters from begining to
specified index.
 Console.WriteLine(firstname.Replace('e','i'))
; // Replace the character
  string[] split = firstname.Split(new char[] {
'e' }); //Split the string based on specified
```

```
value
            Console.WriteLine(split[0]);
           Console.WriteLine(split[1]);
           Console.WriteLine(split[2]);
  Console.WriteLine(firstname.StartsWith("S"
)); //Check wheter first character of string is
same as specified value
   Console.WriteLine(firstname.Substring(2,5
));
//Returns substring
   Console.WriteLine(firstname.ToCharArray(
));
//Converts an string into char array.
   Console.WriteLine(firstname.Trim());
//It removes starting and ending white
spaces from
string.
```

```
            }
    }
}
```

Output

```
Steven Clark
1
True
False
False
1470518261
System.String
String
2
steven clark
STEVEN CLARK
HelloSteven Clark
True
4
```

12

Steve

Stivin Clark

St

v

n Clark

True

even

Steven Clark

Steven Clark

_

الفصل الثالث

تراكيب البيانات

مقدمه

تراكيب البيانات هي بناء منطقي كبير من عناصر البيانات أو مجموعات منظمة من عناصر البيانات تعامل كوحدة واحدة، مثل المصفوفات أو السجلات، تستخدم من أجل تسهيل بناء تطبيقات محددة مثل تخزين معلومات طلبة أو تخزين معلومات زبائن أو طلبات للبيع و الشراء أو معالجة لبيانات مستند ما، و غير ذلك.

علم تراكيب البيانات هو من العلوم الغير أساسية في عالم البرمجة، بمعنى يمكنك تطوير البرمجيات دون الحاجة له، و لكن!! ظهرت الحاجة لعلم تراكيب البيانات بهدف زيادة جودة البرمجيات من حيث سرعة التنفيذ، و تقليل كمية الذاكرة المستخدمة.

البيانات (Data) هي مجموعة من الحقائق التي من خلالها يمكن الحصول على استنتاج ما. أنواع البيانات نصية Textualمثل :اسمك الشخصي (محمد)، عددية Numeric مثل :الرقم الجامعي (١٢٣٤٥٦٧)، صوتية Audio مثل : صوتك الشخصي أو أي صوت لأي كائن آخر.

كما يمكن تعريف البيانات على أنها مجموعة من القيم وظيفتها التعبير عن الأحداث التي تعبر عنها وبالتالي فهي مختلفة فيما بينها من حيث نوع العمليات التي تجرى عليها حسابية أو منطقية أو غير ذلك.

بناء على ذلك يمكن القول...

١- أن لدينا بيانات عددية عندما نتعامل مع الأرقام.

٢- بيانات رمزيه عندما نتعامل مع الحروف والكلمات.

٣- بيانات منطقية عندما نتعامل مع بيانات تحتمل اجابات الصواب والخطأ.

وهذه الأنواع المذكورة أعلاه تسمى **"أنماط بيانية بسيطةSimple Data Types"**.

ويوجد هناك العديد من الحقائق المختلفة التي يصعب متابعتها والإحاطة بها وبجميع أجزاءها أحياناً وهذا لتعدد البيانات التي ينطوي عليه وصف هذه الحقائق مما يدفعنا إلى تنظيم هذه البيانات في مجموعات، قوائم، جداول أو سجلات أو غير ذلك من أساليب التنظيم ويطلق على هذا النوع من البيانات باسم **"أنماط بيانية مركبة Structured Data Types"**.

ويمكن تلخيص ما ذكرناه في الجدول التالي:

أنماط بسيطة	أنماط مركبة
Boolean	Record
Character	Set
Integer	Array
Real	String
	File

وعلى هذا الأساس تعتبر تراكيب البيانات Data Structures مجرد طريقة محددة ومعروفة لتنظيم والتعامل مع البيانات في الكمبيوتر للوصول للمعلومة المخزنة بطريقة سريعة . هي عبارة عن اوعية "حاويات" للبيانات، تتبع طرق معينة لتخزين البيانات فيها والمقصود بالوعاء هو مساحة تخرين.

أنواع هياكل البيانات

تقسم هياكل البيانات إلى عدة أنواع يناظر كل منها أسلوباً خاصاً في تصميم البرامج والمعالجة الالكترونية ويمكن إدراجها تحت ثلاثة أنواع :

١. هياكل بيانات خطية.
٢. هياكل بيانات شجرية.
٣. هياكل بيانات شبكية.

- هياكل البيانات الخطية أي ترص البيانات داخل الذاكرة على خط واحد وتشمل:

١) المصفوفات (Arrays).
٢) القوائم الموصولة المرتبطة (Linked Lists)
٣) السجلات والملفات (الجداول)(الكيانات Structures)
٤) المكدسات (المرصوصات Stacks)
٥) قوائم الإنتظار (الطوابير Queues)

أولاً / المصفوفات:

وأول مثال لهياكل البيانات هي المصفوفات Arraysفالمصفوفة هي عبارة عن حاوية للبيانات " حجمها

ثابت "، ويجب ان يكون معلوم في وقت الترجمة، وجميع محتوياتها من نفس النوع.

تأتي المصفوفة أصلاً من الرياضيات، حيث تُعرف بأنها مجموعة من العناصر لها خصائص متشابهة مُرتبة غالباً في صفوف ولها ترقيم محدد يمكنك من الوصول لأي عنصر عبر رقمه. نفس هذا التعريف ينطبق برمجياً، بما إن البرمجة قدمت من خلفية رياضية بحتة فهي تستقي كثيراً من القواعد الرياضية في بنيتها فالمصفوفة في البرمجة هي سلسلة من العناصر المتشابهة قد تكون رقماً صحيحاً أو حرفاً أو حتى عنصراً جديداً مثل العنصر student ، ترتبط هذه العناصر بترقيم وأرشفة تبدأ من الرقم صفر وتنتهي بطول المصفوفة ناقص واحد (عدد عناصر المصفوفة – ١) وذلك بسبب بدء العد من الصفر لا الواحد. وفيما يلي سوف نتفهم طبيعة المصفوفات بالتفصيل.

ثانياً /القوائم الموصولة:

هي هيكل بياني يحتوي على بيانات من نفس النوع البياني ويتم تخزينها في الذاكرة في أماكن متفرقة بحيث يشير كل عنصر إلى العنصر الذي يليه وعدد عناصرها يتحدد حسب الاحتياج.

ثالثاً /السجلات:

هي عبارة عن تجميع لمجموعة من البيانات من أنواع بيانية مختلفة داخل وحدة واحدة.

```
struct  student
  {
    int  No;

long  UID;
char  Name
[10];                                    };
```

رابعاً /المكدسات:

هي عبارة عن نموذج أو إطار خاص لحفظ البيانات في الذاكرة بشكل مؤقت بحيث يتم إدخال وإخراج البيانات بطريقة الداخل آخراً خارج أولاً أو الداخل أولاً خارج آخراً .

Last Out First In (LOFI) First In Last Out (FILO)

Last In First Out (LIFO) First Out Last In (FOLI)

خامساً /قوائم الإنتظار:

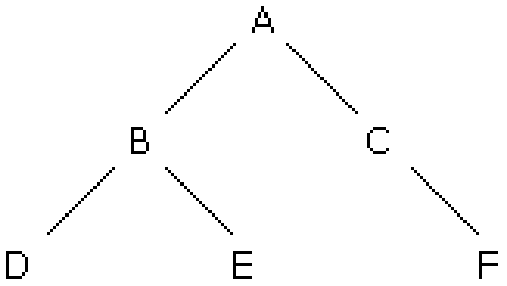
هي عبارة عن نموذج أو إطار خاص لحفظ البيانات في الذاكرة بشكل مؤقت بحيث يتم إدخال وإخراج البيانات بطريقة الداخل أولاً خارج أولاً .

Last In Last Out (LILO) First In First Out (FIFO)

سادساً /الشجيرات:

هي هيكل بياني يحتوي على بيانات من أنواع بيانية مختلفة ويتم تخزينها في الذاكرة في أماكن متفرقة بحيث يمكن أن يشير كل عنصر إلى أكثر من عنصر وعدد العناصر يتحدد حسب الاحتياج.

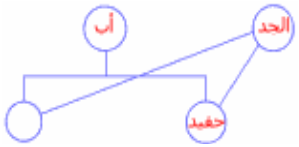
يشبه الهيكل التنظيمي لإحدى الشركات حيث مدير الشركة على رأس التنظيم ثم يمتد خط النائب بعدها يتفرع التنظيم الإداري والأفرع والأقسام.

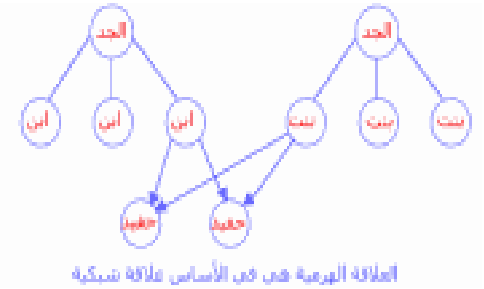


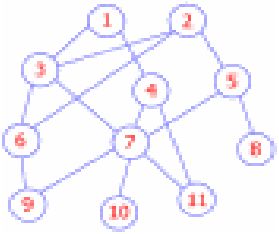
هياكل البيانات الشبكية

الشبكات (الرسوم Graphs)كما تسمى بيانات مضفرة.

إذا اتصل أي عنصر بيان في المستوى الأدنى من هياكل البيانات الشجرية بأكثر من عنصر في مستوى أعلى فيطلق عليه اسم هياكل بيانات شبكية، حتى شجرة العائلة من النوع الشبكي وليست من النوع الهرمي . تنقسم هياكل البيانات الشجرية إلى نوعين، بسيط ومعقد، ففي النوع البسيط يمكن تحديد مستويات الهيكل البنائي للبيانات أما النوع المعقد فيصعب ذلك كثيراً .







المصفوفات كأحد تراكيب البيانات

المصفوفات هي أحد أهم تراكيب البيانات وأكثرها شيوعاً واستخداماً، ففي السابق عندما كنا نريد أن نخزن درجات خمس

طلاب على سبيل المثال فإننا نقوم بتعريف خمس متغيرات،ولكن عندما كنا بصدد أرقام كبيرة كدرجات طلاب الجامعات فهي بمئات الآلاف وعندها ظهرت الحاجة إلى ابتكار ما يسمى بالمصفوفات، فبدلاً من أن نعرف مئة الف متغير فإننا نقوم بإنشاء مصفوفة حجمها مئة ألف لنحفظ البيانات بها ويمكن الرجوع لها بسهولة عند الحاجة لها.

خصائص المصفوفات:

١- تحمل عناصر من نفس النوع فقط.

٢- تحجز مكاناً في الذاكرة بمجرد إنشاءها.

٣- حجمها ثابت Static ولا يمكن تغييره بمجرد انشاءها.

٤- يمكن الوصول لعناصرها مباشرة.

٥- المصفوفات تعتبر كائنات Objects .

٦- يمكن نسخها بواسطة الطريقةObject.clone() .

٧- تنتج الخطأ ArrayIndexOutOfBoundsException عندما يشار إلى عنصر خارج مجالها.

العمليات التي نقوم بها على المصفوفات

هنا سنتحدث عن سرعة العمليات التي نقوم بها على المصفوفات وسهولتها،

١- الوصول للعناصر Accessing Elements

الوصول للعنصر في المصفوفة يعتبر لحظي وسريع جداً فبمجرد كتابة رقم الـ Index فقد اصبح بإمكانك تعديل وتحرير العنصر.

٢- إضافة العناصر Inserting Elements

إضافة العناصر للمصفوفة يملك إحدى حالتين إما تضيف في البداية أو المنتصف وهنا تكون العملية بطيئة جداً وتحتاج لإزاحة كافة العناصر.Shifting

أما إضافة العناصر في آخر المصفوفة فهو لحظي وسريع ولا يحتاج إلى عمل.Shifting

٣- حذف العناصر Removing Elements

حذف العناصر في المصفوفة هو نفس حالة الإضافة، فإذا كان الحذف من بداية أو منتصف المصفوفة فإنه يكون بطيء ويجب عمل Shift لباقي العناصر.

أما حذف عنصر من آخر المصفوفة فهو لحظي وسريع لأنه لا

يحتاج إلى عمل.Shift

٤- البحث عن العناصرSearching for Elements

البحث عن العناصر يعتمد على الـ Algorithm المستخدمة، وسنتحدث عنها لاحقاً.

المصـــفوفة (ومعناهـــا MATRIX أو ARRAY أو CLASS) هي عبارة عن هيكل بياني موجود في جميع لغات البرمجة ويتم تمثيله في الحاسب بمجموعة من مواقع الذاكرة المتجاورة (يجب إن تكون متجاورة) ويكون لجميع عناصر المصفوفة نفس النوع البياني لكن قيمها تختلف من عنصر إلى أخر.

تقسم المصفوفات من حيث تعاملها مع الذاكرة إلى:-

- مصفوفات المتغيرات.
- مصفوفات أدوات التحكم.
- المصفوفات الديناميكية.

إما من حيث إبعاد المصفوفة فأنها تقسم إلي:

- المصفوفات المتجهة (أو المصفوفات الأحادية).
- المصفوفات متعددة الأبعاد (ثنائية ، ثلاثية ، ..... الخ).

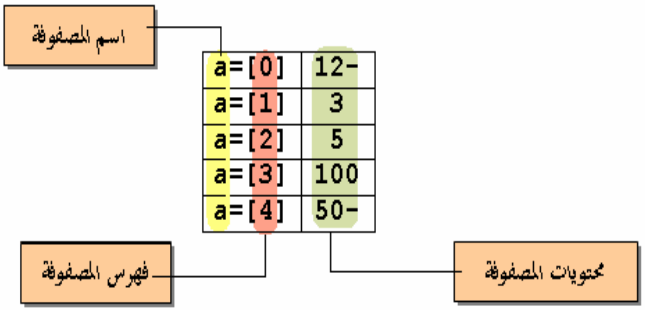
إنشاء المصفوفة

يتم إنشاء تصريح المصفوفات قي البرنامج مثلما يتم تصريح المتغيرات لكن المكان الذي تصرح فيه المصفوفة يحدد مكان استعمالها أو ما يسمى(مجالها) DOMAIN في البرنامج.

اذا تم تصريح المصفوفة محلياً ، يمكن استعمالها في الأجراء الذي صرحته فيه فقط .إما إذا تم تصريحها عموماً في وحده نمطية قياسية فيمكن استعمالها في أي مكان في البرنامج.

و تتميز المصفوفات عن بقية بنى المعطيات الأخرى مثل القوائم list و الأشجارtrees و غيرها بأن لها حجم ثابت طيلة فترة تنفيذ البرنامج.

و المصفوفة هي عبارة عن مجموعة من خانات الذاكرة المتتالية التي لها نفس الاسم و نفس النوع , ومن أجل الرجوع إلى خانة معينة من هذه الخانات ضمن المصفوفة يكفي أن نستخدم اسم المصفوفة و رقم موضع الخانة ضمن المصفوفة الفهرس index الشكل التالي يوضح مصفوفة من الأعداد الصحيحة اسمها a و هي تتضمن خمسة عناصر:



من الواضح أن الفهرس يبدأ من الصفر و ليس من الواحد و هذه نقطة مهمة جدا فمثالا العنصر الرابع هو a[3].

تعتبر المصفوفات في السي شارب صنف class ذا مواصفات معينة تسمى System.Array .

الإعلان عن المصفوفات:

بما أن المصفوفات من النوع المرجعي فهي تأخذ خطوتين قبل استخدامها، الخطوة الأولى: هي الإعلان عنها وفي هذه الخطوة نقوم بإنشاء المرجع و تتم في المصفوفة هكذا:

اسم المصفوفة [ ] نوع عناصر المصفوفة

```
int [ ] employee ;
```

للإنشاء مصفوفة باسم employee تكون عناصرها من نوع int.

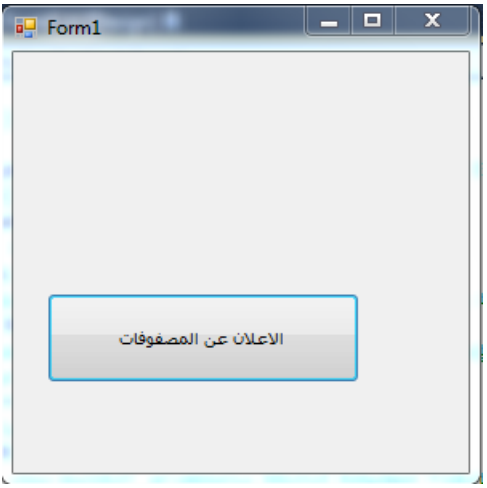
```
string [ ] B;
```

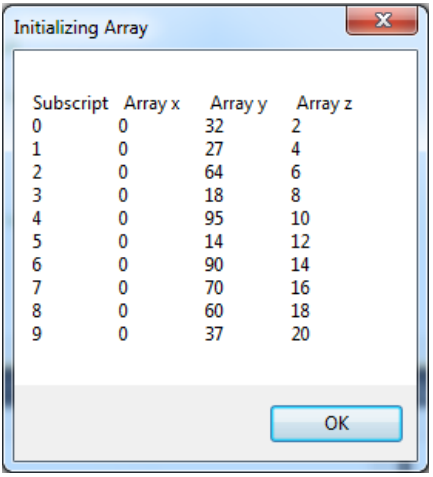
لتكوين مصفوفة باسم B تكون عناصرها سلاسل حرفية.

الخطوة الثانية: وهي تحديد و حجز مقدار معين من الذاكرة للمصفوفة , بما أن المصفوفة من خصائصها أنها ثابتة الحجم فيجب تحديد عدد عناصر المصفوفة في هذه الخطوة , و لفعل ذلك يمكنك استخدام رقم أو ثابت أو أن تضع عناصر المصفوفة مباشرة , المثال التالي يوضح لك كيفية إنشاء مصفوفات بثلاث طرق , و بعد ذلك يقوم بطباعتهن.

```
string output;
//الأولى المصفوفة عن بالإعلان قمنا
int [] x;
//الافتراضية بقيمتها عناصر لعشرة الذاكرة بحجز قمنا
x = new int[10];
//عناصرها بإسناد قمنا و الثانية المصفوفة بإنشاء قمنا
//مباشرة وقيمها
int []y = { 32,27,64,18,95,14,90,70,60,37};
```

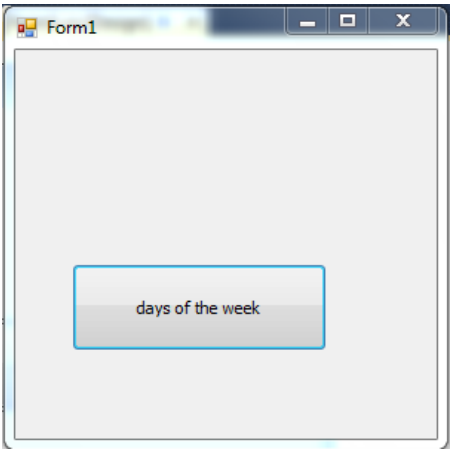
```
// ثابت أنشأنا//
const int ARRAY_SIZE = 10;
// العدد بدل الثابت باستخدام ثالثة مصفوفة بإنشاء قمنا//
int []z = new int[ARRAY_SIZE];
// الثالثة المصفوفة عناصر قيم بوضع قمنا//
for (int i=0;i< z.Length;i++)
z[i] = 2+2*i;
output = "Subscript \t Array x \t Array y "
+ "\t Array z \n";
// المصفوفات عناصر كل إظهار//
for (int i =0; i < ARRAY_SIZE; i++)
output += i + "\t"+ x[i]+ "\t" + y[i] +
"\t" + z[i] + "\n";
MessageBox.Show(output,"Initializing
Array");
```

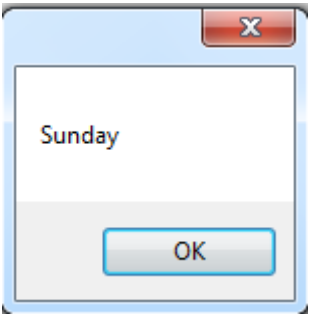

Form1
الاعلان عن المصفوفات


Initializing Array
Subscript Array x Array y Array z
0 0 32 2
1 0 27 4
2 0 64 6
3 0 18 8
4 0 95 10
5 0 14 12
6 0 90 14
7 0 70 16
8 0 60 18
9 0 37 20
OK

مثال ١:

```
String [] week ;
        week=new string[7];
    week[0] = "Sunday";
    week[1] = "Monday";
    week[2]= "Tuesday";
    week[3]= "Wednesday";
    week[4] = "Thursday";
    week[5] = "Friday";
    week[6] = "Saturday";
    for (int i = 0; i < week.Length; i++)
        MessageBox.Show(week[i]);
```





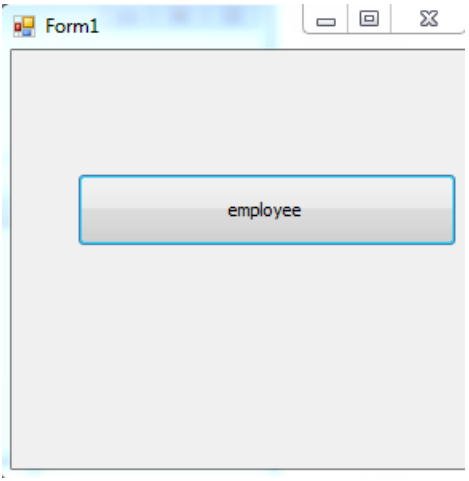
يتم عرض عناصر المصفوفة فى صندوق نصي حتى آخر عنصر بها.

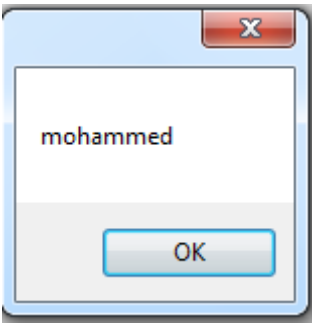
مثال٢:

```
String[] week = { "MENA", "NORHAN",
"GNA", "LOGYN", "ZYAD", "SHAHD",
"mohammed"};

        for (int i = 0; i < week.Length; i++)
            MessageBox.Show(week[i]);
```

يتم عرض عناصر المصفوفة فى صندوق نصي حتى آخر عنصر كما بالشكل.



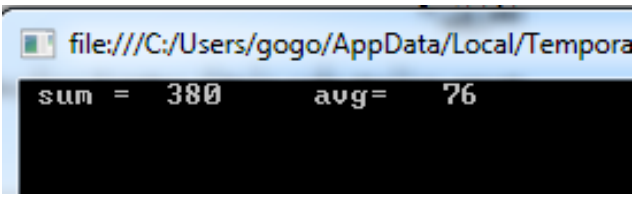


مثال٣

صمم برنامج يقوم بإيجاد مجموع ومتوسط درجات طالب في خمس مواد والدرجات كالتالي:87,67,81,90,55.

```
int[] test = new int[5] { 87, 67, 81, 90, 55 };
int s = 0;
```

```
        for (int i = 0; i < 5; i++)
        {
            s = s + test[i];
        }
        double avg = s / 5;
        Console.WriteLine(" sum =  " + s +
"\t" + "avg=   " + avg);
        Console.ReadKey();
```



طريقة إدخال و تخزين متغيرات نصية في مصفوفة و استخراجها و طباعتها على الشاشة:

مثال٤:

قم بتعريف مصفوفة نصية تحتوي خمسة عناصر ولتكن هذه العناصر أسماء لأشخاص يدخلها المستخدم عن طريق لوحة

المفاتيح، ثم قم بطباعة عناصر هذه المصفوفة على الشاشة .استعمل الحلقة التكرارية for لإدخال العناصر في المصفوفة و استخراجها، كما يمكنك استعمال أي حلقة تكرارية أخرى؟

الحل:

```
public static void Main(string[] args)
    {
        // انشاء مصفوفة تحتوي ٥ متغيرات نصية
        string[] name = new string[5];
        // بداية الحلقة التكرارية لإدخال الأسماء عبر لوحة المفاتيح
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine("enter name number {0} ", i + 1);
            // قراءة المدخلات من لوحة المفاتيح و تخزينها في المصفوفة
            name[i] = Console.ReadLine();    }
```

```
            Console.WriteLine();
            Console.WriteLine("##########   Arr
ay ###### \n");
```

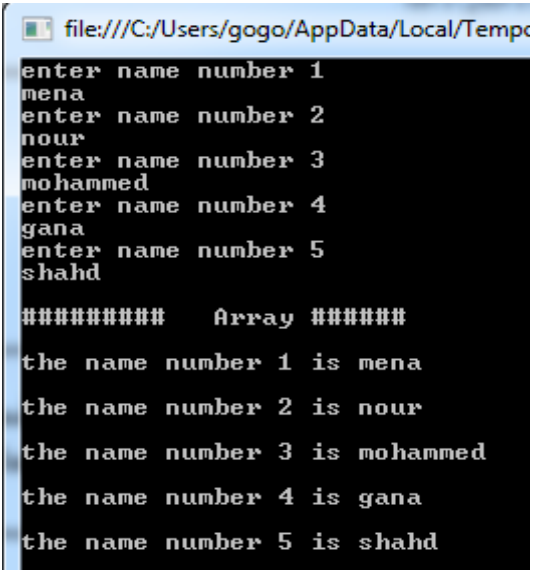
بداية الحلقة التكرارية لطباعة محتوى المصفوفة على الشاشة//

```
            for (int i = 0; i < 5; i++)
            {
                // في كل مرة تنفذ الحلقة يتم اظهار عنصر
```

واحد في الشاشة حتى يصل تكرار الحلقة الى ٥ مرات

```
                Console.WriteLine("the name
number {0} is {1}  \n", i + 1, name[i]);
            }
            Console.ReadKey();
        }
```

ملاحظة: استعملنا عبارة \n في في هذا البرنامج وهي تعني أترك سطر التالي فارغ.



تدريب

```
int[] arrMarks = new int[5];
            string[] arrNames = new string[5];
             int sum = 0;
            Console.WriteLine("Input Students
Marks");
```

```
Console.WriteLine("==========
===");
            //input loop.
           for(int i = 0; i < arrMarks.Length; i++)
           {
               Console.Write("Input student {0}
th name: ", i + 1);
                arrNames[i] =
Console.ReadLine();
               Console.Write("Input student {0}
th mark: ", i + 1);
               string tmpMark =
Console.ReadLine();
               arrMarks[i] = int.Parse(tmpMark);
                Console.WriteLine();
           }
           Console.WriteLine();
```

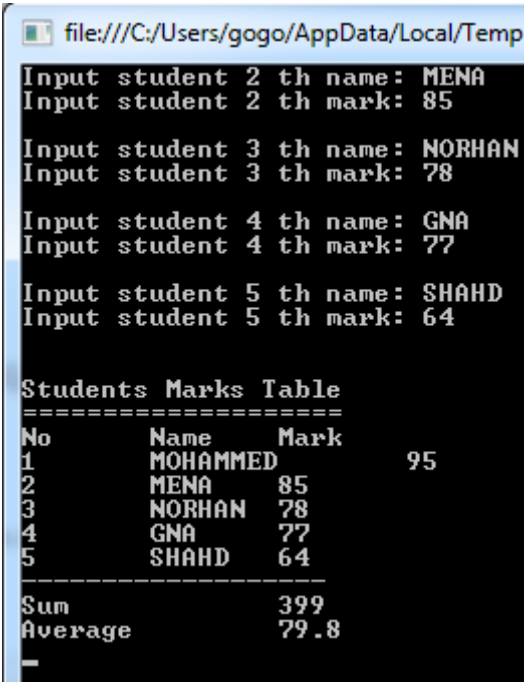
```
            Console.WriteLine("Students Marks
Table");

Console.WriteLine("==================
==");

Console.WriteLine("No\tName\tMark");

            //calculating sum and display output
loop.
            for (int i = 0; i < arrMarks.Length;
i++)
            {
                sum += arrMarks[i];
                Console.WriteLine("{0}\t{1}\t{2}", i
+ 1, arrNames[i], arrMarks[i]);
            }
```

```
            Console.WriteLine("------------------
");
            Console.WriteLine("Sum\t\t{0}",
sum);
            Console.WriteLine("Average\t\t{0}",
sum/(double)arrMarks.Length);
            Console.ReadLine ();
```



المصفوفة ذات البعدين

```
int[,] m = new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };

int [ , ] a = new int [2 , 2];
a[0,0]=1;
a[0,1]=2;
```

```
a[1,0]=3;
a[1,1]=4;

int [,]b = { {1,2}, {3,4} };
```

ArrayList : هي المصفوفات التي تحتوي على أنواع مختلفة في النوع من المتغيرات أو العناصر و نستطيع استعمالها دون تحديد حجمها أي عدد العناصر التى يتم وضعها داخل المصفوفة arraylist

طريقة كتابة و تعريف المصفوفة ArrayList:

```
ArrayList name = new ArrayList();
```

ملاحظة: لإستعمال ArrayList بالطريقة الصحيحة يجب اضافة هذا السطر في بداية البرنامج

```
using System.Collections;
ArrayList MyArry1 = new ArrayList();  // انشاء مصفوفة
MyArry1.Add("Nourhan");
MyArry1.Add(014);          // طريقة الإدخال arraylist_name.Add(string,int,….,…..)
```

```
MyArry1.Add(362.353);
foreach (object i in MyArry1)
{
        Console.WriteLine(i);          // طباعة
المتغيرات الموجودة داخل المصفوفة
}
```

فرز المصفوفة

أي ترتيبها وفق نظام معين مثلا تصاعدي أو تنازلي و هذا المجال قامت فيه دراسات واسعة في مجال علوم الحاسوب و نتجت الكثير من الخوارزميات لفرز هذه المصفوفات , أسهلها الفرز الفقاعي) .

تمرين١

صمم برنامج يستقبل من المستخدم مجموعة من الحروف ثم يطبعها له بالعكس أي إذا استقبل الحروف a,b,c,d يطبعها لك d,c,b,a، ثم يعيد ترتيبهم مرة اخري.

```
string[] name = new string[5];
```

```
        //لوحة عبر الأسماء لإدخال التكرارية الحلقة بداية
المفاتيح
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine("enter name
number {0} ", i + 1);
            // تخزينها و المفاتيح لوحة من المدخلات قراءة
المصفوفة في
            name[i] = Console.ReadLine();
        }

        string[] temp = name;

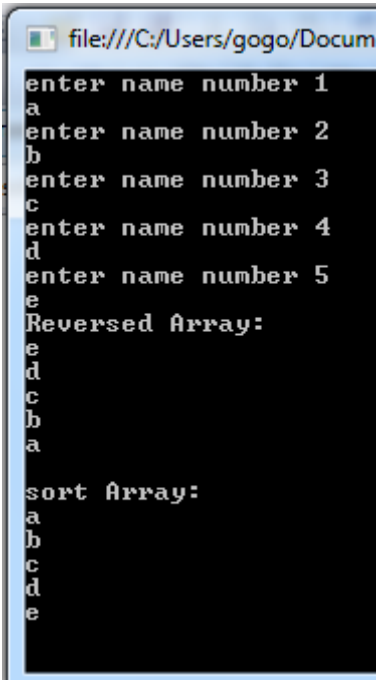

        Array.Reverse(temp);
        Console.Write("Reversed Array: \n");

        for (int i = 0; i < 5; i++)
```

```
    Console.WriteLine(temp[i]);
Console.WriteLine();

Array.Sort (temp);
Console.Write("sort Array: \n");

for (int i = 0; i < 5; i++)
    Console.WriteLine(temp[i]);
Console.WriteLine();
Console.ReadLine();
```



البحث عن مصفوفة

هناك عدة تقنيات منها البحث الخطي و البحث الثنائي وغيرها توجد وسيلة للبحث في المصفوفات تسمى البحث الثنائي و تكتب هكذا:

Array.BinarySearch (

; (القيمة ا لمراد البحث عنها ، اسم المصفوفة

البحث الخطي و الثنائي:

البحث الخطي : يقوم البحث الخطي على مقارنة الرقم المراد بحث مع كل عناصر المصفوفة حتى يجد ذلك الرقم.

قم بكتابة وسيلة تقوم بالبحث الخطي تستقبل المصفوفة و الرقم المراد البحث عنه و ترجع موقع ذلك الرقم في المصفوفة إن وجد بواسطة الفهرس و المصفوفة هي:

a={2,4,6,8,10,12,,14,16,18,22,20,26}

البحث الثنائي :

إن البحث الخطي ليس عملي في المصفوفات الضخمة حيث يستهلك وقتا كثيرا حتى يمر على جميع عناصر المصفوفة , تقوم فكرة البحث الثنائي على فكرة بسيطة تقول إذا كانت عندك مجموعة من الأرقام المرتبة تصاعديا من الأصغر إلى الأكبر , على سبيل المثال:

1, 2, 4, 6, 8, 10,11 و أنت تريد أن تبحث عن الرقم , 8 اتبع الخطوات التالية:

- قم بأخذ الرقم الذي يوجد في منتصف تلك السلسلة و هو 6

- قارن ذلك الرقم مع 8 فإذا كان أصغر منه فخذ الرقم الذي يقع في المنتصف بين الرقم الذي يلي 6 و رقم الأخير وقارنه مع 8 فإذا كان أكبر منه فاجعل الرقم الذي قبله أي 8 هو أكبر رقم وقم بأخذ الرقم الذي في المنتصف و هو في هذه الحالة 8 فكرة البحث الثنائي بسيطة جدا , حيث تقوم بحذف نصف عدد الأرقام

التي بحثت فيها , فلذا يقل مجال البحث كالتالي حيث النجمة تدل على منتصف

11	10	8	6*	4	2	1	a[(0+6)/2]=a[3]
11	10*	8					a[(4+6)/2]=a[5]
		8*					a[(4+4)/2]=a[4]

تدريب:

اكتب برنامج تستخدم البحث الثنائي حيث يستقبل مصفوفة و الرقم المراد البحث عنه مع المصفوفة التالية:

a={0,6,5,4,3,2,1,12,13,14,20}

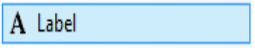
الفصل الرابع

الكائنات البرمجية للتعامل مع السلاسل النصية

الأدوات هي تلك المكونات التي تضاف إلى واجهة المستخدم ليتفاعل معها المستخدم وقت تنفيذ البرنامج وتوجد في بيئة التطوير في صندوق الأدوات على اليسار من بيئة التطوير (بشكل افتراضي وتستطيع تغيير مكانه) بسهولة جدا نقوم بإضافة المكونات إلى الفورم بواسطة السحب والإلقاء بالماوس أوالنقر نقرة واحدة علي الأداة المراد اضافتها ثم النقر نقرة واحدة علي الفورم وسيتم اضافتها.

الأدوات الموجودة في صندوق الأدوات مرتبة بحسب الفئة: فهناك الأدوات الاحتواء الشائعة الاستخدام تحت فئة Common Controls وهناك أدوات Containers وهناك أدوات الطباعة Printing. يقوم المستخدم بالتفاعل مع تلك الأدوات المختلفة للحصول على النتيجة أو الخدمه التى يريدها.

تساعد الأدوات المبرمج اثناء تصميمه برنامج بقدر كبير فتقوم باختصار الكثير من الأكواد الصعبة والطويلة ولكل اداة من الأدوات غرض معين ومهمة معينة وتقوم كل أداة بعمل مختلف عن الأداة الأخرى. والآن شرح لمجموعة من الأدوات الأكثر استعمالا مع النصوص.

- **أداة العنوان** Label 

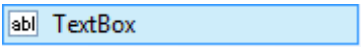
تستخدم أداة العنوان لعرض النصوص الثابتة كذلك إضافة تسميات توضيحية للعناصر الأخرى داخل النموذج، ولا يمكن للمستخدم تحرير أو تعديل البيانات الموجودة فيها إلا باستخدام أوامر البرمجة فقط.

أهم خصائص أداة العنوان Label :

Name	**هو اسم لأداة العنوان يستخدم لتغير خصائص أداة العنوان أثناء البرمجة .**
Text	**تستخدم لتغيير النص الذي تريد إظهاره في أداة العنوان. كما يمكن استخدام الكود** **Label1.Text = " اسم الطالب "**
TextAlign	**خاصية محاذاة النص داخل الأداه هل سيكون من جهة (اليمين أو اليسار أو...).**
AutoSize	**يتغير الحجم بناءا على النص طولا وقصرا وذلك بتفعيل قيمة الخاصية لـ True .**
BackColor	**تغيير لون الخلفية للأداة. كما يمكن استخدام الكود** **Label1.BackColor = Color.Blue**

Dock	وهي خاصية الإلتصاق حيث يمكن أن تجعل الأداه تملأ الجانب الأيمن من النموذج أو الأيسر أو أعلى أو أسفل النموذج أو تملأ النموذج بالكامل.
Enabled	تمكين هذه الأداة مما يجعلها تستجيب للإحداث والقيمة الافتراضية لها True.
Font	لتغير خصائص الخط على هذه الأداة من حيث اسم الخط والحجم و النمط.
ForeColor	لتغيير لون الخط(النص.)
RightToLeft	عرض الكتابة من اليمين أو اليسار.
Visible	جعل العنصر مرئي أو غير مرئي أثناء التنفيذ.
Cursor	التحكم في نوع مؤشر الفأرة عند المرور فوق الأداة المحددة.
Anchor	وهي تستخدم لجعل المسافة بين الأداة والإطار الحاوي لها ثابتة.
BorderStyle	تتحكم هذه الخاصية في شكل الحدود الخارجية للأداة.

- **أداة مربع النص TextBox**



عرض النصوص على الشاشة للسماح للمستخدم بكتابة النصوص في البرنامج وكذلك تعديلها وذلك وقت التنفيذ.

أهم خصائص أداة مربع النص TextBox

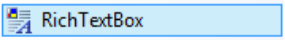
هناك العديد من الخصائص المشتركة بين اداة التسمية و مربع النص مثل

، BackColor ، AutoSize ، ... Anchor.

Borderstyle

CharacterCasing	**تتيح لك التحكم في حالة الأحرف في اللغة الانجليزية فقط هل هي حروف كبيرة Upperأم صغيرة Lower او حروف عادية Normal.**
MultiLine	**يسمح بإضافة سطور متعددة داخل مربع النص وذلك بتفعيل قيمة الخاصية لـ True .**
MaxLength	**يحدد الحد الأقصى من عدد الأحرف التي يمكن للمستخدم إدخالة داخل المربع النصي والقيمة صفر أنه لا يوجد حد اقصى.**

ScrollBars	وهي الخاصية التي تحدد هل ستظهر أشرطة تمرير أم لا حيث: None: تعني لن يكون هناك ظهور لأشرطة تمرير. Vertical: تعني أنه سيكون هناك أشرطة تمرير عمودية. Horizontal: تعني أنه سيكون هناك أشرطة تمرير أفقية. Both: تعني أنه سيكون هناك أشرطة التمرير الأفقية والرأسية معاً.

- أداة مربع النص 

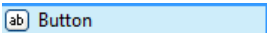
RichTextBox

متشابهة إلى أداة مربع النص ولكنها توفر بعض الميزات المتقدمة أكثر من أداة مربع النص العادية TextBox .

يتيح تنسيق النص إلى إضافة لون خاص لكل كلمه أو أي نص نريده، و كذلك التحكم في حجم بعض النصوص و أنواع الخطوط.

تدريب

استخدم أداة مربع النص RichTextBox للبحث عن الكلمه المراد تغيير لون الخط بها.

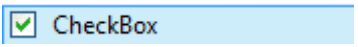
- **زر الأوامر Button** 

هى أداة تقوم بعمل محدد عندما يقوم المستخدم بالنقر عليها ولا تخلو نافذة من النوافذ إلا و بها أزرار الأوامر.

أهم خصائص أداة زر الأوامر Button

BackgroundImage	**إظهار صورة في خلفية الزر.**
Image	**تستخدم هذه الخاصية لتحديد الصورة التي ستكون في المقدمة للزر.**
Text	**النص الذي تريد إظهاره.**
BackColor	**يمكن تغيير لون الخلفية لأداة زر الأمر إثناء التصميم و التنفيذ.**

- **مربع الاختيار CheckBox**

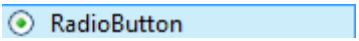


تتيح مربعات الاختيار للمستخدم تحديد خيار واحد أو أكثر.

أهم خصائص أداة الاختيارCheckBox

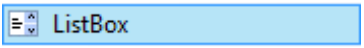
Checked	**تستخدم لمعرفة ما إذا كان المستخدم قد تم اختيار المربع فى حالة القيمة True والقيمة False تعني انه ترك المربع فارغ.**
CheckState	**تستخدم لمعرفة القيمة الحالية لمربع الاختيار CheckBox ولها ثلاث قيم Checked قد تم الاختيار، Unchecked لم يتم الاختيار، Indeterminate غير محدد.**
Appearance	**خاصية التحكم في شكل مربع الاختيار CheckBoxوهناك شكلان Normal وهو المظهر الافتراضي لمربع الاختيار، Button ويأخذ شكل زر.**
CheckAlign	**محاذاة او تنسيق النص بالنسبة لمربع الاختيار CheckBox.**

- **زر الاختيارRadioButton**



تستخدم لإعطاء فرصة للمستخدم لاختيار واحد من عدة اختيارات، ولها نفس خصائص أداة الاختيارCheckBox .

- **أداة مربع السرد ListBox**



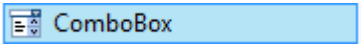
تستخدم هذه الأدوات لعرض مجموعة من العناصرمن String نوع نص
على شكل قائمة. إذا كان عدد العناصر أكبر من حجم القائمة ، يقوم البرنامج بإظهار شريط تمرير رأسي يمكن للمستخدم أن يستعرض محتويات القائمة.

أهم خصائص أداة مربع السرد ListBox

Stored	**تستخدم لتحديد هل سيتم ترتيب العناصر أبجدياً أم لا .**
SelectionMode	**تستخدم لتحديد ما إذا كان المستخدم يستطيع اختيار عنصر أو أكثر من عناصر القائمة والعكس.**

Item	تستخدم هذه الخاصية لوضع العناصر داخل القائمة بحيث يتم كتابة كل عنصر في سطر منفرد ثم نقوم بالنقر على زر الإدخال.
MultiColumn	تستخدم هذه الخاصية لتحديد ما إذا كان سيتم عرض أكثر من عمود في نفس الوقت أم سيتم عرض عمود واحد فقط.
ScroleAlwaysVsible	تستخدم هذه الخاصية لتحديد هل سيتم ظهور شريط الإنزلاق بشكل دائم بغض النظر عن عدد العناصر داخل القائمة أو لا.

- **أداة القائمة المنسدلةComboBox**



تجمع أداة القائمة المنسدلة بين إمكانيات أداة النص وأداة القائمة، حيث يمكن للمستخدم أن يكتب أو يعرض بيانات (عناصر) على شكل قائمة ، وممكن أن تكون مرتبة ، ثم يختار

منها عنصر ما فمثلاً إذا أردنا عرض المدن للمستخدم ليختار المدينة التي ينتمي إليها مع اتاحة الفرصة له لكتابة اسم مدينته ان لم تكن في القائمة ويمكن ذلك باستخدام أداة القائمة المنسدلة .

أهم خصائص أداة القائمة المنسدلة ComboBox

Stored	**تستخدم لتحديد هل سيتم ترتيب العناصر أبجدياً أم لا .**
DropDownStyle	**تستخدم لتحديد أداة القائمة المنسدلة وتحديد امكانيةكتابة عنصر غير موجود بالقائمة المنسدلة.**
Item	**تستخدم هذه الخاصية لوضع العناصر داخل القائمة.**
MaxDropItem	**تستخدم هذه الخاصية لتحديد أقصى عدد من العناصر يمكن عرضه عند انسدال القائمة وتستخدم هذه الخاصية فقط في حالة قيمة DropDownStyle الخاصية لا تساوى القيمة Simple**

- **العبارة الشرطية If-Else**

وهي بنية مألوفة في معظم لغات البرمجة، تشبه هذه البنية في تشكيلها تلك الموجودة في لغات أخرى مثل C++ و Java. تمتلك هذه البنية ثلاثة أشكال سنتحدّث عنها تباعًا.

- **الشكل الأبسط لبنية IF هي:**

```
if ([condition])
{
   statement1;
   statement2;
   ...
}
```

إذا كان تقييم evaluate الشرط [condition] يعطينا true (أي تحقّق الشرط) عندها ستُنفّذ العبارات البرمجيّة الموجودة ضمن البلوك {}، وإلّا (أي لم يتحقّق الشرط) فلن يُنفّذ أيّ منها.

```
class Program{
static void Main(string[] args){
double x;
string str_x;
Console.Write("Input a number: ");
str_x = Console.ReadLine();
x = double.Parse(str_x);
if(x > 5)
{
Console.WriteLine("The value {0} is greater than 5", x);
}
Console.WriteLine("Goodbye!");
} } }
```

جرّب تنفيذ هذا البرنامج وادخل ٦، سيعرض البرنامج الخرج التالي :

```
The value 6 is greater than 5
```

أعد تنفيذ البرنامج وأدخل هذه المرّة القيمة ٣ لتحصل على الخرج التالي:

Goodbye!

لاحظ بأنّ خرج البرنامج قد اختلف باختلاف القيم المدخلة، أي أنّ هناك اختلاف في العبارات البرمجيّة التي تمّ تنفيذها في كلّ مرّة. يعود سبب ذلك إلى البنية if يختبر الشرط الموجود بعد كلمة if فيما إذا كانت قيمة المتغيّر x أكبر تمامًا من ٥. فإذا كانت نتيجة تقييم التعبير x > ٥ تساوي true فهذا يعني أنّ الشرط قد تحقّق وبالتالي تنفّذ جميع العبارات البرمجيّة الموجودة في البلوك. أمّا إذا كانت نتيجة تقييم التعبير x > ٥ تساوي false فعندها سيتجاوز تنفيذ البرنامج البنية if إلى العبارات التي تأتي بعدها.

- **الشكل الثاني لبنية IF**

هذا الشكل للعبارة الشرطية if مفيد أيضًا، ويُستخدم عندما نريد الاختيار بين مجموعتين من العبارات البرمجيّة، والشكل العام له:

```
if ([condition])
{
    statement1;
    statement2;
    ...
}
else
{
    Statement3;
    Statement4;
    ...
}
```

"إذا تحقق الشرط [condition] عندها تنفّذ البلوك الموجود بعد if مباشرةً، وإلّا يتم تنفيذ البلوك الموجود بعد else مباشرةً"

```
class Program  {
static void Main(string[] args)   {
double x;
```

```
string str_x;
Console.Write("Input a number: ");
str_x = Console.ReadLine();
x = double.Parse(str_x);
if (x > 5){
Console.WriteLine("The value {0} is greater than 5", x);
}
else
{
Console.WriteLine("The value {0} is smaller than or equals 5", x);
}
Console.WriteLine("Goodbye!");
}}}
```

يسلك هذا البرنامج نفس السلوك الذي يسلكه البرنامج السابق باستثناء أنّه لو أدخل المستخدم قيمة مثل ٣ سيعمل البرنامج على طباعة الخرج التالي:

```
The value 3 is smaller than or equals 5
Goodbye!
```

السبب في ظهور الخرج الجديد هو وجود القسم else في بنية if السابقة، فعندما يُقيّم الشرط x > ٥ في السطر وتكون نتيجة تقييمه false سينتقل البرنامج فورًا إلى تنفيذ العبارات البرمجيّة الموجودة ضمن البلوك else وهذا هو سبب ظهور هذا الخرج.

- **الشكل الثالث لبنية IF**

وهو الشكل الأكثر شمولًا وفيه نستخدم القسم else if على الصورة التالية:

```
if ([condition])
{
   statement1;
```

```
    statement2;
    ...
}
else if ([condition1])
{
    Statement3;
    Statement4;
    ...
}
else if ([condition2])
{
    Statement3;
    Statement4;
    ...
}
...
else
{
```

```
    Statement3;
    Statement4;
    ...
}
```

"إذا تحقّق الشرط [condition] عندها تنفّذ البلوك الموجودة بعد if مباشرةً، وإلّا إذا (else if) تحقّق الشرط [condition1] يتم تنفيذ البلوك الموجودة بعد else if الأولى مباشرةً، وإلّا إذا تحقّق الشرط [condition2] يتم تنفيذ البلوك الموجودة بعد else if الثانية مباشرةً، وإلّا (else) يتم تنفيذ البلوك الموجودة بعد else مباشرةً"

```
class Program{
static void Main(string[] args){
double x;
string str_x;
Console.Write("Input a number: ");
str_x = Console.ReadLine();
x = double.Parse(str_x);
```

```
if (x > 5)
{
Console.WriteLine("The value {0} is greater than 5", x);
}
else if (x == 5)
{
Console.WriteLine("The value {0} is equals 5", x);
}
else
{
Console.WriteLine("The value {0} is smaller than 5", x);
}
Console.WriteLine("Goodbye!");
 }}}
```

يشبه هذا البرنامج سابقيه إلى حدٍّ بعيد، فهو يقارن القيمة المدخلة مع العدد ٥ ويعرض رسالة مناسبة نتيجة عملية المقارنة. الشيء الجديد هنا هو التمييز بين الحالة التي تكون فيها القيمة المدخلة تساوي العدد ٥ والحالة التي تكون فيها أصغر من العدد ٥. قمنا بذلك من خلال إضافة القسم else if جديد يختبر حالة المساواة مع العدد ٥.

- **العبارة الشرطية switch-case**

تفيد هذه البنية في الاختيار من بين عدّة حالات منفصلة. لهذه البنية الشكل العام التالي:

```
switch(expression)
{
   case [A]:
      [statements]
      break;

   case [B]:
```

```
        [statements]
        break;

    ...

    [default:]
        [statements]
        break;
}
```

القسم الأخير default هو قسم اختياري، كما يجب أن يكون هناك قسم case واحد على الأقل.

```
class Program{
static void Main(string[] args){
double x, y;
string str_x, str_y, operation;
```

```
Console.Write("Input first number: ");
str_x = Console.ReadLine();
Console.Write("Input second number: ");
str_y = Console.ReadLine();
Console.Write("Choose operation (+, -, *, /): ");
operation = Console.ReadLine();
x = double.Parse(str_x);
y = double.Parse(str_y);
switch (operation){
case "+":
Console.WriteLine("{0} + {1} = {2}", x, y, x + y);
break;
case "-":
Console.WriteLine("{0} - {1} = {2}", x, y, x - y);
break;
```

```
case "*":
Console.WriteLine("{0} * {1} = {2}", x, y, x *
y);
break;
case "/":
Console.WriteLine("{0} / {1} = {2}", x, y, x /
y);
break;
default:
Console.WriteLine("Unsupported
operation.");
break;
}}}}
```

البرنامج السابق عبارة عن برنامج آلة حاسبة بسيطة تدعم العمليات الحسابية الأربع: الجمع والطرح والضرب والقسمة. يطلب البرنامج من المستخدم إدخال قيمتين عدديّتين، بعد ذلك يطلب اختيار العمليّة الحسابيّة المراد إجراؤها على هاتين

القيمتين (+ ، - ، * ، /) وتخزين العمليّة المختارة ضمن المتغيّر النصّي operation. تعمل البنية switch على مقارنة قيمة المتغيّر النصيّ operation مع القيم الموجودة في أقسام case فإذا طابقت القيمة الموجودة في operation إحدى تلك القيم، فإنّ العبارات البرمجيّة الموجودة ضمن هذا القسم سيتمّ تنفيذها. أمّا إذا لم يحدث مثل هذا التطابق، فستنفّذ العبارات البرمجيّة الموجودة في القسم الاختياري default والتي ستخبر المستخدم (في هذا المثال) بأنّ العمليّة الحسابيّة التي يرغبها لا يدعمها البرنامج.

نستفيد من القسم default في تنفيذ عبارات برمجيّة في حال لم يحدث التطابق مع أيّ قسم case سابق. كما نلاحظ أنّ العبارة break الموجودة في كلّ قسم من أقسام case بالإضافة إلى قسم default هي عبارة ضرورية وتؤدّي إلى انتقال تنفيذ البرنامج إلى خارج بنية switch. جرّب تنفيذ البرنامج وإدخال قيم متنوّعة بالإضافة إلى تجريب العمليات الحسابيّة الأربع. جرّب إدخال عامل باقي القسمة مثلًا (%) وانظر كيف سيجيب البرنامج بالرسالة Unsupported operation.

تمرين ١

في البرنامج السابق إذا أدخل المستخدم القيمة ٠ للعدد الثاني، ثم اختار عمليّة القسمة (/) سيؤدّي ذلك إلى القسمة على صفر، وهذا يسبّب خطأً أثناء التنفيذ runtime error يؤدّي إلى رمي استثناء وتوقّف البرنامج عن العمل. أجرِ تعديلًا على البرنامج ليأخذ هذا الأمر بالحسبان.

(تلميح: أضف شرط if ضمن قسم case الموافق للعمليّة (/) لاختبار قيمة المتغيّر y فيما إذا كانت تساوي الصفر أم لا).

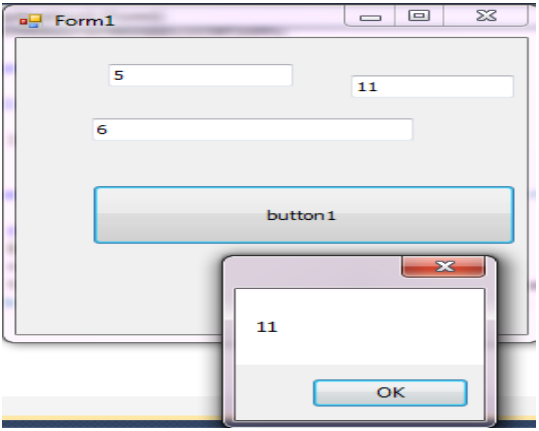
تمرين ٢

اكتب برنامجًا يطلب من المستخدم إدخال درجة الحرارة الحاليّة. فإذا كانت درجة الحرارة أقل من ٤ مئوية يعرض البرنامج الرسالة "Very Cold". أمّا إذا كانت درجة الحرارة بين ٤ وأقل من ١٠ مئويّة يعرض الرسالة "Cold". وفي حال كانت درجة الحرارة بين ١٠ وأقل من ٣٠ مئويّة يعرض الرسالة "Normal". أمّا إذا كانت درجة الحرارة ٣٠ فما فوق فيعرض البرنامج الرسالة "Hot".

- **أمثلة متنوعة على استخدام الكائنات البرمجية المختلفة**

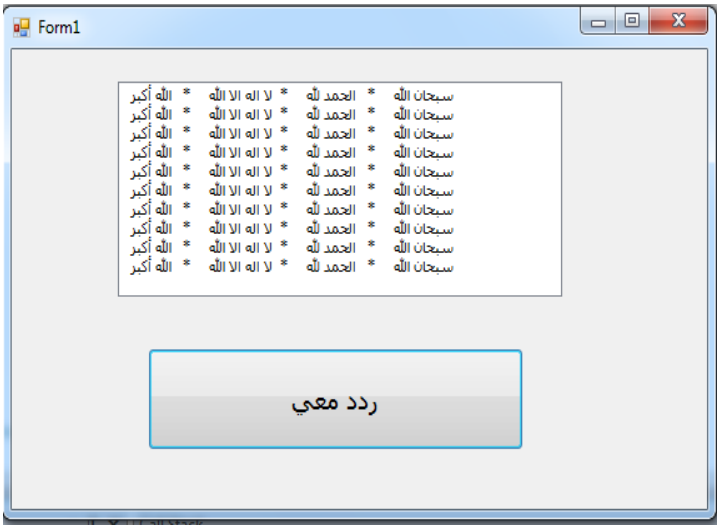
مثال جمع رقمين

```
double num1, num2;
num1 = double.Parse(textBox1.Text);
num2 = double.Parse(textBox2.Text);
textBox3.Text = Convert.ToString(num1 + num2);
MessageBox.Show((num1 + num2).ToString());
```



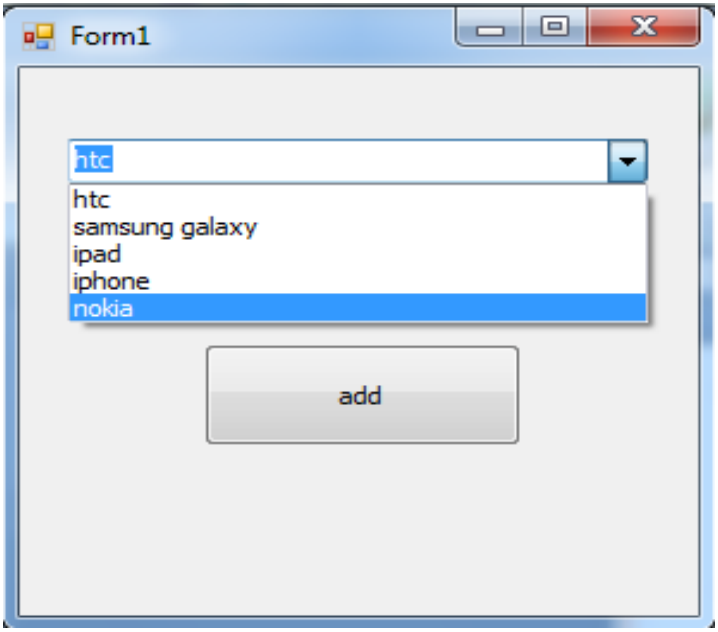
مثال ١:

```
for (int i = 0; i < 10; i++)
{
   listBox1.Items.Add(" سبحان الله  *  الحمد لله  * لا اله الا الله  *  الله أكبر ");
}
```

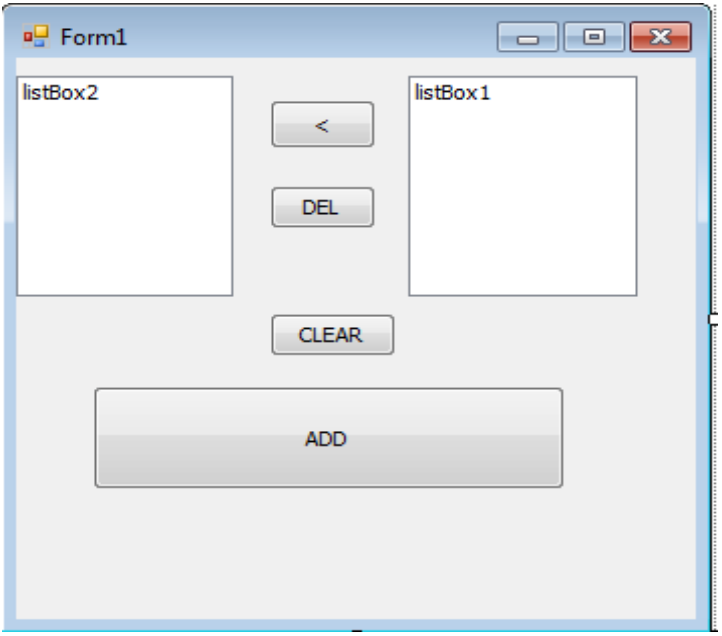


مثال٢:

```
private void button1_Click(object sender, EventArgs e)
{
    comboBox1.Items.Add("nokia");
}
```



مثال ٣:



```
private void button1_Click(object sender, EventArgs e)
{
    String[] EMPLOYE = { "MENA", "NORHAN", "GNA", "LOGYN", "ZYAD", "SHAHD", "mohammed" };
    listBox1.Items.AddRange(EMPLOYE);
}

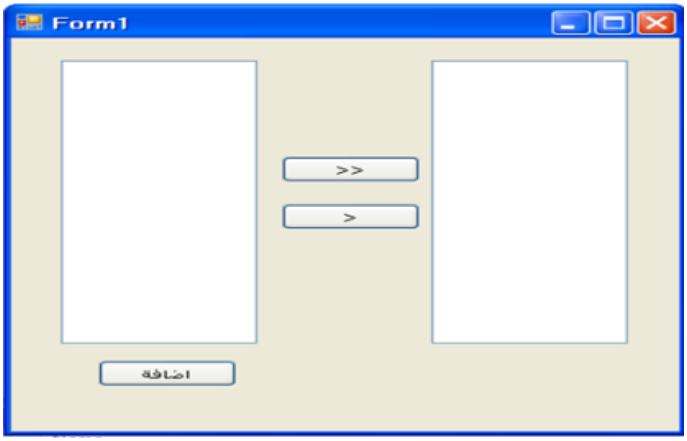
private void button2_Click(object sender, EventArgs e)
{
    listBox2.Items.Add(listBox1.SelectedItem);
}

private void button3_Click(object sender, EventArgs e)
{
    listBox2.Items.Remove(listBox2.SelectedItem);
}

private void button4_Click(object sender, EventArgs e)
{
    listBox2.Items.Clear();
    if (listBox2.Items.Count == 0)
        MessageBox.Show("listBox2  is empty");
}
```

مثال ٤:

انشئ نموذج وأضف إليه الأدوات كما توضح الصورة .

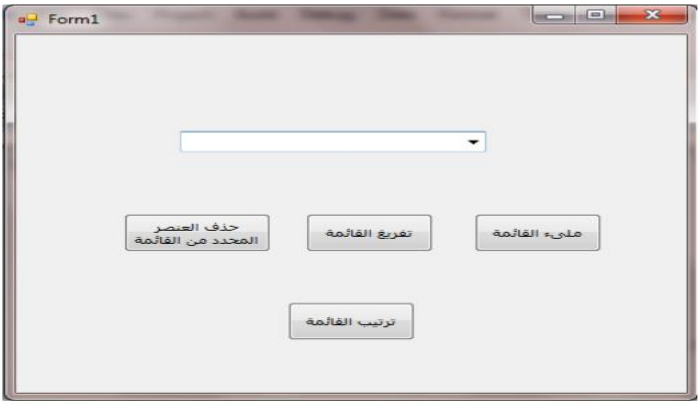



```
// >>شفرة الزر
private void button2_Click(object sender, EventArgs e)
{
listBox2.Items.Clear();
if (listBox1.Items.Count == 0)
MessageBox.Show("listbox1 is empty");

for (int i = 0; i < listBox1.Items.Count; i++)
{
listBox2.Items.Add(listBox1.Items[i]);
}
}
```

```
//شفرة الزر إضافة
private void button1_Click(object sender, EventArgs e)
{
listBox1.Items.Clear();
string[] Array = new string[12];
for (int i = 0; i < 12; i++)
Array[i] = ("number "+i.ToString());
listBox1.Items.AddRange(Array);
}
// >شفرة الزر
private void button3_Click(object sender, EventArgs e)
{
if (listBox1.Items.Count == 0)
MessageBox.Show("listbox1 is empty");
else if (listBox1.SelectedIndex != -1)
{
listBox2.Items.Add(listBox1.SelectedItem);
listBox1.Items.Remove(listBox1.SelectedItem);
}
else
{
MessageBox.Show("القائمة من عنصر إختار");
}
}
```

مثال٥:



```
private void button1_Click(object sender, EventArgs e)
{
comboBox1.Items.Clear();
primary(15);
}


private void primary(int x)
{
for (int j = 1; j <= x; j++)
comboBox1.Items.Add(j);
}

private void button2_Click(object sender, EventArgs e)
{
comboBox1.Items.Clear();
primary(15);
}

private void button3_Click(object sender, EventArgs e)
{
comboBox1.Items.Remove(comboBox1.SelectedItem);
}

private void button4_Click(object sender, EventArgs e)
{
comboBox1.Sorted= true;
}
```

- **التعامل مع الأدوات (CheckBox, RadioButton ,GroupBox)**

تمكنك الأداه GroupBox من انشاء مجموعات ضمن النموذج.

الأداة CheckBox والأداة RadioButton متشابهة من حيث إمكانية تفعيلها وعدم تفعيلها باستخدام الخاصية Checked وتحمل هذه الخاصية القيمة true or false والفرق بينهما أن الأداة CheckBox صممت لتعمل ضمن مجموعات بحيث يمكنك تحديد اكثر من زر اختيار معاً حتي ولو كانت ضمن مجموعة بينما الأداة RadioButton لايمكنك تحديد اكثر من زر اختيار واحد إذا كانت ضمن مجموعة.

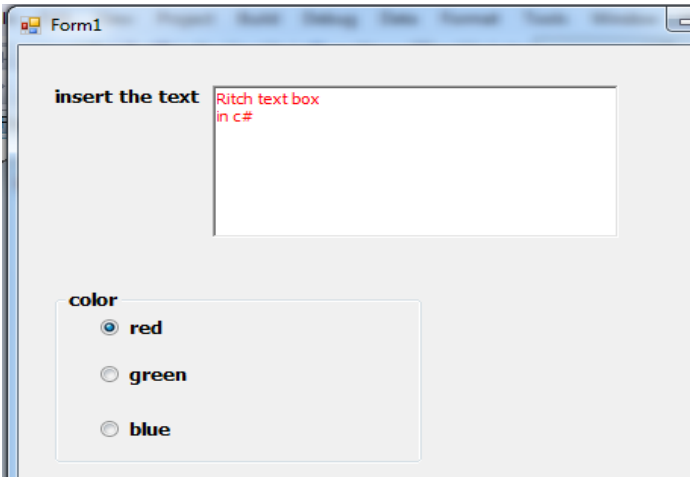
مثال ١ :

```
private void radioButton1_CheckedChanged(object sender, EventArgs e)
{
    richTextBox1.ForeColor  = Color.Red;
}

private void radioButton2_CheckedChanged(object sender, EventArgs e)
{

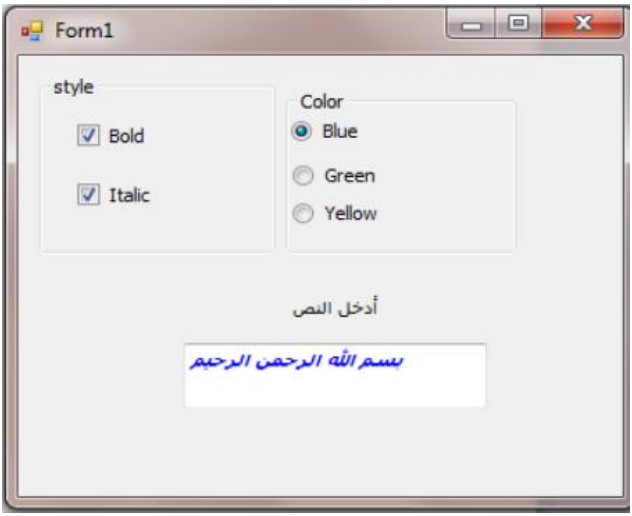
    richTextBox1.ForeColor  = Color.Green;
}

        private void radioButton3_CheckedChanged(object sender, EventArgs e)
{
    richTextBox1.ForeColor  = Color.Blue;
}
```



مثال٢:

انشئ نموذج وأضف الأدوات كما توضح الصورة التالية.



```
// Bold شفرة الأداة
private void CheckBold_CheckedChanged(object sender,
EventArgs e)
        {
  if (CheckBold.Checked == true & CheckItalic.Checked ==
false )
        textBox1.Font = new Font(textBox1.Font,
FontStyle.Bold);
  else if (CheckBold.Checked == false & CheckItalic.Checked
== true )
        textBox1.Font = new Font(textBox1.Font,
FontStyle.Italic);
  else if (CheckBold.Checked == true & CheckItalic.Checked
== true )
        textBox1.Font = new Font(textBox1.Font,
FontStyle.Italic | FontStyle.Bold);

  else
        textBox1.Font = new Font(textBox1.Font,
FontStyle.Regular);
        }
```

```
// Italic شفرة الأداة
 private void CheckItalic_CheckedChanged(object sender,
EventArgs e)
        {
   if (CheckBold.Checked == true & CheckItalic.Checked ==
false )
          textBox1.Font = new Font(textBox1.Font,
FontStyle.Bold);
   else if (CheckBold.Checked == false & CheckItalic.Checked
== true)
          textBox1.Font = new Font(textBox1.Font,
FontStyle.Italic);
   else if (CheckBold.Checked == true & CheckItalic.Checked
== true )
          textBox1.Font = new Font(textBox1.Font,
FontStyle.Italic | FontStyle.Bold);

   else
          textBox1.Font = new Font(textBox1.Font,
FontStyle.Regular);
        }
// Blue شفرة الزر
private void radioButton1_CheckedChanged(object sender,
EventArgs e)
        {
            textBox1.ForeColor = Color.Blue;
        }
```

```
// Green شفرة الزر
private void radioButton2_CheckedChanged(object sender,
EventArgs e)
        {
            textBox1.ForeColor = Color.Green;
        }
// Yellow شفرة الزر
 private void radioButton3_CheckedChanged(object sender,
EventArgs e)
        {
            textBox1.ForeColor = Color.Yellow;
        }
```

- **عرض مربعات الرسائل باستخدام الطريقة (MessageBox.Show)**

مثال ١:

قم بإنشاء نموذج يظهر للمستخدم رسالة " هل تريد إغلاق البرنامج " اذا كانت الإجابة موافق يغلق النموذج وإذا إلغاء الأمر يغلق الرسالة.

١. أضف زر (Button) من قائمة الأدوات واضبط الخاصية (Text=خروج) والخاصية (name=Exit).

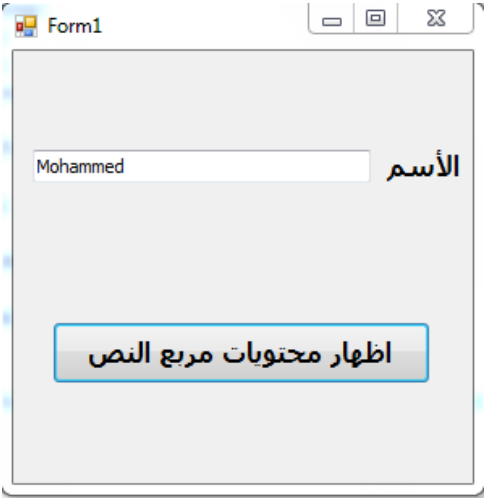
٢. ثم انقر نقره مزدوجة على الزر وأكتب الكود كما يلي:

```
private void Exite_Click(object sender, EventArgs e)
{
DialogResult re;
re = MessageBox.Show("هل تريد الخروج من البرنامج", "رسالة",
MessageBoxButtons.OKCancel, MessageBoxIcon.Information,
MessageBoxDefaultButton.Button2);
if (re == DialogResult.OK)
System.Environment.Exit(0);}
```



مثال٢:

```
/*
This is a Comment
*/
//This is a String Example
string firstName;
firstName = textBox1.Text;
MessageBox.Show(" محتوى مربع النص هو: " + firstName,"رسالة");
```


Form1
Mohammed
الأسم
اظهار محتويات مربع النص


رسالة
Mohammed محتوى مربع النص هو:
OK

مثال٣ :

```
public partial class Form1 : Form
{
string x = "ahmed", y = "1234";
int i=1;
Form2 OB = new Form2();

private void button1_Click(object sender, EventArgs e)
{
if (i <= 3)
if (textBox1.Text == x && textBox2.Text == y)
OB.Show();
else
{
i++;
MessageBox.Show("أخرى مرة حاول","رسالة");
}
else
Close();

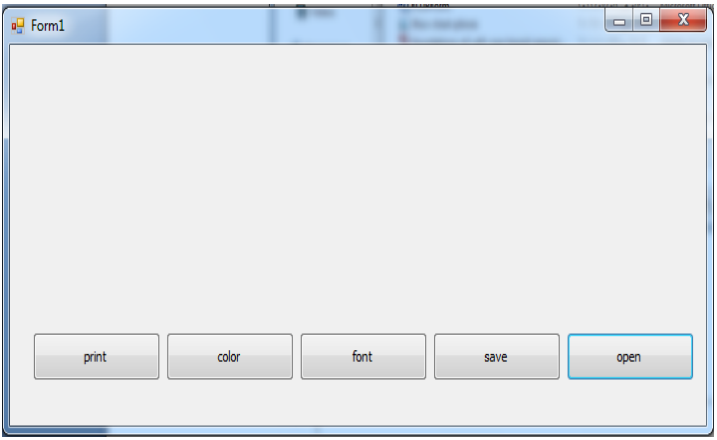
}
```



صناديق الحوار (Dialog Boxes)

- **OpenfileDialog**
- **SavefileDialog**
- **Font Dialog**
- **ColorDialog**

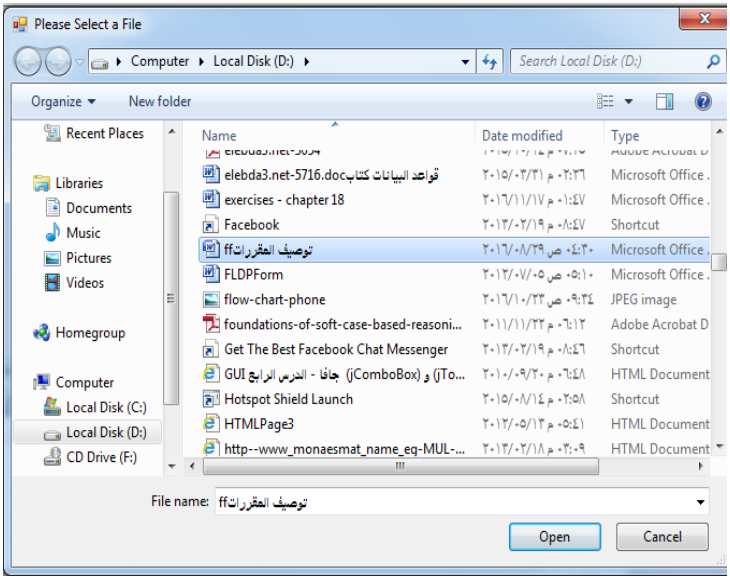
١. اضف الأداة RichtextBox من صندوق الأدوات.
٢. أضف صناديق الحوار الأربعة .
٣. أضف ٤ أزرار تحكم (Button) وغير الخاصية text لها كما توضح الصورة، واجعل الخاصية name لها كما يظهر في كود الحدث Click .

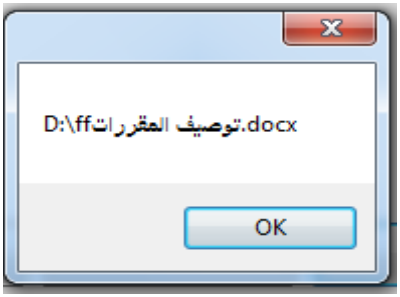


- صندوق حوار فتح الملفات

OpenFileDialogs

```
string  strFileName ;
openFileDialog1.Title = "Please Select a File";
openFileDialog1.InitialDirectory = "d:";
openFileDialog1.ShowDialog();
openFileDialog1.Filter = "textfiles|*.docx,*.txt";
strFileName = openFileDialog1.FileName;
MessageBox.Show (strFileName);
```

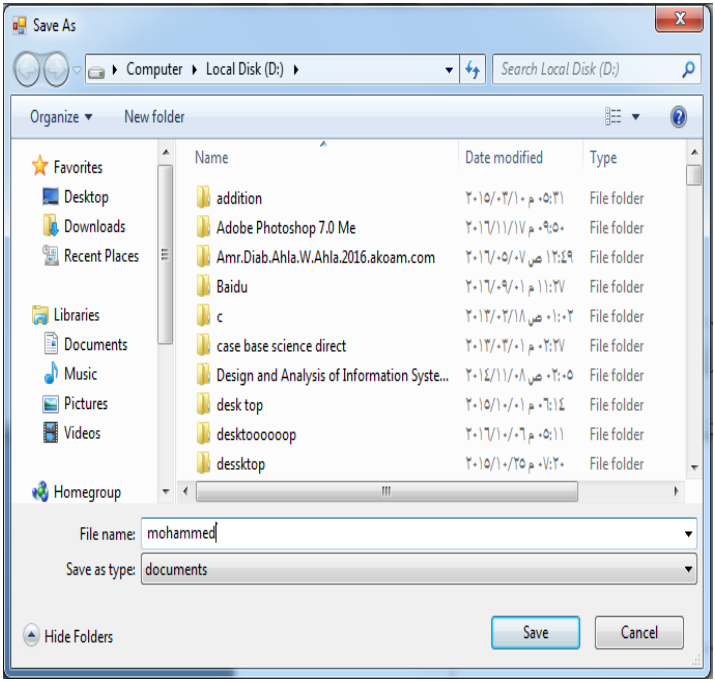

Please Select a File
Computer ▸ Local Disk (D:) ▸
Search Local Disk (D:)
Organize
New folder
Recent Places
Libraries
Documents
Music
Pictures
Videos
Homegroup
Computer
Local Disk (C:)
Local Disk (D:)
CD Drive (F:)
Name
Date modified
Type
elebda3.net-5716.doc قواعد البيانات كتاب
exercises - chapter 18
Facebook
توصيف المقررات ff
FLDPForm
flow-chart-phone
foundations-of-soft-case-based-reasoni...
Get The Best Facebook Chat Messenger
GUI جافا - الدرس الرابع (jComboBox) و (jTo...
Hotspot Shield Launch
HTMLPage3
http--www_monaesmat_name_eq-MUL-...
Microsoft Office .
Shortcut
JPEG image
Adobe Acrobat D
HTML Document
File name: توصيف المقررات ff
Open
Cancel

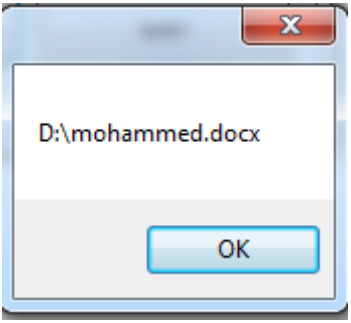

D:\ffتوصيف المقررات.docx
OK

- **SaveFileDialogs**

تستخدم لحفظ ملف نصي.

```
string  strFileName ;
saveFileDialog1.Filter = "documents|*.docx";
saveFileDialog1.ShowDialog();
saveFileDialog1.InitialDirectory = "d:";
strFileName = saveFileDialog1.FileName;
MessageBox.Show(strFileName);
```



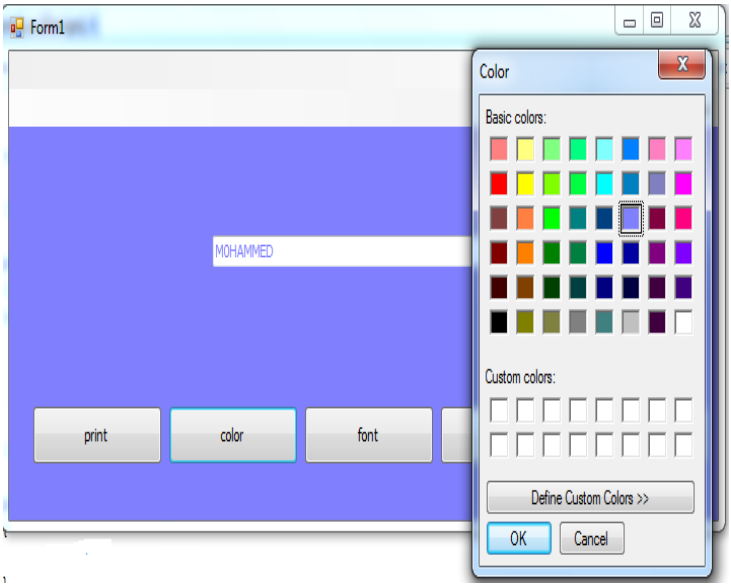


- **Color Dialog**

استخدم الأداة لاختيار لون الخط وخلفية النموذج Form

```
private void button4_Click(object sender, EventArgs e)
{
if (colorDialog1.ShowDialog() == DialogResult.OK ){
    textBox1.ForeColor = colorDialog1.Color;
    this.BackColor = colorDialog1.Color;
          }

}
```

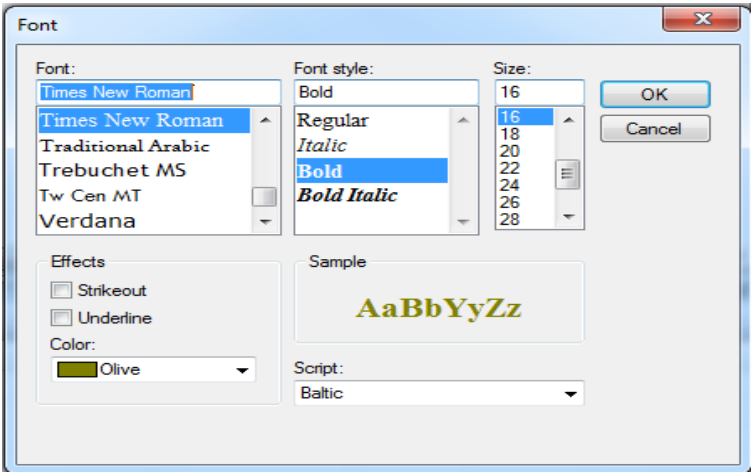


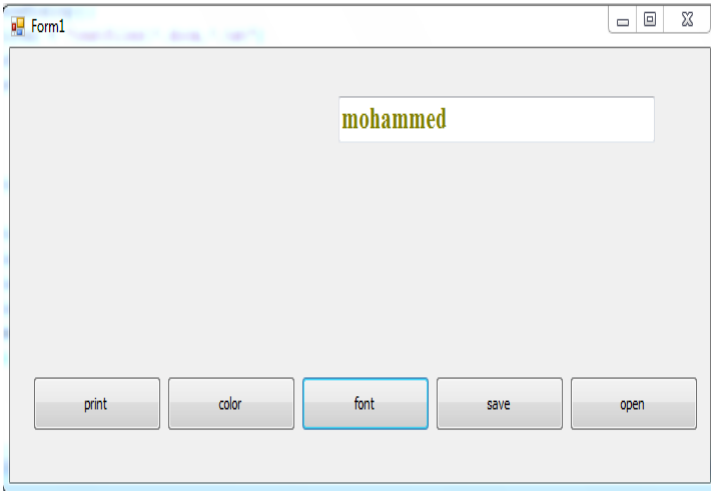
- **صندوق حوار الخط Font Dialog**

تسمح للمستخدم باختيار نوع الخط وطريقة عرضه.

```
    fontDialog1.ShowColor=true ;
if (fontDialog1.ShowDialog () == DialogResult.OK )
    {
    textBox1.Font = fontDialog1.Font;
textBox1.ForeColor = fontDialog1.Color;}

}
```

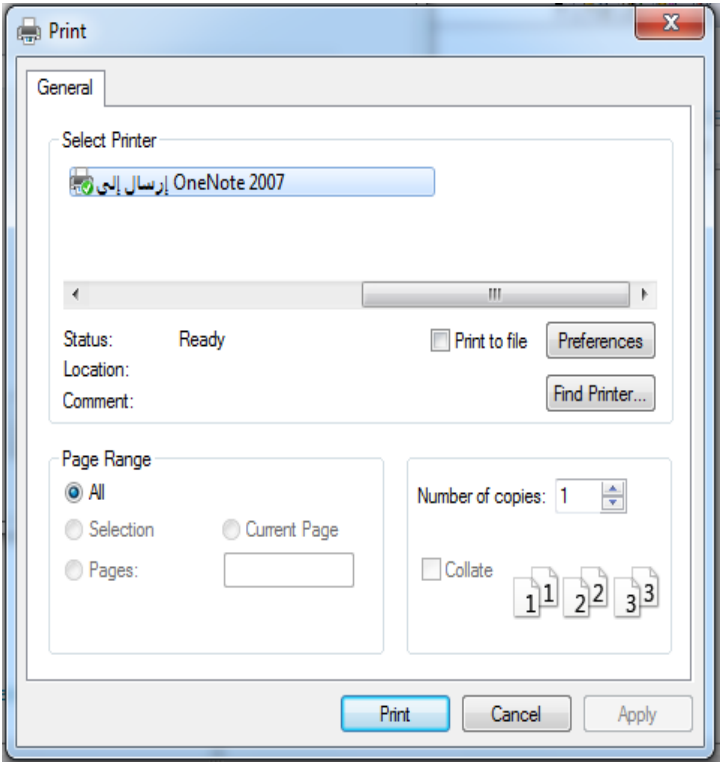




- صندوق **PrintDialog**

تسمح للمستخدم بتغيير خيارات الطباعة

```
private void button5_Click(object sender, EventArgs e)
{
    printDialog1.ShowDialog();
}
```



عند وضع الأداه PrintDialog إلى النموذج يجب اسناد الأداه PrintDocument أيضاً واسنادة إلى الخاصية Document من الأداة PrintDialog وكذلك اسناد الأداة PrinterSettings (وهي تحوي إعدادات الطباعة) للأداة PrintDocument إلى الخاصية PrinterSettings من

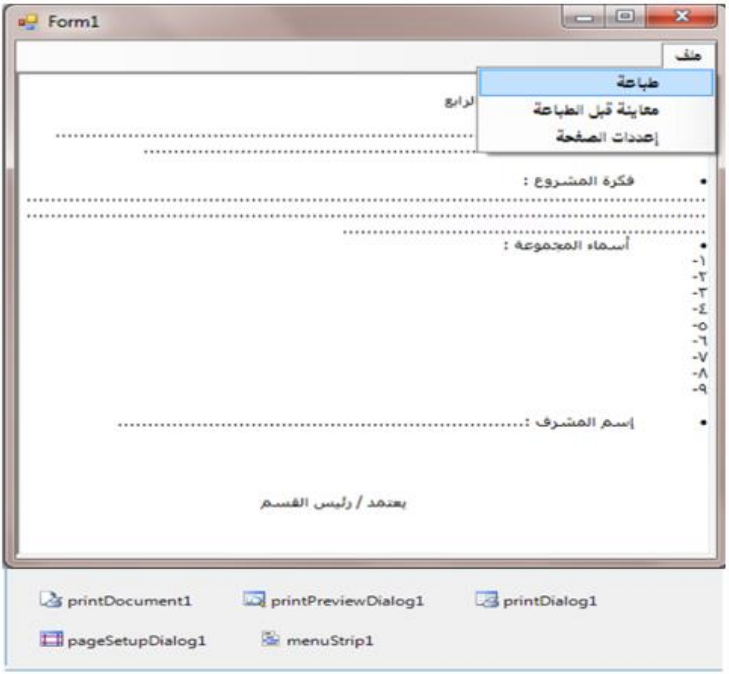
ألأداة PrintDialog ثم نستخدم الطريقة ()print من الأداة PrintDocument لطباعة الملف.

وسيتم توضيح ذلك من الكود التالي:

- افتح مشروع جديد وأضف نموذج إليه.
- أضف الأدوات التالية:

PrintDialog, PrintDocument, PrintPrivewDialog

- غير الخصائص للأداة PrintDialog وأجعلها كلها تأخذ القيمة true.
- قم بإضافة الأداة MenuStrip والأداة richTextBox.
- اجعل تصميم النموذج يظهر بالشكل التالي بعد اضافة النص للأداة richTextBox.



بعد تصميم النموذج غير الخاصية name لكل عنصر من عناصر الـ MenuStrip كما ستظهر في الكود التالي.

```
//شفرة العنصر طباعة
private void Print_Click(object sender, EventArgs e)
{
printDialog1.Document = printDocument1;
printDialog1.PrinterSettings= printDocument1.PrinterSettig ;
if (printDialog1.ShowDialog() == DialogResult.OK)
{
printDocument1.PrinterSettings=
printDialog1.PrinterSettings;
printDocument1.Print();
}

//شفرة العنصر معاينة قبل الطباعة
private void PeforPrint_Click(object sender, EventArgs e)
{
printPreviewDialog1.Document = printDocument1;
printDocument1.PrintPage +=
new System.Drawing.Printing.PrintPageEventHandler(
printDocument1_PrintPage);
if (printPreviewDialog1.ShowDialog() == DialogResult.OK)
```

```
printDocument1.Print();
}

// شفرة إعدادات الصفحة
private void Pagesetup_Click(object sender, EventArgs e)
{
pageSetupDialog1.Document = printDocument1;
          pageSetupDialog1.PrinterSettings =
printDocument1.PrinterSettings;

if (pageSetupDialog1.ShowDialog() == DialogResult.OK)
{             printDocument1.PrinterSettings =
pageSetupDialog1.PrinterSettings;
printDocument1.Print();
}
}

// printpage شفرة الحدث
private void printDocument1_PrintPage(object sender,
System.Drawing.Printing.PrintPageEventArgs e)
{
System.Drawing.Font printFont =
new Font("Arial", 25, FontStyle.Regular);

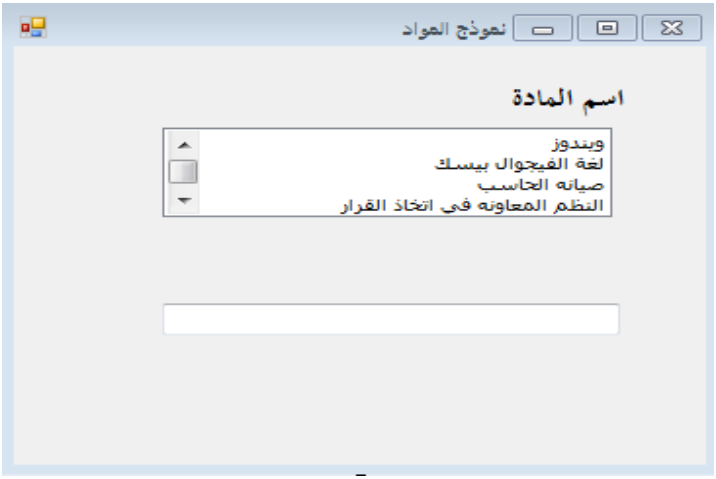
e.Graphics.DrawString(richTextBox1.Text, printFont,
Brushes.Black, 10, 10);
}
```

هذا الكود ينفذ عندما يتم الضغط على زر الطباعة.

تدريب ١:

قم بتصميم نموذج يتكون من أداة سرد باسمListBox1

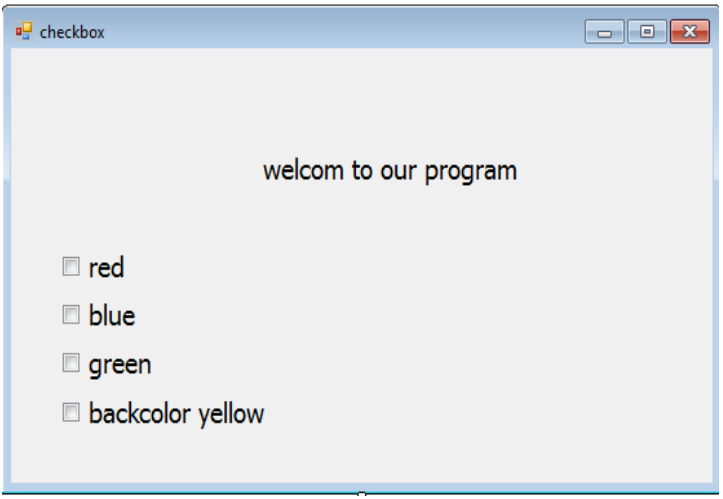
نريد عرض اسم العنصر المختار في مربع نص حينما يختاره المستخدم من القائمة.



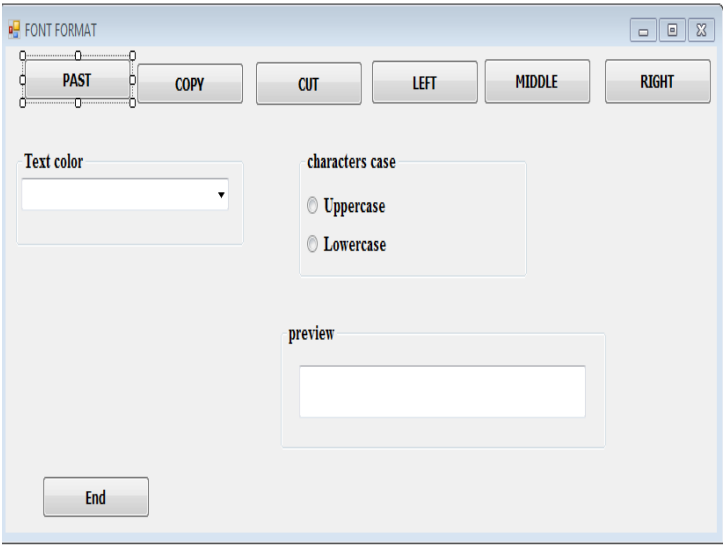
الكود:

```
private void
listBox1_SelectedIndexChanged(object
sender, EventArgs e)       {
            textBox1.Text = " " +
listBox1.SelectedItem;
      }
```

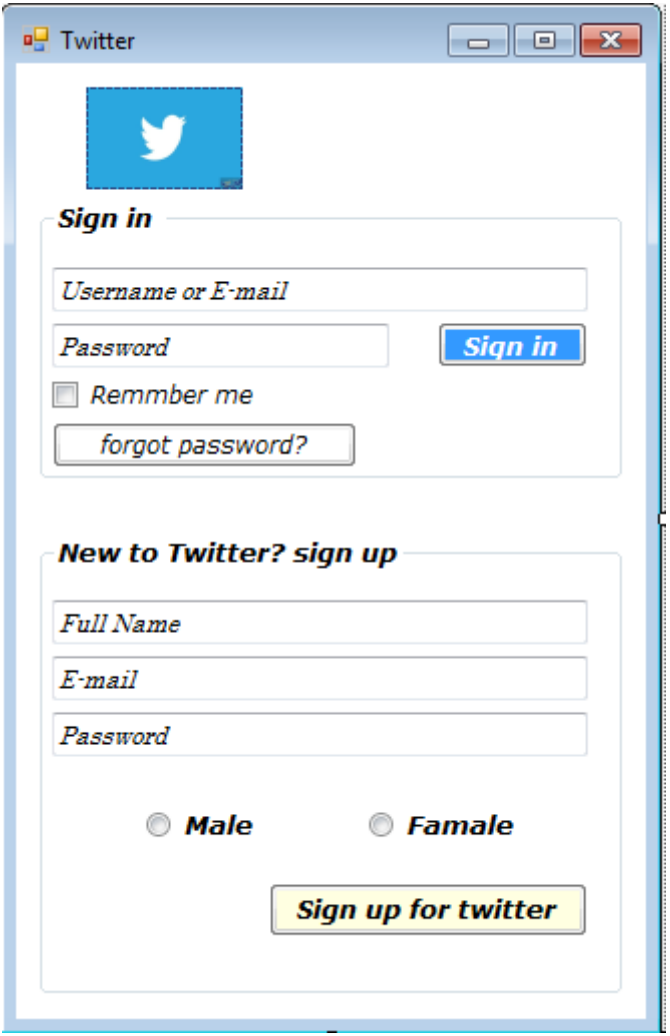
تدريب٢: قم بتصميم النموذج التالي.



تدريب ٣: قم بتصميم النموذج التالي:



تدريب ٤: صمم المشروع التالي




Welcome
مرحبا بك في تويتر
تعرف علي ما يحدث في العالم العربي الأن
عرض الصفحة الشخصية

الكود:

Form1:

```
namespace WindowsFormsApplication4{
   public partial class Form1 : Form    {
      Form2 OB = new Form2();
      public Form1()       {
                  InitializeComponent();       }

      private void button1_Click(object
sender, EventArgs e)       {
         string a = textBox1.Text;
         string b = textBox2.Text;
         if (a == "menna" && b ==
"123456789")          {
            OB.Show();
            Hide();          }
         else
```

```
            MessageBox.Show(" الاسم من التأكد رجاء
المرور وكلمة");
            textBox1.Text = "";
            textBox2.Text = "";           }

        private void button2_Click(object
sender, EventArgs e)
        {
            OB.Show();
            Hide();       }
  private void textBox2_TextChanged(object
sender, EventArgs e)
        {

        }
        private void button3_Click(object
sender, EventArgs e)
        {
```

```
            string x = " لالكترونى البريد رابط ارسال تم
الجديده المرور كلمة تعيين لاعاده بك الخاص";
            MessageBox.Show(x);        }

        private void
radioButton1_CheckedChanged(object
sender, EventArgs e)        {
            string x = textBox3.Text;
            if (radioButton1.Checked == true)
                MessageBox.Show("مرحبا " + x + " تم
بنجاح البيانات ملئ");
            else if (radioButton2.Checked ==
true)
                MessageBox.Show("مرحبا " + x + " تم
بنجاح البيانات ملئ");       }}}
Form2:
   private void button1_Click(object sender,
EventArgs e)
```

```
{
    string x = " يتم الأن الصفحة تحميل الشخصية
بك الخاصه .. انتظر دقيقة :) ";
    MessageBox.Show(x);
}
```

تدريب ٥: صمم المشروع التالي





تدريب ٦: صمم المشروع التالي

قم بتصميم النموذج التالي



المطلوب : عندما يضغط المستخدم على الزر الراتب المتوقع يقوم بحساب راتب الموظف بناء على الجدول التالي:

المؤهل	الراتب
الإعدادية	١٠٠٠
الثانوية	٢٠٠٠
الدبلوم	٢٥٠٠
مؤهل جامعي	٣٠٠٠

ويزيد الراتب على حسب المهارة المتقنه كما هو موضح بالجدول التالي:

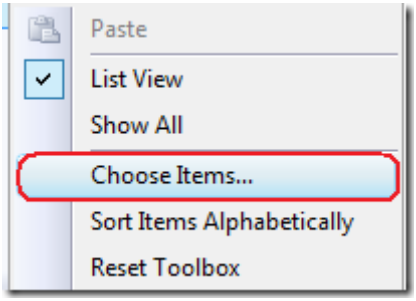
المؤهل	الراتب
اسنخدام الكمبيوتر	١٠٠
البرمجة	٢٠٠
يجيد اللغة الإنجليزية	٢٠٠
شهادات خبرة	٣٠٠

- **لتشغيل الصوت**

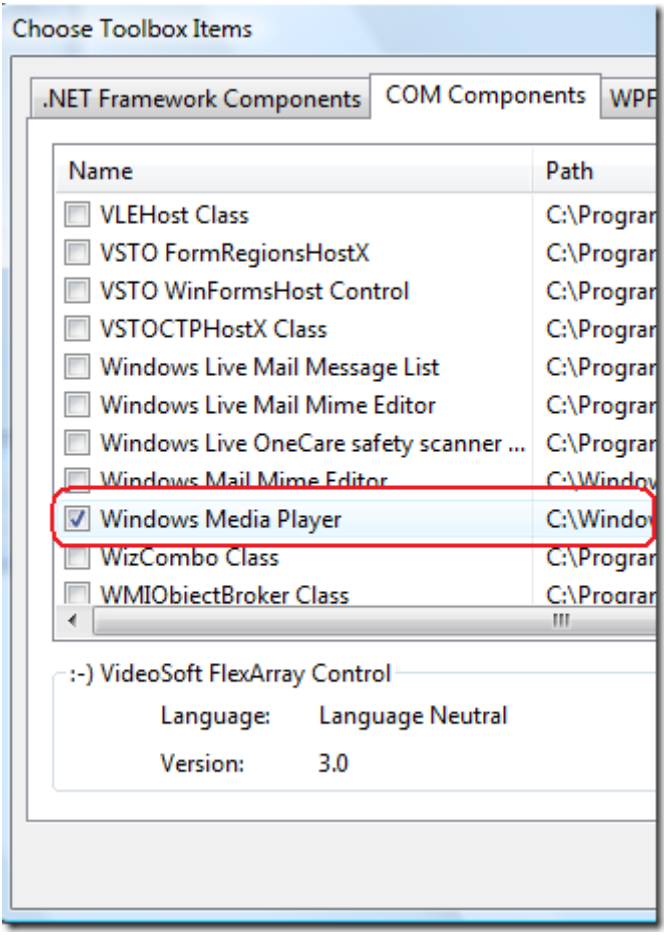
قم بفتح مشروع جديد…window application

و في البداية يجب عليك إضافة الـ Window Media Player Controlإلى صندوق الأدوات لديك وذلك كالتالي:

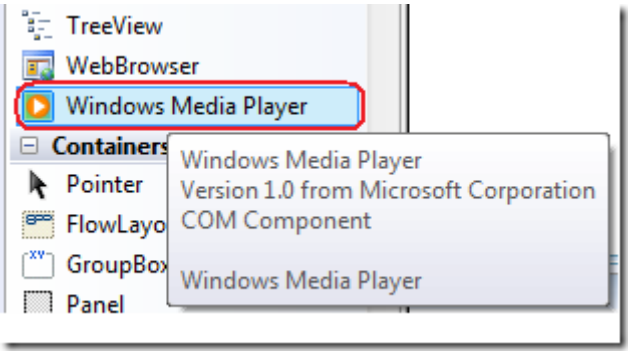
بعد النقر على الزر الأيمن في صندوق الأدوات:



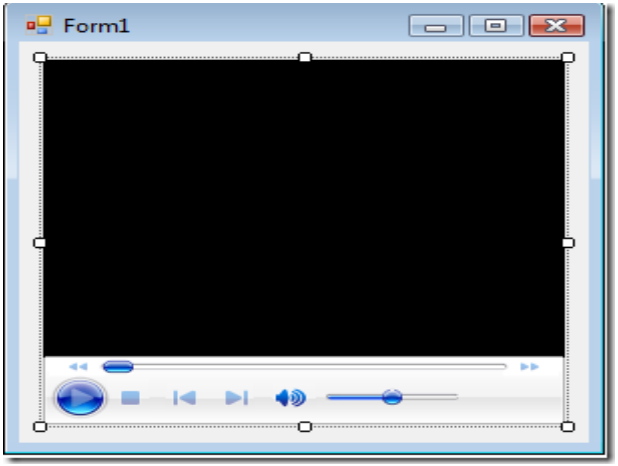
ثم اختيار choose Item : ستظهر لك نافذة منها قم بتحديد الـ ..... Window Media Player Control ثم الضغط على زر موافق.



لتظهر لك الأداة في صندوق الأدوات بعد ذلك كالتالي.



الآن لاستخدامها كل ما عليك هو سحبها كأي أداة ووضعها على الـ Form وتستطيع وضعها بالحجم الذي يناسبك انظر الشكل التالي:



ومنها تلاحظ أن الـ Window Media Player Control توفر العديد من الأزار والتي تتيح للمستخدم تشغيل الملف، إيقافه، إعادته، والتحكم في حجم الصوت وغيرها من الوظائف الهامة .

- برنامج يستطيع بواسطته المستخدم من تشغيل أي ملف من القرص ليتم عرضه على الـ Windows Media Player Control .

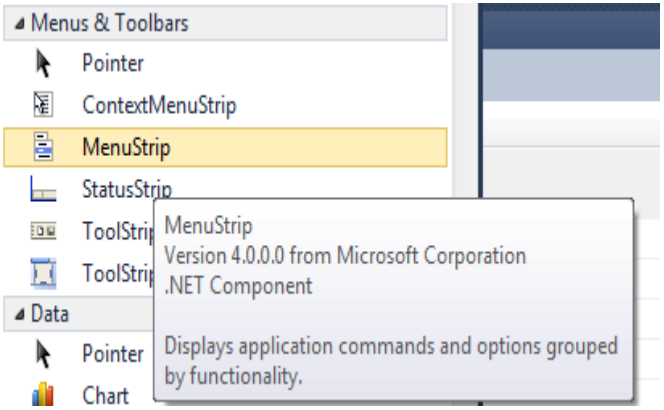
سنقوم بإضافة Main menu إلى البرنامج و الذي يتكون من القائمة:

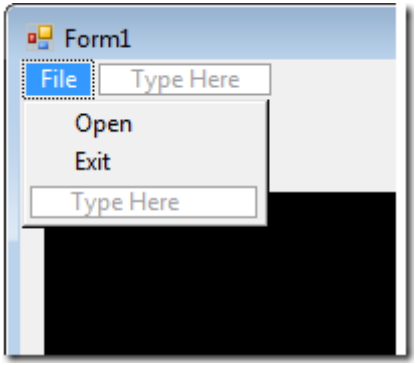
~~: File: ~~

والتي تحتوي على open وهي لاختيار أي ملف من الجهاز لكي يتم تشغيله ، Exit لإغلاق البرنامج.

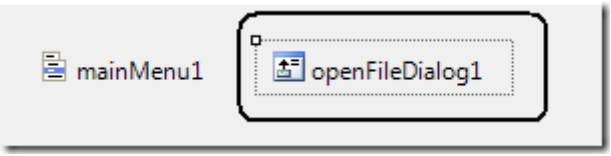
لعمل ذلك:

١.اسحب أداة الـ **MainMenu Control** من صندوق الأدوات و قم بإسقاطها على الـ form ، و ابدأ في كتابة اسم القائمة File و محتوياتها.





٢.و لكي نجعل الأمر Open يقوم بفتح نافذة open File كالتي تظهر لنا في أي برنامج من برامج الويندوز، فإننا سنستخدم الـ OpenFileDialog Component والتي تسمح للمستخدم بالبحث داخل المجلدات عن الملف الذي يريد فتحه وتشغيله، ولإضافتها إلى البرنامج قم بإضافتها مباشرة من صندوق الأدوات و إسقاطها على الـform.



لاحظ أن الـ OpenFileDialog Component لن تظهر لك على الـ form ولكنها ستظهر أسفل النافذة كما في الشكل السابق، وسنتعامل مع الـ openFileDialog داخل البرنامج على أنه Class.

و بذلك نكون قد أضفنا المكونات و الأدوات اللازمة للبرنامج، لنبدأ الآن بكتابة الأوامر: code

١- قم بالضغط على الأمر open من القائمة File مرتين .... لتنتقل إلى نافذة الـ ... code و اكتب الأمر التالي:

```
private void menuItem2_Click(object sender, EventArgs e)
{
 //Use the ShowDialog method to display the dialog at run time.
 this.openFileDialog1.ShowDialog();
 this.axWindowsMediaPlayer1.URL = this.openFileDialog1.FileName;
}
```

في البداية استخدمنا الدالة ShowDialog و التي تعمل على إظهار الـ Dialogعند الضغط على الأمر , open و يستطيع المستخدم هنا أن يبحث عن أي ملف و يختاره

this.axWindowsMediaPlayer1.URL هنا استخدمنا خاصية الـ URL للـ Windows Media Player Controlالحالي أي مسار الملف المراد تشغيله.

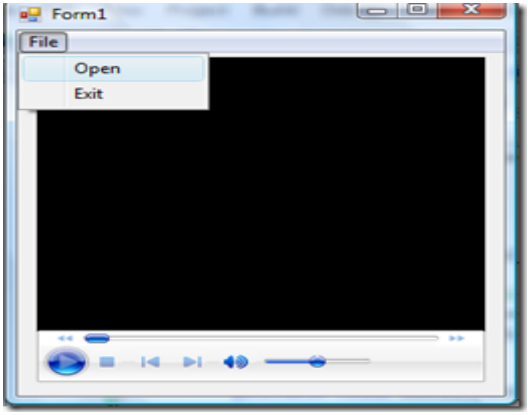
this.openFileDialog1.FileName وهنا الخاصية FileNameتعود بمسار و اسم الملف الذي قام المستخدم باختياره من النافذة open dialog

و بنهاية هذا السطر نكون قد جعلنا مسار الملف الذي سيتم تشغيله = مسار الملف الذي قام المستخدم باختياره من النافذة open dialog

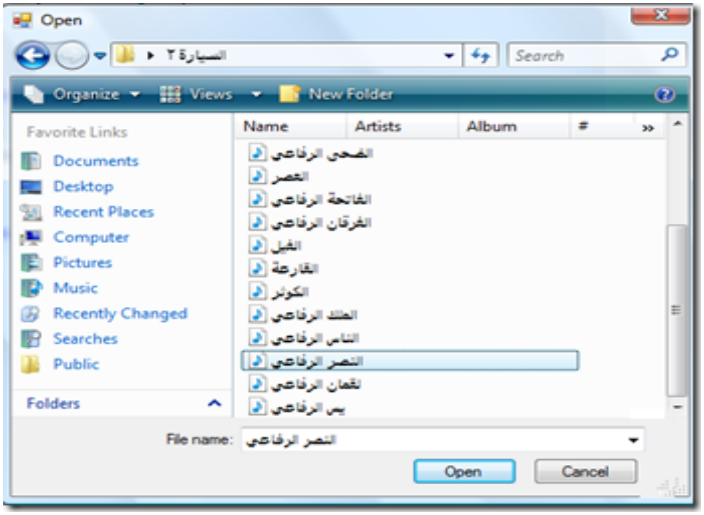
٢- قم بالضغط على الأمر Exit مرتين من القائمة File واكتب الأمر التالي لإغلاق البرنامج:

```
private void menuItem3_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

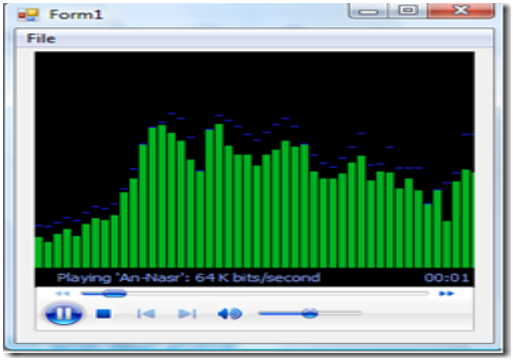
و بذلك اكتمل البرنامج و عند تشغيل البرنامج:



اختيار الملف المراد فتحه



ثم:



الفصل الخامس

التعرف الضوئي على الحروف

Optical Character Recognition

التعرف الضوئي على الحروف

التعرف الضوئي على الحروف أو ما يعرف بـ Optical Character Recognition OCR تقوم هذه التقنية بقراءة وتحويل المستندات المصورة والتي بها نصوص مكتوبة والمدخلة للكمبيوتر بالماسح الضوئي (سكانر) على سبيل المثال إلى نصوص حية يمكن تحريرها ونسخها ثم التعديل عليها، وهذه التقنية المتطورة تختصر علينا وقت إعادة كتابة المستند المصور من جديد، وهي مهمة جدا بالنسبة للذين ينجزون البحوث أو غيرها.





Before performing OCR
(Image-based PDF)



After performing OCR
(Text-based PDF)

- **أهمية التعرف الضوئي على الحروف**

١. مساعدة المكفوفين في قراءة النصوص التي لم تترجم إلى لغة بريل.

٢. إدخال، وحفظ الكتب، والمستندات داخل الحاسب بطريقة سهلة، وسريعة، من أجل معالجاتها، والبحث في محتوياتها، والتعديل عليها.

٣. القدرة على استرجاع المعلومات، واستنباط المعرفة، من الوثائق والمستندات المطبوعة.
٤. توفير الوقت، والجهد والتكلفة لعملية إدخال عدد كبير من النصوص وحفظها داخل الحاسب.
٥. استخدام وحدات الادخال المباشرة مثل القلم الضوئي، ولوحة الرسم، بديلا عن لوحة المفاتيح لكتابة المستندات.

- **أنواع التعرف الضوئي على الحروف**
  - التعرف على الخط الثابت
  - التعرف على خط اليد
  - التعرف المباشر أو المتزامن
- **التعرف الضوئي على الحروف العربية**

من المعروف أن للكتابة باللغة العربية خصائص مميزة، أوجدت الحاجة لبرامج تعرف ضوئي خاصة باللغة العربية، ومن هذه الخصائص: اتجاه الكتابة، اتصال الكتابة، أشكال الحروف في مقاطع الكلمة، حركات الضبط بالشكل، وجود علامات الترقيم، وجود تشابه كبير بين بعض الحروف.

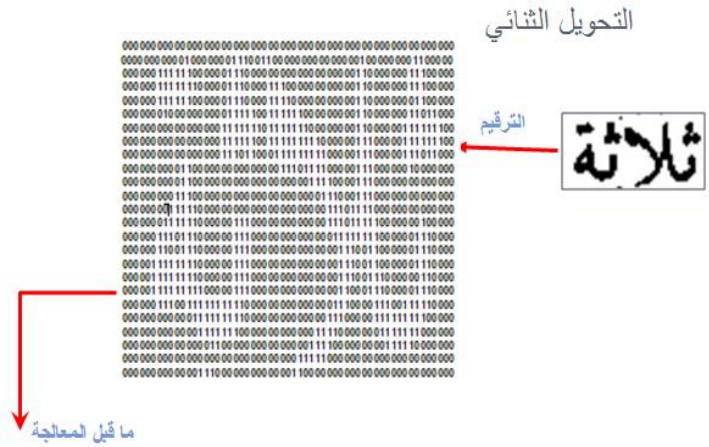
تقوم نظم التعرف الضوئي على الحروف بمجموعة من المراحل تتم على التوالي حتى تتمكن من تحويل المستند المطبوع إلى سلاسل نصية إلكترونية يستطيع الحاسب التعامل معها، ومعالجتها.

١- مرحلة الإدخال Input

يتم في هذه المرحلة إدخال المستند النصي المطبوع إلى الحاسب الآلي على هيئة صور رقمية عن طريق الماسح الضوئي Scanner.

٢- مرحلة ما قبل المعالجة Pre-processing

يتم في هذه المرحلة إعداد النص ليكون جاهزاً للمراحل الأخرى، وذلك بتنقية النص من الشوائب، وهناك العديد من تقنيات معالجة الصور.



يتم في هذه المرحلة مايلي:

- تنقية الصورة من الشوائب.
  - إزالة الحواف السوداء المحيطة بالصورة والصور المرافقة للنص.
  - إزالة أي زائد موجود في النص كالأخطاء الناتجة من التصوير.
- تصحيح الالتواء.
- تقسيم الأسطر.
- تطبيع خطوط النص.
- تقسيم الكلمات.

- استخراج النقاط.
- تنعيم هياكل الكلمات أو التنحيف.

من العوامل المؤثرة في مستوى وضوح الصورة المأخوذة عن طريق الماسح الضوئي:

١. تاريخ الأصل أو قدم الورقة.
٢. جودة الورق.
٣. طريقة الطباعة (ليزر، نقطية، آلة كاتبة).
٤. سلامة الأصل من التظليل أو التخطيط.
٥. وضوح الخط.
٦. حجم الخط.

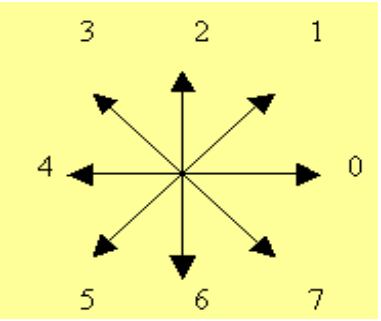
ما قبل المعالجة - تعديل الحروف

- تقدير وتصحيح ميلان الحروف وتحديد الحروف بحجم معين.
- تختلف الأنظمة في طريقة تعاملها مع الحرف العربي.

ما قبل المعالجة – التنعيم (الملاسة)

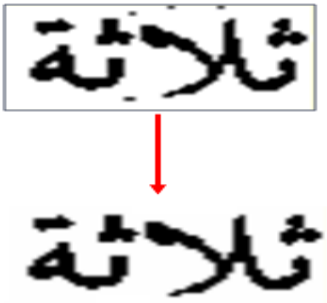
- يمكن تحديد حالة نقطة معينة (pixel) من خلال الاستفادة من النقاط الثمان المحيطة بها.

- تشمل عملية الملاسة إزالة التشويش والأخطاء بالإضافة الى عملية التنحيف.



ما قبل المعالجة – إزالة التشويش

تتضمن عملية تنقية الصورة من الشوائب إزالة جميع الحواف السوداء المحيطة بالصورة والصور المرافقة للنص كما تتضمن إزالة أي زوائد موجودة في النص كالأخطاء الناتجة من التصوير.



ما قبل المعالجة – التنحيف

ينتج عن عملية التنحيف إيجاد هيكل الحرف.

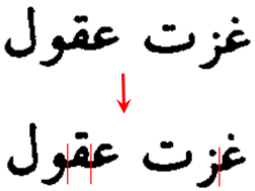


ما قبل المعالجة – الأجزاء المتصلة

تقسيم الكلمات إلى أجزاء وتقسم الكلمة نفسها الى أجزاء أصغر بحيث تكون الحروف المتصلة في جزء والمعزولة في جزء آخر.



التقطيع



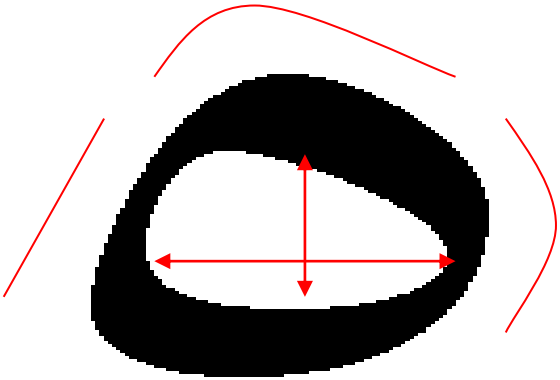
استخلاص السمات

- بعد التقطيع تستخلص الخصائص المميزة لكل شكل.
- تجمع الأشكال إلى مجموعات وتعطى كل مجموعة رمز فمثلا.

الرمز	نوع الشكل
١	حرف
٢	جزء من حرف
٣	مجموعة من حروف
٤	نقطة عليا

استخلاص الخصائص

- الخصائص التركيبية



- الخصائص الإحصائية

الطول

العرض

الكثافة في الاتجاه الرأسي

الكثافة في الاتجاه الأفقي

عدد التقاطعات في الاتجاه الرأسي

عدد التقاطعات في الاتجاه الأفقي

نسبة النقاط البيضاء إلى السوداء



التصنيف Classification

وهي المرحلة الأساسية في عملية التعرف الضوئي حيث تقوم بنسبة الحرف إلى إحدى مجموعات الحروف الممكنة. وهناك العديد من التقنيات التي تستخدم في التصنيف، منها:

- الشبكات العصبية الاصطناعية Artificial Neural Networks (ANN).
- الخوارزميات الجينية Genetic Algorithm.
- الجار الأقرب Nearest Neighbor.
- Weighted Euclidean Distance Measure.

٤- مرحلة ما بعد المعالجة Post-processing

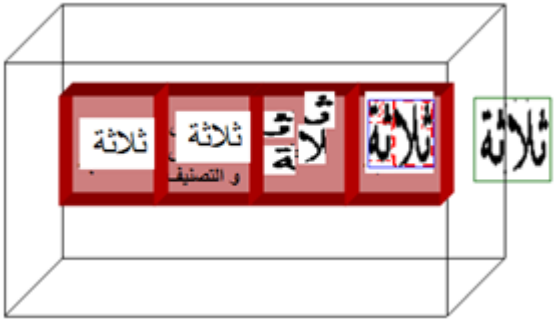
ويتم في تلك المرحلة عمليتين رئيسيتين وهما:

- **الفحص الإملائي** Spell Checking

حيث يقوم النظام باستخدام نظم التصحيح الإملائي، وذلك للتأكد من خلو السلاسل النصية التي تم الحصول عليها من الأخطاء.

- **فحص بناء الجملة Sentence Structure Checking**

حيث يقوم النظام بمراجعة التركيب المنطقي للغة، وذلك للتأكد من خلو السلاسل النصية التي تم الحصول عليها من الأخطاء النحوية.



- **معايير اختيار برمجيات OCR**

تعتمد المكتبات ومؤسسات المعلومات في اختيارها لبرمجيات التعرف الضوئي على الحروف على مدى صلاحيتها للاستخدام والتطبيق، واشتمالها على قواميس متخصصة غنية وثرية بمفردات المحتوى الموضوعي لنصوص مصادر المعلومات التي يمكن معالجتها بواسطة مثل هذه البرمجيات.ومن اهم هذه المعايير مايلي:

١. الدقة: ويمكن تقدير معدل الدقة من خلال التعرف على نسبة الأخطاء الناجمة عن إجراء القراءة الضوئية للنصوص، ويمكن قياس معدل دقة البرنامج عبر النسبة المئوية للكلمات.

٢. التطابق والتوافق مع أجهزة المسح الضوئي المتنوعة: من المهم التحقق من نوع أجهزة الماسحات الضوئية التي يمكن أن تتعامل معها برمجيات ""OCR، أي ضرورة التوافق بين جهاز الماسح الضوئي المستخدم من ناحية وبرنامج التعرف الضوئي على الحروف من ناحية أخرى، وفي هذه الحالة من الضروري تطابق

وتوافق البرنامج مع معيار " "TWAINالمستخدم من جانب غالبية الماسحات الضوئية.

٣. واجهة المستفيد: يمكن أن يخلق التصميم الشكلي الجذاب نوع من التحاور والتفاعل بين المستخدم من ناحية وبين البرنامج من ناحية أخرى، ومن بين الآليات التي تحتويها تلك البرمجيات بعض الأزرار التي تسمح عند الضغط عليها بواسطة "الفارة" القيام بالعديد من المهام، وذلك في أثناء عملية التعرف الضوئي للحروف في نص معين.

٤. القدرة على التعرف على الجداول: تعتبر إمكانية التعرف الشكلي والهيكلي للجداول خاصية مميزة لبعض فئات برمجيات ""OCR، ولهذا السبب من المفيد أن يكون البرنامج قادراً على اكتشاف وجود جداول في النص ومن ثم إعادة صياغتها في إحدى برمجيات معالجة النصوص.

٥. أشكال الحفظ والتخزين: من المعروف أن تسجيل البيانات والمعلومات المرقمنة في ملف يجب أن يكون وفقاً لشكل أو قالب أو امتداد معين وذلك بهدف إمكانية

القراءة والإطلاع عليه في وقت لاحق، ولذلك فإن برمجيات OCR تتضمن أشكالاً مخصصة لتسجيل ومن ثم حفظ واسترجاع المعلومات في الملفات، ومن أهم هذه الأشكال: doc,pdf,excel

٦. الالتزام بشكل النص الأصلي: يحقق الالتزام بشكل النص الأصلي إمكانية الحصول في صورة الكترونية على نفس التكوين الهيكلي والتنظيمي للصفحة الأصلية التي يتم رقمنتها ، إلي جانب استنساخ نفس الخصائص النصية من نوع الحرف وحجمه وشكله، وجسم النص، والأسلوب.

٧. اللغات: كلما زادت وتعددت اللغات التي يمكن التعرف عليها كلما كانت برمجيات OCR أكثر قدرة على التعرف على النصوص المكتوبة بتلك اللغات المتنوعة.

- **التعرف على الأنماط Pattern recognition**

علم التعرف على الأنماط Pattern recognition وهو مجال متفرع ومرتبط بمجالات علمية أخرى كتعليم الآلة Machine learningوإبصار الحاسب Computer

vision والذكاء الاصطناعي Artificial Intelligence وغيرها من المجالات الأخرى.

النمط هو كيان غامض التعريف يمكن إطلاق اسم عليه، فقد يكون بصمة إصبع، كلمة مكتوبة، وجه إنسان، صوت، أو حتى شريط الحمض النووي.DNA .

معنى التعرف على النمط **هو اكتشاف الفئة أو الصنف "class" الذي ينتمي إليه هذا النمط.**

بمعنى أنه عندما يكون لديك صورة لبرتقالة على سبيل المثال، وتستطيع أن تميز أن ما في الصورة ينتمي إلى فئة الفاكهة فقد قمت بعمل تصنيف classification لصورة النمط وإذا استطعت أن تعرف اسمها فقد قمت بالتعرف على النمط "recognition"

وعلم تمييز الأنماط ما هو إلا محاكاة حسابية لقدرات البشر في تمييز الأنماط لخدمة العديد من الأغراض ما بين خدمة الإنسان وتوفير وقته وجهده في الأعمال التي لا تستدعي تفكير إبداعي أو مهارات خاصة، وما بين تلافي الأخطاء البشرية في الأعمال التي تستلزم قدرًا كبيرًا من الصبر أو دقة بالغة في إتمامها.

والتطبيقات كثيرة جدًا نستخدمها كل يوم. فما بين نظم التأمين والحماية بالتعرف على بصمة الإصبع والوجه وبصمة العين لحماية المنشآت، ومطابقة التوقيع وبصمة الصوت في المعاملات المصرفية، وتطبيقات طبية كالتشخيص الذاتي للأمراض، وتطبيقات تقنية كمحركات البحث على الإنترنت واسترجاع المعلومات من المواقع الإلكترونية، وتطبيقات الفلك وتنبؤات الطقس والاستشعار عن بعد، والتطبيقات الصناعية كفرز المنتجات وتحديد العيوب، وصولًا إلى برمجيات للحاسب كالتعرف على خط اليد والحروف، وتطبيقات للهاتف للتعرف الذاتي على الصوت دون الحاجة لكتابة رسالة أو البحث في قائمة الاتصال لإجراء مكالمة.

كل ذلك ليس سحرًا! هو فقط نظام يقوم بتقطيع الإشارة الداخلة إليه كالصورة أو الصوت أو حتى مقطع الفيديو، إلى كيانات أصغر ويعتبرها أنماطًا جديدة، ويحاول التعرف عليها، ثم إعادة تركيبها كما كانت، أو يقوم باتخاذ قرار بناءً على تلك المعرفة.

هل كان لديك مرة مجموعة من الأوراق لا تخصك وتريد أن تقوم بتعديل محتواها؟ أو كتبًا تريد أن تقوم بتلخيص

محتواها؟ ماذا كنت تفعل حينها؟ في الغالب كنت تقوم بإعادة كتابتها على الحاسب حتى تستطيع تعديلها. ولكن هناك حل آخر. وهو أن تقوم بمسحها ضوئيًا على جهاز ماسح ضوئي (سكانر scanner)، الذي يقوم بإصدار ملف pdf. به صور للصفحات الممسوحة، ثم تستخدم برمجية التعرف على الحروف Optical character recognition OCR لتقوم بالتعرف على الكتابة المحتواة.

يتم ذلك ببساطة إن صورة الكتابة التي صدرت عن الماسح يتم تقطيعها بواسطة تلك البرمجية، إلى فقرات وسطور وكلمات ثم حروف. وتقوم البرمجية بالتعرف على صور الحروف وبالتالي الكلمات، فتعيد كتابتها مرة أخرى في ملف يمكن تعديله .doc بصمتك يتم مطابقتها بقاعدة بيانات من البصمات للتعرف على سجلك الجنائي. صورة وجهك يتم التعرف عليها في المطار لمطابقتها مع صورتك على جواز السفر. صور الأشعة وأرقام التحاليل يتم مقارنتها بالصور الصحيحة والمعتلة لمعرفة إصابتك بمرض ما من عدمه، أو مدى تطور المرض لديك. كلماتك التي تكتبها على محركات البحث يتم مطابقتها بالكلمات داخل الملفات، والكتب المرفوعة على الإنترنت لاسترجاع

البيانات التي تبحث عنها. صور المحاصيل الزراعية والصحراء وباطن الأرض التي يرسلها القمر الصناعي تعالج لمعرفة أماكن المعادن، ومناطق الجفاف، ومواقع آبار البترول والفحم. كذلك صور الغلاف الجوى تحلل للقيام بتنبؤات الطقس، وصور الكواكب والنجوم تستخدم لمعرفة طبيعة الكوكب وتكوينه الداخلي. سيارتك ونظام GPS لتعرف اتجاهاتك وقراءة أرقام اللوحة ذاتيًا في أماكن المبيت لحساب التكلفة، هاتفك وتطبيقاته وحتى احتياجاتك الترفيهية كالكاميرا الرقمية التي تحدد موقع الأشخاص قبل التصوير في مربعات، وحاسبك الذي يمكنه لعب الشطرنج معك.

هناك كل يوم باختراع جديد عن سيارة ذاتية القيادة، ورجل آلي يفهم تعبيرات الوجه، ويرعى المسنين، ويلعب الكرة. فكل تلك الإمكانيات تعتمد في المقام الأول على تسجيل كل أنواع الإشارات الصوتية والضوئية والصور وتحليلها ببرمجيات ذكية، للتعرف على مختلف الأنماط والتصرف بحسب ما تم تدريب (تعليم) الآلة عليه حسب كل حالة.

ولكن كيف يتم تحديدًا التعرف على الأنماط؟

من خلال ما يسمى سمات features فإذا رجعنا لمثال البرتقالة، كيف تعرف أن ما بالصورة هو برتقالة؟ ببساطة إنها دائرية ولونها برتقالي.

كيف تعرف صديقك في الصورة؟ أنت تعلم مثلًا لون عينيه، ورسم وجهه، ولون جلده، وطوله وحجمه.

كيف تقرأ كلمة "قطة"؟ أنت تعلم أن حرف القاف هو دائرة متصلة بخط عرضي وعليها نقطتان "قـ"، وأن حرف الطاء هو حلقة مستعرضة يعلوها خط طولي "ط"، وأن التاء المربوطة هي خط طولي متصل بدائرة تعلوه نقطتان "ـة". إذًا فإن قـ – ط – ـة هي "قطة"

ماذا نطلق على هذه الخصائص؟ نسميها "سمات" features.

والسمات يمكن أن تكون أي شيء: لون، طول، عرض، وزن، أو أي شيء يمكن ترجمته إلى رقم.

ولكن هناك أشياء مثل شكل الوجه مثلًا، كيف نستخرج منها سمات مميزة؟ ببساطة هناك ما يسمى بالتحويلات Transforms، وهي دوال رياضية تعبر عن أشكال المنحنيات، فيتم تحليل الصور إلى مجموعة من خطوط الرسم،

هذه الخطوط يرمز لها بتلك الدوال، ويتم اختيار نقاط محددة على هذه الخطوط، أو معاملات coefficients الدوال نفسها كـ"سمات".

الفكرة ببساطة أن قيم هذه السمات يمكن رسمها كنقاط في فراغ ثنائي أو ثلاثي الأبعاد أو أكثر من ذلك، فتجد أن الأنماط المنتمية لصنف محدد تتقارب في مواقعها في ذلك الفراغ، بينما تتباعد عن تلك التي تنتمي لصنف آخر. وهنا يأتي دور المصنِّف classifier في أن يقوم بتقسيم هذا الفراغ بحيث يتم فصل كل صنف في حيز معين منه.

معنى ذلك أنه كلما جئنا بصورة جديدة أو صوت يمكننا أن نستخرج سماته، ونرى أين يقع في ذلك الفراغ، ونستطيع أن نتعرف عليه بمعرفة موقعه هذا بالنسبة لأقسام الفراغ الخاصة بكل صنف. فإن كان لدينا مجموعة من صور الوجوه مثلًا، وقمنا باستخراج سمات كلون الجلد، وحجم العين، والأنف. وقمنا بتمثيل الصور كلها في فراغ ثلاثي الأبعاد، تمثل كل صورة نقطة في ذلك الفراغ، فإنه يمكن للمصنّف أن يقوم بتقسيم هذا الفراغ إلى: وجوه آسيوية، ووجوه أفريقية، ووجوه أوروبية. كل قسم تتقارب نقاطه نظرًا لتقارب القيم الرقمية للسمات، بينما

تتباعد عن القسم الآخر. وإذا حصلنا على صورة جديدة، نستطيع ببساطة معرفة إلى أي صنف ينتمي صاحب الصورة، نظرًا لموقع سمات الصورة في الفراغ الذي قام المصنّف بتقسيمه.

هذا المصنف ما هو إلا حل رياضي، بعدما قمنا بتحويل المشكلة الحقيقة إلى تمثيل رياضي، فأصبح الهدف المباشر هو حساب دوال رياضية، وسلاسل متعددة الحدود polynomials بدرجات مختلفة؛ لنستطيع تقسيم الفراغ الخاص بتمثيل المشكلة. نفترضه في البداية، ثم نقوم بتعليمه وتشكيله على أمثلة محلولة فيصبح جاهزًا للاستخدام على عينات حقيقية مجهولة، والنتيجة مذهلة حقًا.

هذه المصنفات أشهرها على الإطلاق الشبكات العصبية neural networks بأنواعها، وسلاسل ماركوف المخفية Hidden Markov models، وماكينات دعم الموجهات Support vector machines.

هذا العلم على الرغم من بساطة منطقه وفكرته إلا أن تطبيقاته المذهلة لا تتوقف. وعلى الرغم من صعوبة مشكلاته التقنية، إلا

أن القدرات التي يمنحها لتغيير حياة البشر، تستحق ذلك العناء. والمتعمق في ذلك العلم يقف عاجزًا أحيانًا أمام مشكلة تقنية توازي مهارة بسيطة للغاية، يستطيع طفل على فطرة الله أن يقوم بها بمنتهى السهولة، بينما تستغرقه أيامًا وأسابيع لكي يستطيع أن يحاكيه في مهارته.

تحويل مستندات ممسوحة ضوئيًا إلى نص قابل للتحرير

يمكنك مسح مستند ضوئيًا وتحويل النص الذي يحتويه هذا المستند إلى بيانات يمكنك تحريرها باستخدام أحد برامج معالجة الكلمات. تسمى هذه العملية OCR التعرف الضوئي على الأحرف. لإجراء المسح الضوئي واستخدام OCR، فإنك تحتاج إلى استخدام أحد برامج OCR ، مثل برنامج.ABBYY FineReader

لا يستطيع برنامج OCR التعرف على الأحرف المكتوبة بخط اليد، كما يصعب على برنامج OCR التعرف على بعض أنواع المستندات أو النصوص، مثل:

- العناصر التي تم نسخها من نُسخ أخرى
- الفاكسات

- النص الذي يشتمل على مسافات صغيرة بين الأحرف أو الأسطر
- النصوص الموجودة في جداول أو المرسوم تحتها خط
- الخطوط المتصلة أو المائلة.

المراجع

المراجع العربية:

١- رامي القضمالي:" الخوارزميات وبنى المعطيات باستخدام visual basic،سورية .حلب،٢٠٠١،ط١.

٢- مجدي محمد ابو العطا:"المرجع الأساسي لمستخدمي#c"،سلسلة العربية لعلوم الحاسب،٢٠٠٢،ط١.

٣- بيل إفجن جيسون بيريس وآخرون، ترجمة خالد العمري:" فيجوال بيسك دوت نت "، دار الفاروق، ٢٠٠٣.

٤- عذب محمد عذب :" تعلم بسهولة فيجوال بيسك Visual Basic .Net 2010" ، دار الكتب العلمية للنشر والتوزيع،٢٠١١.

٥- عذب محمد عذب :" موسوعة لغات البرمجة Visual Basic.Net 2005"، دار الكتب العلمية للنشر والتوزيع، القاهرة، ط١، ٢٠٠٧.

٦- محمد حمدي غانم:" من الصفر إلى الأحتراف (برمجة إطار العمل) فى الفيجوال بيسك دوت نت٢٠١٠ "، مكتبة دار المعرفة، ٢٠١١.

٧- مجدي محمد أبو العطا:" المرجع الأساسي لمستخدمي visual basic 6 "، القاهرة، العربية لعلوم الحاسب، ط١، ٢٠٠٣.

٨- ميشيل هالفرسون، ترجمة مروان المفلحي:" فيجوال بيسك ٢٠٠٨خطوة بخطوة"، صنعاء.

Available on line at:

http//:vb4arab.com/vb

المراجع الأجنبية :

1- Arnson,R. , et. al :"Visual Basic How-To", Wait Group, Inc , 1992.

2- Wakefield, C., et. al: " VB.NE Developer's Guide ", United States :Group West, 2001.

3- Kifer,M. :"Programming With Logic And Objects" ,Stony Brook University,2002.

4- Mauer,L.:" More Visual Basic .NET in21 Days" ,Sams Publishing, United States of America ,2002.available on line at:file:///C:/Users/GOGO/Downloads/Sams%20Teach%20Yourself%20More%20Visua

l%20Basic%20.NET%20in%2021%20Days.pdf

5- Utley,C. "A Programmer's Introduction to Visual Basic.NET", Sams Publishing,2001.available on line at: http://www.interplat.com/vbnet.pdf

6- Mkaatr:" VB.NET Programming",2010.available on line at: http://mkasoft.com/downloads/VB.NET%20programming.pdf

7- Dan Jurafsky:" Basic Text Processing Regular Expressions",Available on line at: https://web.stanford.edu/class/cs124/lec/textprocessingboth.pdf

8- Mark Allen Weiss, "Data Structures and Algorithms in C++", Addison Wesley, 2003.

9- Adam Drozdek, "Data Structures and Algorithms in C++," Brooks and Cole, 2001.

تم بحمد الله تعالى

دار الكتب والوثائق القومية

رقم الإيداع

٢٠١٧/ ٩٥٢٠